

CODING IN JAVA



```
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True
```

```
#selection at the end -add  
mirror_ob.select= 1  
mirror_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier  
mirror_ob.select = 0  
= bpy.context.selected_obj  
data.objects[one.name].select  
print("please select exactly
```

--- OPERATOR CLASSES ---

```
types.Operator):
```

```
on X mirror to the selected  
object.mirror_mirror_x"  
mirror X"
```

SHALINI SHEKHAR

CODING IN JAVA



```
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y"  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z"  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True
```

```
selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob))  
mirror_ob.select = 0  
bpy.context.selected_objects  
data.objects[one.name].select  
print("please select exactly one")
```

--- OPERATOR CLASSES ---

```
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
error X"
```

SHALINI SHEKHAR

Coding In Java

Coding In Java



Shalini Shekhar

BookMedia Publishing House
Pondicherry, 605001, India



Copyright © 2020 BookMedia
ISBN: 978-81-948299-4-2

Coding In Java
Shalini Shekhar

All rights reserved. No part of this book may be reproduced or used in any manner without written permission of the copyright owner except for the use of quotations in a book review.

Published by BookMedia
www.bookmedia.co
info@bookmedia.co

DEDICATED TO

My Husband

Amit Shekhar

For your faith, love and support. I would never have made it without you.

My Parents

Shri R. K. Verma & Smt. Renuka Verma

for being the first teachers of my life.

My children

Twisha and Suvigya, for their unconditional love.

Acknowledgements

I would like to express my special thanks to Mr. **Surya Bhushan Kumar** , Assistant Professor at the Department of Computer Applications, K. B. Women's College, Hazaribag, for his aid and support. His review and valuable suggestions were of great help throughout the writing of this book.

I would also like to extend my appreciation to Mr. **Santosh Kumar Srivastava** , Assistant Professor, Department of Computer Applications, Vinoba Bhave University, Hazaribag and Mr. **Baleshwar Yadav** , Head of Department, Department of Computer Applications, St. Columba's College, Hazaribag.

Most importantly, Thank you almighty God for everything!

Contents

- [1. Introduction](#)
- [2. Java Program execution structure](#)
- [3. Java Components](#)
- [4. Features of Java](#)
- [5. Installing Java](#)
- [6. Setting up the path for Windows](#)
- [7. Objects and classes](#)
- [8. Java Keywords](#)
- [9. Simple Java Programs](#)
- [10. Variables in Java](#)
- [11. Data Types in Java](#)
- [12. Typecasting](#)
- [13. Operators](#)
- [14. Operator Precedence and Associativity](#)
- [15. Control Flow Statements](#)

- [16. Methods](#)
- [17. Constructor](#)
- [18. Java Polymorphism](#)
- [19. Method Overloading](#)
- [20. Method Overloading and Type Promotion](#)
- [21. Inheritance](#)
- [22. Method Overriding](#)
- [23. Super Keyword](#)
- [24. Final](#)
- [25. Abstract Class](#)
- [26. Abstract Method](#)
- [27. Interface](#)
- [28. Arrays](#)
- [29. Java ArrayList](#)
- [30. Wrapper Class](#)
- [31. Strings](#)
- [32. Java Package](#)
- [33. Access Modifiers in Java](#)
- [34. Exception handling in Java](#)
- [35. Throws](#)
- [36. Thread](#)
- [37. Java Input/Output](#)

38. Database Management

39. Java AWR

40. Event Handling

41. Applet Programming

Program List

- [1. To print a statement](#)
- [2. To illustrate variable in java program](#)
- [3. To illustrate variable that stores a constant value](#)
- [4. To illustrate byte data type](#)
- [5. To illustrate short data type](#)
- [6. To illustrate int data type](#)
- [7. To illustrate long data type](#)
- [8. To illustrate float data type](#)
- [9. To illustrate double data type](#)
- [10. To illustrate char data type](#)
- [11. To illustrate Boolean data type](#)
- [12. To illustrate Widening Casting.\(double into an int\)](#)
- [13. To illustrate Narrow Casting.\(double into an int\)](#)
- [14. To add two given numbers](#)
- [15. To illustrate arithmetic operations](#)

- [16. Accept numbers from user and then add](#)
- [17. To read a character from user and print its ASCII value](#)
- [18. To illustrate Increment and Decrement Operations](#)
- [19. To illustrate Relational Operations](#)
- [20. To illustrate Logical operations](#)
- [21. To illustrate assignment operations](#)
- [22. To illustrate Conditional operations](#)
- [23. Accept three number and find the largest number](#)
- [24. To illustrate BitWise operations](#)
- [25. To illustrate Shift operations](#)
- [26. To illustrate instance of Operation](#)
- [27. To illustrate Operator Precedence](#)
- [28. To illustrate Simple if](#)
- [29. To check whether the given number is even or odd](#)
- [30. To check for the leap year](#)
- [31. To check largest of three numbers using if-else-if ladder](#)
- [32. To check whether the given number is positive or negative](#)
- [33. To illustrate Switch Case](#)
- [34. To make simple calculator using switch case](#)
- [35. To Check whether the given character is vowel or consonant](#)
- [36. To convert Fahrenheit to Celsius and vice versa](#)
- [37. To find number of days in a month of a particular year](#)

38. To illustrate for loop
39. To check whether the given number is prime or not
40. Accept a number and calculate cube of all the numbers upto that number
41. To print Floyd's triangle
42. To accept a number from user and calculate factorial
43. To generate Fibonacci series
44. To print multiplication table of a number
45. To print factors of a number
46. To find frequency(occurrence) of character in a word/ sentence
47. To print pyramid
48. To print right angle triangle star pattern
49. To print diamond shape pattern
50. To print reverse pyramid
51. To print Pascal's triangle number program
52. To print simple number pattern
53. To display alphabets rhombus structure
54. To illustrate while loop
55. Accept a number from user and reverse it
56. To check whether the given number is Palindrome or not
57. To check whether the given number is Armstrong number or not
58. To illustrate addition of two binary numbers

- [59. To illustrate do-while-loop](#)
- [60. To calculate HCF and LCM of two numbers](#)
- [61. To illustrate break keyword in Java](#)
- [62. To illustrate continue keyword in Java](#)
- [63. To illustrate Labelled Break](#)
- [64. To illustrate Labelled continue](#)
- [65. To convert binary number to decimal](#)
- [66. To convert octal to decimal](#)
- [67. To convert hexadecimal to decimal](#)
- [68. To illustrate conversion of decimal number to other number system](#)
- [69. To get current date and time](#)
- [70. To illustrate method with parameter](#)
- [71. To calculate power of a number](#)
- [72. To calculate area of a circle](#)
- [73. Accept marks from user and calculate CGPA](#)
- [74. To calculate product of two binary numbers](#)
- [75. To illustrate Instance method](#)
- [76. To illustrate Constructor](#)
- [77. To illustrate parameterized \(with parameters\) constructor](#)
- [78. Method overloading with different number of parameters](#)
- [79. Method overloading with different data types of parameters](#)

- [80. To illustrate Method overloading with Type Promotion](#)
- [81. To illustrate single inheritance](#)
- [82. To illustrate multilevel inheritance](#)
- [83. To illustrate hierarchical inheritance](#)
- [84. To illustrate method overriding](#)
- [85. To illustrate simple overridden program without super keyword](#)
- [86. To illustrate overridden program with super keyword](#)
- [87. To illustrate the program when super class and subclass have same data members without super keyword](#)
- [88. To illustrate the program when super class and subclass have same data members with super keyword](#)
- [89. To access super\(parent\) class constructors where super\(\) is implicitly provided by compiler](#)
- [90. To access super\(parent\) class constructors using where super\(\) explicitly.](#)
- [91. To illustrate final keyword with variables](#)
- [92. To illustrate final keyword with method](#)
- [93. To illustrate final keyword with Class](#)
- [94. To illustrate abstract class](#)
- [95. To illustrate abstract class with abstract method](#)
- [96. To illustrate interface](#)
- [97. To illustrate multiple interfaces](#)
- [98. To illustrate declaration & initialisation of single dimensional arrays](#)

- [99. To accept the array elements from user and print](#)
- [100. To sort an array elements in ascending order](#)
- [101. To sort an array elements in ascending order using methods of Array class](#)
- [102. To accept the elements of an array from user and reverse](#)
- [103. To print sum of all the elements of an array.](#)
- [104. To check for the elements of array whether they are even or odd](#)
- [105. To print the largest element of the given array.](#)
- [106. To search for an element in an array.](#)
- [107. To insert an element at any specific position of the array.](#)
- [108. To concatenate two arrays](#)
- [109. To concatenate two arrays using arraycopy\(\) method](#)
- [110. To illustrate declaration & initialisation of multi dimensional arrays](#)
- [111. To illustrate addition of two matrices](#)
- [112. To illustrate subtraction of two matrices](#)
- [113. To illustrate multiplication of two matrices](#)
- [114. To check whether the matrices are equal](#)
- [115. To transpose matrix](#)
- [116. To illustrate add\(\) method of Java ArrayList String type](#)
- [117. To illustrate add\(\) method of Java ArrayList Integer type](#)
- [118. To illustrate size\(\) and get\(\) method of Java ArrayList](#)
- [119. To illustrate remove\(\) & clear\(\) methods of Java ArrayList](#)

- [120. To illustrate contains\(\) methods of Java ArrayList](#)
- [121. To illustrate set\(\) and isEmpty\(\) methods of Java ArrayList](#)
- [122. To illustrate iteration through ArrayList using the Iteration interface](#)
- [123. To illustrate iteration through ArrayList using for loop](#)
- [124. To illustrate iteration through ArrayList using for each loop](#)
- [125. To illustrate iteration through ArrayList using while loop](#)
- [126. To illustrate Iteration.remove\(\) method of ArrayList in Java](#)
- [127. To illustrate addAll\(\) method to join two ArrayList in one ArrayList](#)
- [128. To sort elements of ArrayList String type](#)
- [129. To sort elements of ArrayList integer type](#)
- [130. To convert primitive data type to Wrapper object](#)
- [131. To convert Wrapper class object to primitive data type](#)
- [132. To check for the length and total number of characters in a given string](#)
- [133. To count total number of vowel and consonant in a given string](#)
- [134. To capitalize the first letter of each word in a sentence\(Title Case\)](#)
- [135. To reverse a given string](#)
- [136. To concatenate two strings](#)
- [137. To find duplicate characters in a string](#)
- [138. Comparison of two strings](#)
- [139. Replacing old character with new character](#)
- [140. To check for string.palindrome](#)

- [141. Accessing Scanner class of Java API package using import keyword](#)
- [142. Accessing Scanner class of Java API package using fully qualified class name](#)
- [143. Program Without Private access modifier](#)
- [144. Program with Private access modifier added](#)
- [145. Without Default access modifier](#)
- [146. With Default access modifier](#)
- [147. To illustrate Protected access modifier](#)
- [148. Accessing without inheritance](#)
- [149. Accessing using inheritance](#)
- [150. To illustrate Public access modifier](#)
- [151. Program without exceptional handling](#)
- [152. Exceptional handling using try-catch keyword](#)
- [153. To illustrate NullPointerException](#)
- [154. To illustrate ArrayIndexOutOfBoundsException](#)
- [155. To illustrate multiple catch block](#)
- [156. To illustrate nested try block](#)
- [157. To illustrate finally block when exception doesn't occur](#)
- [158. To illustrate finally block when exception occur and handled](#)
- [159. To illustrate finally block when exception occur and not handled](#)
- [160. To illustrate throw keyword](#)
- [161. To illustrate throws keyword in Java](#)

- [162. To illustrate thread creating by extending Thread class](#)
- [163. Another example of creating threads by extending Thread class](#)
- [164. To illustrate Thread creation by implementing Runnable interface](#)
- [165. To illustrate Thread priority](#)
- [166. To illustrate isAlive\(\) method of class Thread](#)
- [167. Program without join\(\) method of class Thread](#)
- [168. Program with join\(\) method of class Thread](#)
- [169. To illustrate yield\(\) method](#)
- [170. To illustrate stop\(\) method in java thread](#)
- [171. To illustrate read\(\) method of InputStream](#)
- [172. To read all characters of any text file \(testfile.txt\)](#)
- [173. To illustrate available\(\) method of InputStream](#)
- [174. To illustrate skip\(\) method of InputStream](#)
- [175. To illustrate write\(\) method of OutputStream](#)
- [176. To add string to a file](#)
- [177. To read data from testfile.txt using BufferedReader class](#)
- [178. To illustrate Writer using the FileWriter class](#)
- [179. To illustrate RandomAccessFile class methods](#)
- [180. To illustrate createNewFile\(\) method of File class](#)
- [181. To illustrate write\(\) method of FileWriter class](#)
- [182. To illustrate read\(\) method of FileReader class](#)
- [183. To illustrate delete\(\) method of Java File class](#)

- [184. To illustrate rename\(\) method of Java file class](#)
- [185. To illustrate few methods for getting information about a file](#)
- [186. To illustrate list\(\) method](#)
- [187. To insert a record in to the table Student using java environment](#)
- [188. To view record in to the table Student using java environment](#)
- [189. Java AWT button with ActionListener](#)
- [190. To illustrate an Applet program to set font, background and foreground colour](#)
- [191. To insert image in an Applet](#)
- [192. An applet program to add two numbers with layout setting](#)
- [193. Program to illustrate parameter passing in Applet program](#)
- [194. To Calculate EMI on a given loan amount](#)
- [195. To encrypt a plain text into Caesar cipher encryption technique](#)
- [196. To check for valid 10-digit ISBN number](#)
- [197. To Print calendar of given month and year](#)
- [198. To find the number of denominations for a given amount](#)
- [199. To calculate simple interest and compound interest](#)
- [200. To calculate Depreciation](#)

Preface

Java is the most popular programming language used to develop desktop GUI applications, web applications, mobile platforms and games for android apps and many more. According to Tiobe index, Java holds the 1st position in the top programming languages among developers from 2017 to 2019 (Feb.) Java is celebrating its 24th birthday this year and Oracle says, more than 3 billion devices run on Java. Learning java can open plenty of career doors.

The most important part of learning any programming language is to write good codes and follow the best practices .

The more you practice the more you learn any language. This is the premise behind writing this book.

About the book

My book “Coding in Java” covers every minute detail of learning java programming and is thus designed to make it particularly easy for students with little or no background of java programming. Every topic of Java

Programming is not only decoded but practically implemented with programs explaining its application. This book consists of 200 java programs with its running codes and output to learn and practice. All programs are successfully executed and discussed with the reference of output.

Both beginners and intermediate programmers can be benefitted from this book. Although the primary consideration are college students who are pursuing computer science at the UG and PG level or anyone who is interested in learning Java Programming.

Java Programming

1. Introduction

Java is an open source and one of the most popular high level programming language which is easy to learn. Java works on different platform (Windows, Mac, Linux etc.) thus support the concept of “*write once, run anywhere*”. Java is both interpreted and compiled. The java program can be compiled on any platform having a Java compiler. The resulting bytecode (compiled java program) can be run on any hardware platform having Java Virtual Machine(JVM) installed on it.

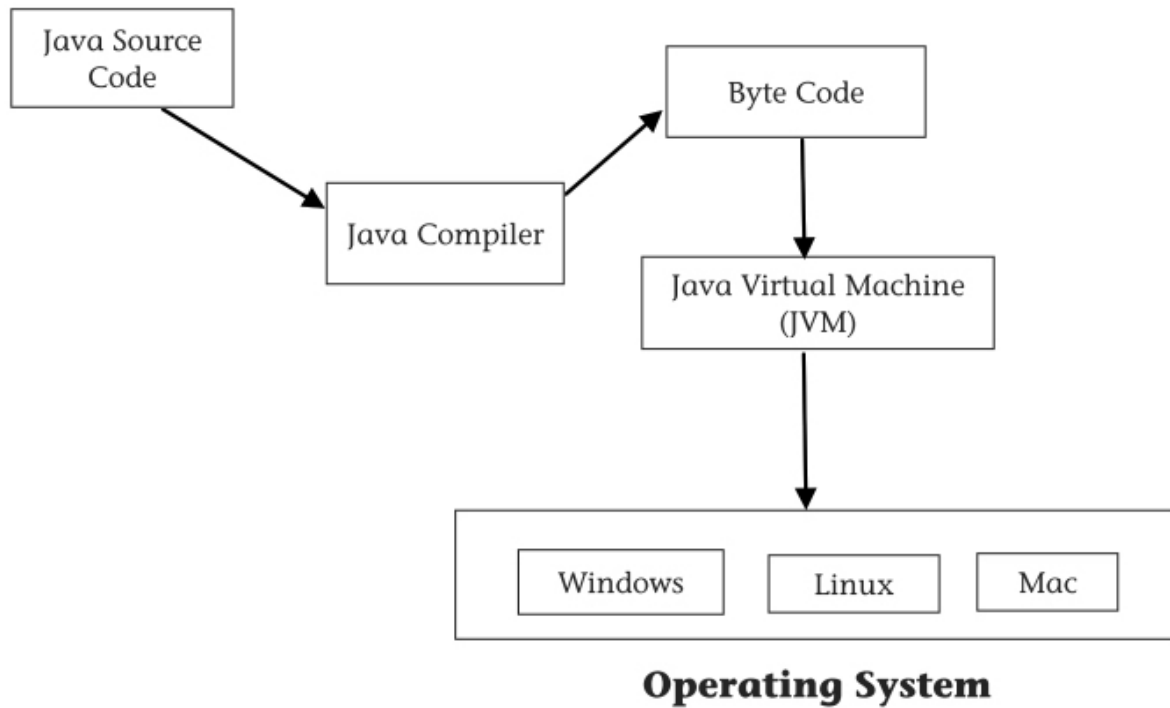
Java was developed by James Gosling along with his team at Sun Microsystems (acquired by Oracle now)in the year 1995. The language initially called “Oak” after an oak tree and ended up later being renamed as Java.

The following definition of Java by Sun Microsystems lists all the features of Java.

“Java is a simple, object oriented, distributed, interpreted, robust, secure architecture neural, portable, high performance, multithreaded and dynamic language.”

Java is commonly used to develop robotics, games, mobile apps, web apps, mobile operating system, embedded systems and much more. There is hardly anything that can't be done using Java.

2. Java Program execution structure



3. Java Components

1. Java Virtual Machine (JVM)

It is a virtual machine which is platform dependent that provides an environment for executing Java bytecodes. JVM interprets the bytecode and convert it into machine code. The main functions of JVM includes

- Loading the code
- Verifying the code
- Executing the code
- Runtime environment.

2. Java Runtime Environment (JRE)

JRE is a software package which provides the runtime environment in which JVM runs. JRE also contains set of libraries classes and the supporting files necessary to run a Java application.

$$\text{JVM} + \text{Library classes} = \text{JRE}$$

3. Java Development Kit (JDK)

JRE is a part of JDK. Along with Java Runtime Environment JDK also contain various development tools such as appletviewer, java interpreter(java), java compiler (javac), java debugger (jdb) and many more powerful tools for building any Java application.

4. Features of Java

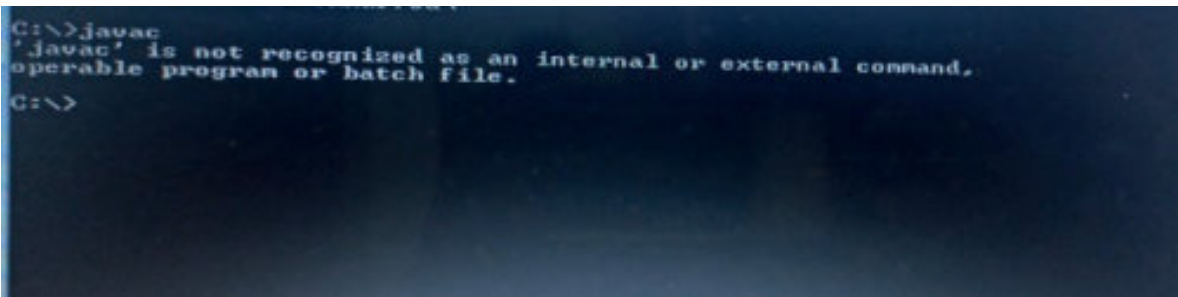
- **Simple** : Java codes are easy to read, learn and write. Its syntax are easy to understand.
- **Object-Oriented** : Java is purely object oriented and thus provides all OOP features such as abstraction, encapsulation, inheritance and polymorphism. Even the most basic program has a class. Any code in java is written inside a class.
- **Distributed** : Java is designed for the distributed environment of the internet. Java programs can access data across the web as easily as they can do in local system.
- **Compiled and Interpreted** : Java programs are both compiled and interpreted. Java codes is first compiled to bytecode (which can execute on any platform) and then interpreted when java programs are executed.
- **Robust** : Java enables you to handle unexpected errors thus ensuring reliability and bug free programming. It offers automatic garbage collection, handles memory management and different exception handling mechanisms.
- **Secure** : Java ensures security. It provides many safeguard for data against data thieves and uses its own internal memory (instead of pointers) and data management systems to strict any unauthorized data access.
- **High Performance** : Java is faster than other interpreter-based languages.
- **Platform-Intependent and Portable** : Code longevity and portability was a prime concerned in the development of Java language. Java is a platform

independent language that simply means that once java programs are compiled(bytecode), it can run on any platform. “Write once, runs anywhere” is the motto of Java language.

- **Multithreaded** : Java supports multithreaded programming, which allow multiple parts of a program to run simultaneously.
- **Dynamic** : It maintain different versions of an application very easily. It supports dynamic loading of classes.

5. Installing Java

- To check whether Java is installed in your system. Open the command prompt and simply type **javac** and press enter.
- If Java is not installed, following message will be displayed.



```
C:\>javac
'javac' is not recognized as an internal or external command,
operable program or batch file.
C:\>
```

- If Java is installed, the following information will be displayed.

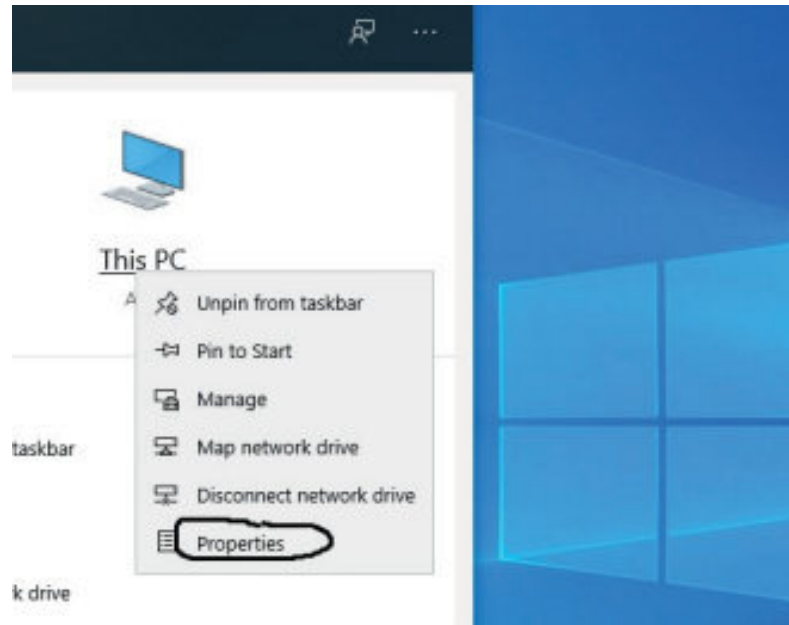
```

C:\> javac -h %filename%
Usage: javac [options] {source files}
where possible options include:
  -g               Generate all debugging info
  -g:none          Generate no debugging info
  -g:{lines,vars,source}  Generate only some debugging info
  -nowarn          Generate no warnings
  -verbose         Output messages about what the compiler is doing
  -deprecation     Output source locations where deprecated APIs are used
  -classpath {path} Specify where to find user class files and annotation processors
  -cp {path}       Specify where to find user class files and annotation processors
  -sourcepath {path} Specify where to find input source files
  -bootclasspath {path} Override location of bootstrap class files
  -extdirs {dirs}   Override location of installed extensions
  -endorseddirs {dirs} Override location of endorsed standards path
  -proc:{none,only} Control whether annotation processing and/or compilation is done.
  -processor {class}[,{class},...] Names of the annotation processors to run; bypasses default discovery process
  -processorpath {path} Specify where to find annotation processors
  -parameters      Generate metadata for reflection on method parameters
  -d {directory}   Specify where to place generated class files
  -s {directory}   Specify where to place generated source files
  -h {directory}   Specify where to place generated native header files
  -implicit:{none,class} Specify whether or not to generate class files for implicitly referenced files
  -encoding {encoding} Specify character encoding used by source files
  -source {release} Provide source compatibility with specified release
  -target {release} Generate class files for specific VM version
  -profile {profile} Check that API used is available in the specified profile
  -version         Version information
  -help           Print a synopsis of standard options
  -Xkey[=value]   Options to pass to annotation processors
  -X             Print a synopsis of nonstandard options
  -X(flag)        Pass <flag> directly to the runtime system
  -Werror         Terminate compilation if warnings occur
  @{filename}     Read options and filenames from file

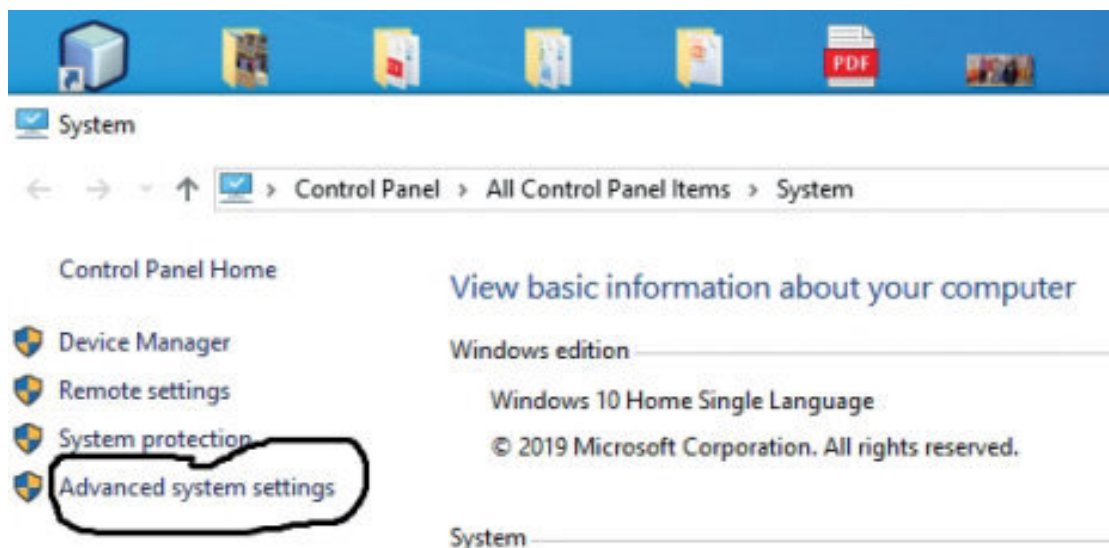
```

6. Setting Up the Path for Windows

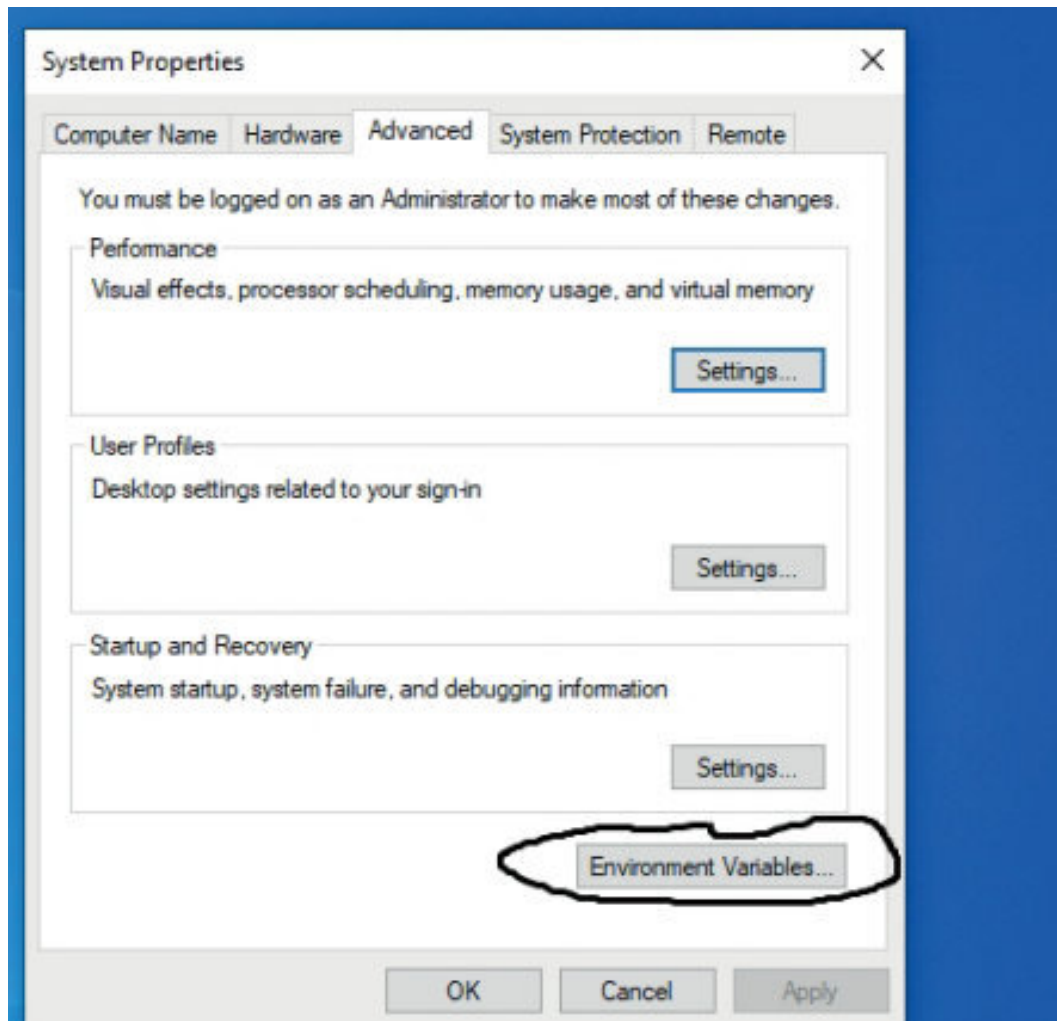
1. Right click on **My Computer/This PC** and select properties.



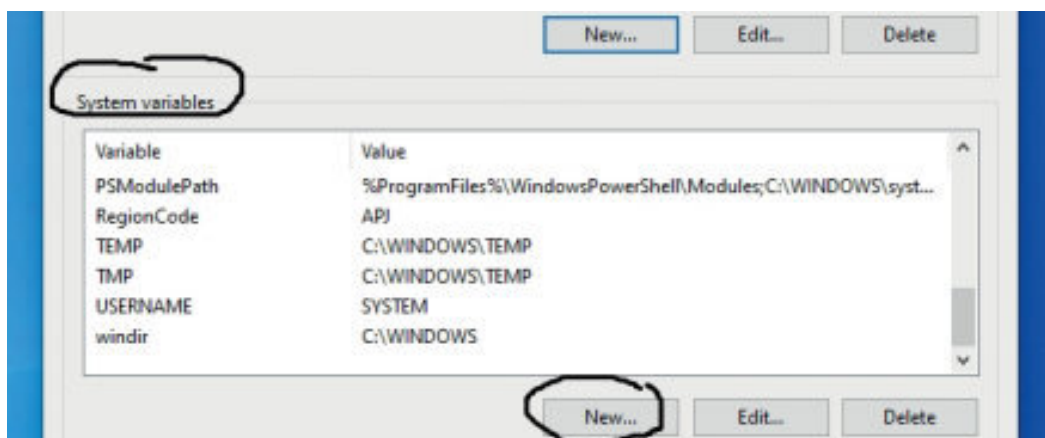
2. Select **Advanced system settings**



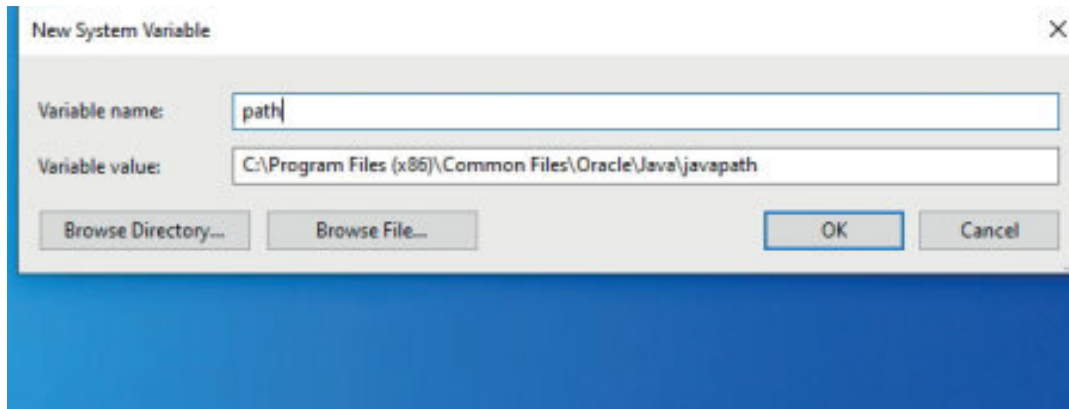
3. Click on **Environment Variables** tab.



4. In the System Variables, click **New**



5 Specify the value of the PATH environment variable and click **OK**



6. Close all windows by clicking **OK** .

To write Java programs, we need a text editor. Most commonly used text editors are Notepad (text editor for Microsoft Windows), Netbeans and Eclipse. Netbeans and Eclipse are open source Integrated Development Environment for Java Application development.

7. Objects and Classes

Java is an object oriented programming language, thus every application we develop in java is done by using objects and classes.

An **Object** is an entity that has an identity(name), state (properties) and behaviour (functionality). For example an Apple iPhone has states as 'width', 'height', 'color' and behaviour as 'to call', 'browse internet', 'play movie'. An Object is the instance of a class. So, before creating an object in Java, a class should be defined. To access member of any particular class, an object is created of the respective class. An object is created by using the [new](#) keyword in java.

A **Class** is defined as a group of objects which have common properties. Identity, state and behaviour of an object are declared in a class. With the help of an object, members of one class can be called from another class. Any number of objects can be created of a class.

To define a class and creating its object.

Syntax:

```
class MobilePhones // defining a MobilePhone class
{
    String name= "Apple iPhone"; // data member
```

```

String color= "White"; // data member
int height= 5 ; // data member

public static void main(String args[])
{
    // creating an object obj of class MobilePhone
    MobilePhones obj=new MobilePhones();

    // accessing its member variable.
    System.out.println(obj.name);
    System.out.println(obj.color);
    System.out.println(obj.height);

}
}

```

Output

```

Apple iPhone
White
5

```

8. Java Keywords

Keywords are reserved words that are predefined in Java. Each keywords have special meaning and cannot be used as variable or class name.

abstract	assert	boolean	break	byte
case	catch	char	class	const
continue	default	do	double	else
enum	extends	final	finally	float
for	goto	if	implements	import
instanceof	int	interface	long	native
new	package	private	protected	public
return	short	static	strictfp	super
switch	synchronized	this	throw	throws
transient	try	void	volatile	while

9. Simple Java Programs

- Open a notepad and write the following code

Example 1: To print a statement

```
public class First
{
    public static void main(String args[])
    {
        System.out.println("This is my first Java Program"); // This is a
comment
    }
}
```

Output:

This is my first Java Program

- Java codes must be written inside a class. **class** keyword is used to declare a class in java. In the above example we have written our codes in the class named *First* . The first letter of a class always starts with uppercase and it is case sensitive.
- Every Java program has at least one class and one main method.
- The **main()** method is necessary for every java program.
- The code written inside the **main()** method is executed. The JVM executes everything between the curly braces {} of the main method
- The curly “{” marks the beginning and “}” marks the end of the block of code.
- **System.out.println()** method to print a text on screen where **System** is a predefined class in java, **out** is a variable declared in the **System** class and **println()** is a method name.
- Each statement must end with a semicolon(;).

9.1 Java main() method

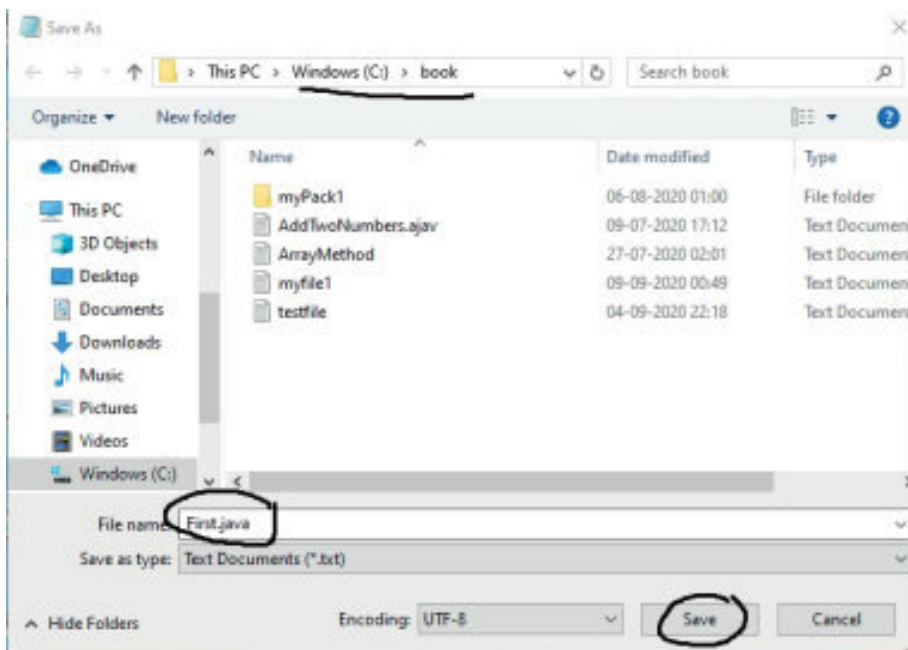
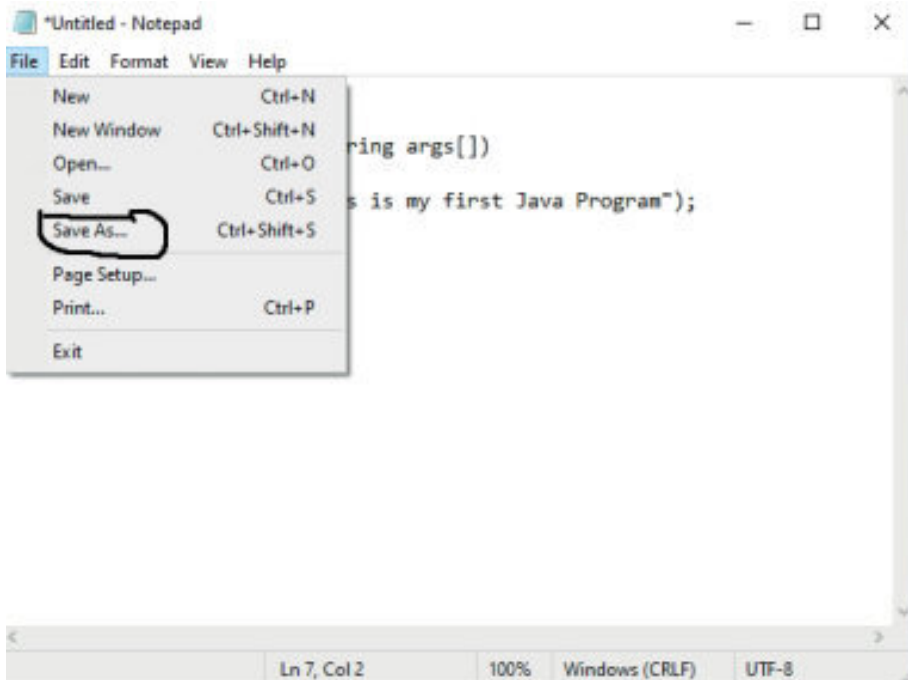
- Java program file (in which the code is written) should be saved with the same name of the class that has the **main()** method.
- In the above case the program is saved with the name First.java.
- **public** keyword is an access modifier (access modifier discussed later).
- **static** is a keyword used to declare any method as static. static methods can be called without creating an object of the class.
- **void** is the return type of the method that returns no value.
- **String[] args** is command line argument (study later).

9.2 Java Comments

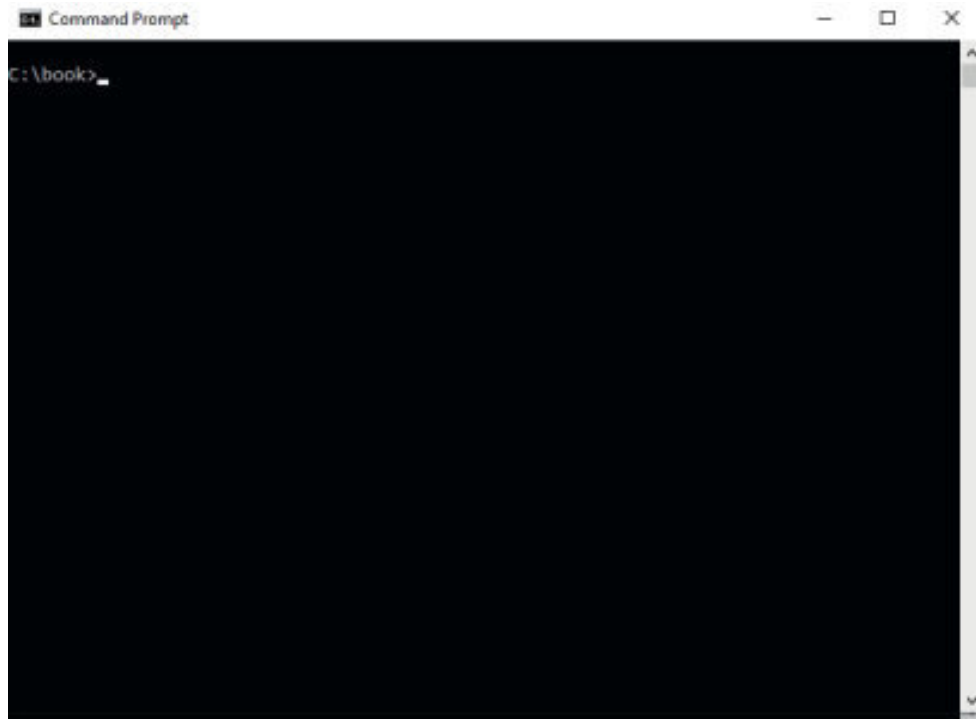
- Single line comment `// comment`
- Multi-line comment `/* comment */`

9.3 Java Programs Execution

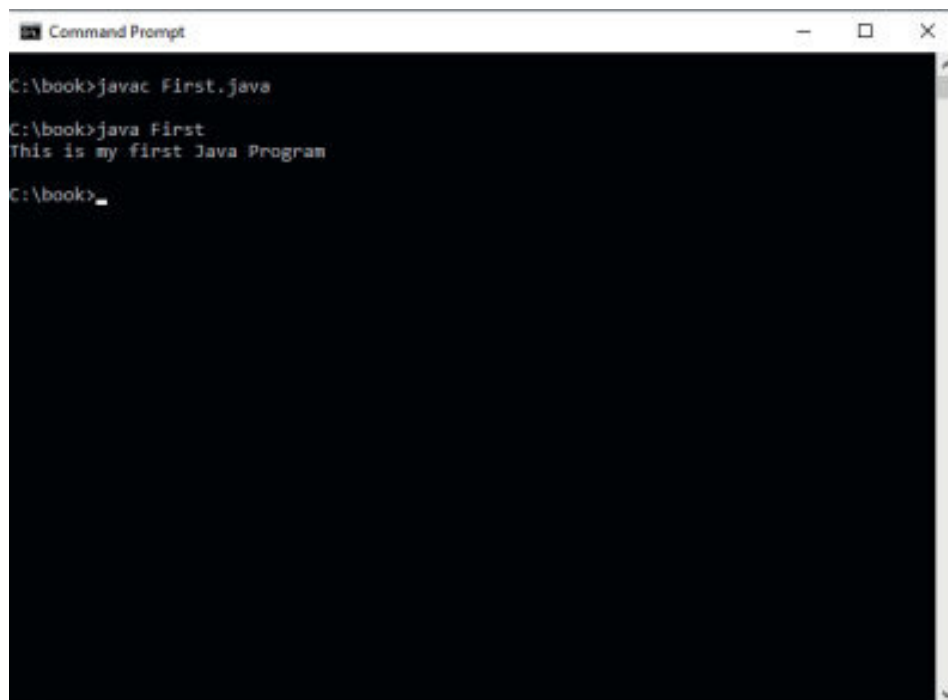
- Save the file name as First.java. Extension .java is required to save the file because name of the class and filename should match in Java.



- Open a command prompt window and go to the directory where you saved the java program.



- Every Java program should be first compiled and then executed.
- To compile : `javac <filename>.java` // `javac First.java`
- To execute: `java <filename>` // `java First`



10. Variables in Java

Variables are used to store data in Java programs. Variables must be declared before it is used in Java programs specifying what type of data the variable will hold. Variable names are case sensitive. The value of a variable can change during execution of a program.

Example 2: To illustrate variable in java program.

```
public class JavaVariable
{
    public static void main (String args[])
    {
        int v=5;
        System.out.println("Variable v= " +v);
        v=7;
        System.out.println("After value changed, Variable v= " +v);
    }
}
```

Output:

```
Variable v= 5
After value changed, Variable v= 7
```

Certain rules and conventions govern the naming of variables.

A variable name :

- Must not be a keyword in Java
- Must not begin with a digit
- Must not contain embedded spaces.

To store a fixed (constant) values which you don't want to change during execution of a program, **final** keyword is used.

Example 3: To illustrate variable that stores a constant value.

```
public class JavaConstant
{
    public static void main (String args[])
    {
        final int num=5;
        num=7;
    }
}
```

Output:

```
JavaConstant.java:7: error: cannot assign a value to final variable num
    num=7;
    ^
1 error
```

11. Data Types in Java

Variables in Java must be declared before they can be used in Java Programs. Data types specify the size and type of values that a variable can take.

There are two types of data types in Java.

- i. Primitives Data Types (predefined data types)
- ii. Non-Primitive(Derived)

11.1 Primitives Data Types

These are basic data type which are independent of any other data type .

Data type	Size (in bits)	Description	Example	Range	Default values
byte	8	Integer	-100, 126	-128 to 127 (signed)	0
short	16	Integer	-1000, 1400	-2^{15} to $2^{15}-1$	0
int	32	Integer	-4300, 2500	-2^{31} to $2^{31}-1$	0
long	64	Integer	-450000L, 350000L	-2^{63} to $2^{63}-1$	0
float	32	Floating Point	-4.67f, 28.44f	$3.4e-038$ to $3.4e+038$	0.0f
double	64	Floating Point	-234.89, 1000.456	$1.7e-308$ to $1.7e+308$	0.0d
char	16	Unicode Character	'a', 67, '\u0011'	\u0000 to \uFFFF	'\u0000'
boolean	1	true or false			false

i. byte

Example 4 : To illustrate byte data type.

```
class DataTypeByte
{
    public static void main(String[] args)
    {
        byte b;
        b = 120;
        System.out.println("Value of byte b = " +b);
    }
}
```

Output:

Value of byte b = 120

ii. short

Example 5 : To illustrate short data type.

```
class DataTypeShort
{
    public static void main(String[] args)
    {
        short s;
        s = -150;
        System.out.println( " Value of short s = " +s);
    }
}
```

Output:

Value of short s = -150

iii. int

Example 6 : To illustrate int data type.

```
class DataTypeInt
{
    public static void main(String[] args)
    {
        int i;
        i = 2250000;
        System.out.println("Value of int i = " +i);
    }
}
```

Output:

Value of int i = 2250000

iv. long

Example 7 : To illustrate long data type.

```
class DataTypeLong
{
    public static void main(String[] args)
    {
        long l = -432460000L;
        System.out.println("Value of long l= "+l);
    }
}
```

Output:

Value of long l= -432460000

long variable values should end with "L"

v. float

Example 8 : To illustrate float data type.

```
class DataTypeFloat
{
    public static void main(String[] args)
    {
        float f = -64.4f;
        System.out.println("Value of flaoat f= "+f);
    }
}
```

Output:

Value of flaoat f= -64.4

float variable value should end with "f".

vi. double

Example 9 : To illustrate double data type.

```
class DataTypeDouble
{
    public static void main(String[] args)
    {
        double d = 425.326;
        System.out.println("Value of double d = "+d);
    }
}
```

Output:

Value of double d = 425.326

vii. char

Example 10 : To illustrate char data type.

```
class DataTypeChar
{
    public static void main(String[] args)
    {
        char c1,c2;
        c1='a';
        c2=67;
        System.out.println("Value of char c1= "+c1);
        System.out.println("Value of char c2= "+c2);
    }
}
```

Output:

Value of char c1= a
Value of char c2= C

viii. boolean

Example 11 : To illustrate Boolean data type.

```
class DataTypeBoolean
{
    public static void main(String[] args)
    {
        boolean b = true;
        System.out.println("Value of boolean b = "+b);
    }
}
```

Output:

Value of boolean b = true

11.2 Non-Primitive(Derived)

They are user defined data types. Example includes Classes, Arrays and Interfaces(discussed later).

12. Type Casting

The process of converting one primitive data type to another primitive data type is called Type Casting.

Java supports two types of Type Casting

1. Widening/Implicit Type Casting
2. Narrowing/Explicit Type Casting

12.1 Widening Casting/Implicit Type Casting

This is also known as automatic type casting because in this type of type casting Java automatically converts smaller size data type to greater size data type.

Smaller Data type (From)	Larger Data type (To)
byte	short, char, int, long, float, double,
short	int, long, float, double
char	int, long, float, double
int	float, long, double
long	double
float	double

Example 12 : To illustrate Widening Casting (double into an int)

```
public class WideningTypeCasting
{
    public static void main(String[] args)
    {
        int i = 65;
        double d = i;

        System.out.println("Value of int i= " +i);
        System.out.println("Value of double d= "+d);
    }
}
```

Output:

```
Value of int i= 65
Value of double d= 65.0
```

Narrowing Casting/Explicit Type Casting

In this type of casting, larger size data type are converted to smaller size data type manually. Casting into smaller size data type may result in the loss of data.

Example 13 : To illustrate Narrowing Casting (double into an int)

```
public class NarrowingTypeCasting
{
    public static void main(String[] args)
    {
        double d = 65.44;
        int i = (int) d;

        System.out.println("Value of double d= " +d);
        System.out.println("Value of int i= " +i);
    }
}
```

Output:

```
Value of double d= 65.44
Value of int i= 65
```

13. Operators

13.1 Arithmetic Operator

Operators	Expression	Description
+	num1 + num2	Adds num1 & num2
-	num1 - num2	Subtracts num2 from num1
*	num1 * num2	Multiplies num1 & num2
/	num1 / num2	Divides num1 by num2
%	num1 % num2	Remainder of dividing num1 by num2

Example 14 : To add two given numbers

```
Public class Add
{
    Public static void main(String args[])
```

```
{
    int num1=15;
    int num2= 20;
    int sum = num1 + num2;
    System.out.println("Sum = "+sum);
}
}
```

Output:

Sum=35

Example 15 : To illustrate arithmetic operations.

```
public class Operators
{
    public static void main(String args[])
    {
        int a=35, b=2;

        System.out.println("a + b = " + (a + b));
        System.out.println("a - b = " + (a - b));
        System.out.println("a * b = " + (a * b));
        System.out.println("a / b = " + (a / b));
        System.out.println("a % b = " + (a % b));

    }
}
```

Output:

a + b = 37
a - b = 33
a * b = 70
a / b = 17
a % b = 1

Example 16 : Accept numbers from user and then add.

```
import java.util.Scanner;
public class AddTwoNumbers
{
    public static void main(String[] args)
    {
        int num1, num2, Sum;
        Scanner sc = new Scanner(System.in); // object sc created for
Scanner class
        System.out.print("Enter the first number: ");
        num1 = sc.nextInt(); // First number stored in num1
        System.out.print("Enter the second number: ");
        num2 = sc.nextInt(); // second number stored in num2

        Sum = num1 + num2;
        System.out.println("Sum of the given numbers " + num1 + " & " +
num2 + " = " + Sum);
    }
}
```

Output:

```
Enter the first number: 45
Enter the second number: 34
Sum of the given numbers 45 & 34 = 79
```

- **import** is a keyword used to access packages (packages will study later) and classes in java program.
- **java.lang** packages are automatically imported in every java program.
- **System** is an inbuilt class of **java.lang** package.
- **Scanner** is an inbuilt class of **util** package used in Java to get user input.
- Create object of Scanner class to use the methods defined in Scanner class.

- Object of the class is specified as class name followed by the object name and use the keyword **new**
- In the above program object **sc** is created for the **Scanner** class using keyword **new**
- **nextInt()** method of **Scanner** class is used to read integer value.

Other methods to read data of different types (table

Method	Task Perform
nextLine()	Reads a String.
nextFloat()	Reads a float value.
nextShort()	Reads a short value.
nextLong()	Reads a long value.
nextDouble()	Reads a double.
nextByte()	Reads a byte value
nextBoolean()	reads a boolean value.

Example 17 : To read a character from user and print its ASCII value

```
import java.util. Scanner;
public class Ascii
{
    public static void main(String[] args)
    {
        Scanner obj = new Scanner(System.in);
        System.out.print("Enter the character : ");
        char ch= obj.next().charAt(0);
        int ascii = ch;
        System.out.println("The ASCII value of " + ch + " is: " + ascii);
    }
}
```

Output:

```
Enter the character : a
The ASCII value of a is: 97
```

- In the above program, **next()** method followed by **charAt(0)** is used to read a character.

13.2 Increment and Decrement Operators

Operators	Expression	Description
++	++x	Increment x by 1 then use the new value of x in the expression
	x++	Use the current value of x in the expression, then increment x by 1
--	--x	Decrement x by 1 then use the value of x in the expression
	x--	Use the current value of x in the expression, then decrement x by 1

Example 18 : To illustrate Increment and Decrement Operations.

```
public class Operators2
{
    public static void main(String args[])
    {
        int x=5, y=5, res1, res2, a=7,b=7, res3,res4;

        res1=x++;
        System.out.println("Result after x++ = " + res1);
        System.out.println("Value of x " + x);

        res2=++y;
        System.out.println("Result after ++y = " + res2);
        System.out.println("Value of y " + y);
        res3=a--;
        System.out.println("Result after a-- = " + res3);
        System.out.println("Value of a " + a);

        res4=--b;
        System.out.println("Result after --a = " + res4);
        System.out.println("Value of a " + b );
    }
}
```

```
}  
}
```

Output:

Result after x++ = 5
Value of x 6
Result after ++y = 6
Value of y 6
Result after a-- = 7
Value of a 6
Result after --a = 6
Value of a 6

13.3 Relational / Comparison Operators

Operators	Expression	Description
==	num1 == num2	num1 is equal to num2
<	num1 < num2	num1 is less than num2
>	num1 > num2	num1 is greater than num2
<=	num1 <= num2	num1 is less than or equal to num2
>=	num1 >= num2	num1 is greater than or equal to num2
!=	num1 != num2	num1 is not equal to num2

Example 19 : To illustrate Relational Operations

```
public class Relational  
{  
    public static void main(String[] args)  
    {  
        int x = 50, y = 25;  
  
        System.out.println("x == y : " + (x == y));  
        System.out.println("x < y : " + (x < y));  
        System.out.println("x <= y : " + (x <= y));
```

```

        System.out.println("x > y : " + (x > y));
        System.out.println("x >= y : " + (x >= y));
        System.out.println("x != y : " + (x != y));
    }
}

```

Output:

```

x == y : false
x < y : false
x <= y : false
x > y : true
x >= y : true
x != y : true

```

13.4 Logical Operators

Operators	Expression	Description
&&	num1 && num2	If Both are true
	num1 num2	If one is true
!	! num	num is false

Example 20 : To illustrate Logical operations

```

public class LogicalOperator
{
    public static void main(String[] args)
    {

        int num1 = 5, num2 = 10, num3 = 15;
        boolean result;

        result = (num1 > num2) || (num3 > num1);
    }
}

```

```

        System.out.println(result);

        result = (num1 > num2) && (num3 > num1);
        System.out.println(result);

        result=! (num1<num2);
        System.out.println(result);
    }
}

```

Output:

```

true
false
false

```

- The **Boolean** keyword in java is used to store only two possible values, either true or false.

13.5 Assignment Operators

Operators	Expression	Description
=	x=20	x=20
+=	x+=20	x=x+20
-=	x-=20	x=x-20
=	x=20	x=x*20
/=	x/=20	x=x/20
%=	x%=20	x=x%20

Example 21 : To illustrate assignment operations

```

public class AssignmentOperators
{

```

```

public static void main(String[] args)
{
    int a=10,b=20,c=30,d=40,x;
    x = b;
    System.out.println("Value of x = " + x);
    a += 1;
    b -= 1;
    c *= 2;
    d /= 2;
    System.out.println("a= " +a);
    System.out.println("b= " +b);
    System.out.println("c= " +c);
    System.out.println("d= " +d);
}
}

```

Output:

```

Value of x = 20
a= 11
b= 19
c= 60
d= 20

```

13.6 Conditional/Ternary Operator

Operator	Expression	Description
?:	Condition ? expression 1 : expression 2	

Example 22 : To illustrate Conditional operations

```

public class TernaryOperator
{
    public static void main(String[] args)
    {
        int x = 20, y = 10, res, res2;

        res=((x > y) ? (x + y) : (x*y));
        res2= ((x == y) ? (x + y) : (x*y));

        System.out.println( "Condition satisfied then Result= " +res);
        System.out.println( "Condition not satisfied then Result= " +res2);
    }
}

```

Output:

```

Condition satisfied then Result= 30
Condition not satisfied then Result= 200

```

Example 23 : Accept three number and find the largest number.

```

import java.util.Scanner;
public class LargeNumber
{
    public static void main(String[] args)
    {
        int num1, num2, num3, res, t1;

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter First Number: ");
        num1 = sc.nextInt();
        System.out.print("Enter Second Number: ");
        num2 = sc.nextInt();
        System.out.print("Enter Third Number: ");

```

```

        num3 = sc.nextInt();

        t1= num1>num2 ? num1:num2;
        res= num3>t1 ? num3:t1;
        System.out.println("Largest Number is "+res);
    }
}

```

Output:

```

Enter First Number: 45
Enter Second Number: 90
Enter Third Number: 87
Largest Number is 90

```

13.7 Bitwise Operators

Operators	Expression	Description
&	x & y	BitWise AND(exist in both operands)
!	x! y	BitWise OR (exist in either operand)
^	x^y	BitWise Exclusive OR (set in one operand but not both)
~	~x	One's complement

Example 24 : To illustrate BitWise operations

```

public class BitwiseOperators
{
    public static void main(String args[])
    {
        int x = 12;
        int y = 10;
        int Res = 0;

        Res = x & y;
    }
}

```

```

        System.out.println("x & y = " + Res );

        Res = x | y;
        System.out.println("x | y = " + Res );

        Res = x ^ y;
        System.out.println("x ^ y = " + Res );

        Res = ~x;
        System.out.println("~x = " + Res );
    }
}

```

Output:

```

x & y = 8
x | y = 14
x ^ y = 6
~x = -13

```

- Bitwise operators works on bits and perform bit by bit operation.
- In the above program value of x=12 whose binary equivalent is 1100 and value of y=10 whose binary equivalent is 1010
- $x \& y = 1100 \& 1010 = 1000$ which is equal to decimal 8
- $x | y = 1100 | 1010 = 1110$ which is equal to decimal 14
- $x \wedge y = 1100 \wedge 1010 = 0110$ which is equal to decimal 6
- $\sim x = -13$

13.8 Shift Operators

Operators	Expression	Description
<<	x << y	Shift left
>>	x >> y	Shift right
>>>	X >>> y	Shift right with zero fill

Example 25 : To illustrate Shift operations

```

public class ShiftOperators
{
    public static void main(String args[])
    {
        int a = 12;
        int b = 10;
        int Res = 0;

        Res = a << 2;
        System.out.println("a << 2 = " + Res );

        Res = a >> 2;
        System.out.println("a >> 2 = " + Res );

        Res = a >>> 2;
        System.out.println("a >>> 2 = " + Res );
    }
}

```

Output:

```

a << 2 = 48
a >> 2 = 3

```

13.9 instance of Operator

- The operator checks whether the object belongs to a particular class or not.

Syntax:

(Object reference variable) **instanceof** (class/subclass/interface type)

Example : To illustrate instanceof Operation

```
public class InstanceofOperators
{
    public static void main(String args[])
    {
        String name = "Alice";
        System.out.println( name instanceof String);
    }
}
```

Output:

true

14. Operator Precedence and Associativity

When more than one operator is used in expression, Java has established operator precedence to determine the order in which the operators are evaluated. The operator at the higher level of precedence is evaluated first. When the operators are having same precedence, they are evaluated either from left to right or from right to left, depending upon the level.

Following order is followed when evaluating an expression:

- Increment and decrement operations
 - Arithmetic operations
 - Comparisons
 - Logical operations
 - Assignment operations
-

Example 27 : To illustrate Operator Precedence

```
public class OperatorPrecedence
{
    public static void main(String[] args)
    {
        int a = 10, b = 5, c = 8, d = 15, e = 5;
        System.out.println("Result= " + (a + b *c-d / e));
    }
}
```

Output:

Result = 47

15. Control Flow Statements.

As the name implies, these statement are used to control the flow of statement in any java program while running. Java Control flow statements are categorized in three types

15.1 Decision Making

15.1.1 Simple if Statement

Syntax :

```
if (condition)
{
    Statements to be executed if condition is true
}
```

Example 28 : To illustrate Simple if

```
public class Simpleif
{
```

```
public static void main(String args[])
{
    int weight = 50;
    if (weight > 45)
    {
        System.out.println("You are Eligible");
    }
    System.out.println(count);
}
}
```

Output:

You are Eligible

15.1.2 if-else Statement

Syntax:

```
        if (condition)
    {
        Statements to be executed if condition is true
    }
    else
    {
        Statements to be executed if condition is false
    }
```

Example 29: To check whether the given number is even or odd

```
import java.util.Scanner;
class EvenOddCheck
{
    public static void main(String args[])
    {
```

```
int num;  
System.out.print("Enter the number:");  
  
Scanner obj = new Scanner(System.in);  
num = obj.nextInt();  
  
if ( num % 2 == 0 )  
    System.out.println("Entered number is even");  
else  
    System.out.println("Entered number is odd");  
}  
}
```

Output:

```
Enter the number:56  
Entered number is even
```

15.1.3 Nested if-else statement

Syntax:

```
if (condition 1)  
{  
    if (condition 2)  
    {  
        Statements to be executed if condition 2 is true  
    }  
    else  
    {  
        Statements  
    }  
}
```

Example 30 : To check for the leap year.

```
import java.util.Scanner;  
public class LeapYear {
```

```
public static void main(String[] args) {  
  
    int year;  
    Scanner obj = new Scanner(System.in);  
    System.out.print("Enter any Year:");  
    year = obj.nextInt();  
  
    boolean leap = false;  
  
    if(year % 4 == 0)  
    {  
        if( year % 100 == 0)  
        {  
            if ( year % 400 == 0)  
                leap = true;  
            else  
                leap = false;  
        }  
        else  
            leap = true;  
    }  
    else {  
        leap = false;  
    }  
  
    if(leap==true)  
        System.out.println(year + " is a Leap Year.");  
    else  
        System.out.println(year + " is not a Leap Year.");  
    }  
}
```

Output:

```
Enter any Year:2020  
2020 is a Leap Year.
```

15.1.4 if-else-if ladder

Syntax:

```
if (condition 1)
    {statements if condition 1 is true
    }
else if (condition 2)
    { statements to be executed if condition 2 is true
    }
else if (condition3)
    { statements to be executed if condition 3 is true
    }
else
    { default statements
    }
```

Example 31: To check largest of three numbers using if-else-if ladder

```
public class IfElseIf
{
    public static void main(String[] args)
    {
        int num1 = 20, num2 = 50, num3 = 10;

        if ( num1 >= num2 && num1 >= num3)
            System.out.println(num1+" is the largest Number");
        else if (num2 >= num1 && num2 >= num3)
            System.out.println(num2+" is the largest Number");
        else
            System.out.println(num3+" is the largest Number");
    }
}
```

Output:

50 is the largest Number

Example 32: To check whether the given number is positive or negative.

```
import java.util.Scanner;
public class NumberCheck
{
    public static void main(String[] args)
    {
        int num;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the Number to check: ");
        num = sc.nextInt();

        if(num > 0)
        {
            System.out.println(num+" is a positive number");
        }
        else if(num < 0)
        {
            System.out.println(num+" is a negative number");
        }
        else
        {
            System.out.println(num+" is neither positive nor negative");
        }
    }
}
```

Output:

First Case

Enter the Number to check: 54

54 is a positive number

Second Case

Enter the Number to check: -56

-56 is a negative number

Third Case

Enter the Number to check: 0

is neither positive nor negative

15.1.5 switch Statement

Syntax:

```
switch(expression)
{
    case value-1 :
        statement -1
        break;

    case value-2 :
        statement-2
        break;

    .....
    .....

    default :
        default statement
        break;
}
```

- **break** is a keyword in java used to break out of the switch block.

Example 33: To illustrate Switch Case

```
public class SwitchCase
{
    public static void main(String args[])
    {
```

```
char grade = 'A';

switch(grade)
{
    case 'A' :
        System.out.println("Excellent!");
        break;

    case 'B' :
        System.out.println("Good");
        break;

    case 'C' :
        System.out.println("You Passed");
        break;

    case 'D' :
        System.out.println("Fail");

    default :
        System.out.println("Invalid grade");
}
System.out.println("Your grade is " + grade);
}
```

Output:

```
Excellent!
Your grade is A
```

Example 34 : To make simple calculator using switch case.

```
import java.util.Scanner;
```

```
public class Calculator
{
    public static void main(String[] args)
    {
        int num1, num2;
        Scanner obj = new Scanner(System.in);
        System.out.print("Enter first number:");

        num1 = obj.nextInt();
        System.out.print("Enter second number:");
        num2 = obj.nextInt();

        System.out.println("Enter an operator (+, -, *, /) ");
        char operator= obj.next().charAt(0);
        int Result=0;
        switch(operator)
        {
            case '+':
                Result = num1 + num2;
                break;

            case '-':
                Result = num1 - num2;
                break;

            case '*':
                Result = num1 * num2;
                break;

            case '/':
                Result = num1 / num2;
                break;

            default:
```

```

        System.out.printf("You have entered wrong operator");
        return;
    }
    System.out.println(num1+" "+operator+" "+num2+" = "+Result);
}
}

```

Output:

```

Enter first number:12
Enter second number:13
Enter an operator (+, -, *, /)
*
12 * 13 = 156

```

Example 35 :To Check whether the given character is vowel or consonant.

```

import java.util.Scanner;
class VowelCheck
{
    public static void main(String[ ] arg)
    {
        boolean charVowel=false;
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter a character : ");
        char ch=sc.next().charAt(0);

        switch(ch)
        {
            case 'a' :
            case 'e' :
            case 'i' :
            case 'o' :
            case 'u' :

```

```
        case 'A' :
        case 'E' :
        case 'I' :
        case 'O' :
        case 'U' : charVowel = true;
    }
    if(charVowel == true)
    {
        System.out.println(ch+" is a Vowel");
    }
    else
    {
        if((ch>='a'&&ch<='z')||(ch>='A'&&ch<='Z'))
            System.out.println(ch+" is a Consonant");
        else
            System.out.println(ch+ " is not an alphabet");
    }
}
```

Output:

First Case

Enter a character : o
o is a Vowel

Second Case

Enter a character : r
r is a Consonant

Third case

Enter a character : 9
9 is not an alphabet

Example 36 : To convert Fahrenheit to Celsius and vice versa.

```
import java.util.Scanner;
class CelsiusFahrenConversion
{
    public static void main(String arg[])
    {
        double fahrenheit,celsius;
        Scanner sc=new Scanner(System.in);
        System.out.println("Choose the type of conversion : \n
1.Fahrenheit to Celsius \n 2.Celsius to Fahrenheit");
        int ch=sc.nextInt();
        switch(ch)
        {
            case 1: System.out.print("Enter temperature in Fahrenheit : ");
                    fahrenheit=sc.nextDouble();
                    celsius=(fahrenheit-32)*5/9;
                    System.out.println("Celsius temperature is = "+celsius);
                    break;

            case 2: System.out.print("Enter temperature in Celsius: ");
                    celsius=sc.nextDouble();
                    fahrenheit=((9*celsius)/5)+32;
                    System.out.println("Fahrenheit temperature is = 
"+fahrenheit);
                    break;
            default: System.out.println("Invalid Choice");
        }
    }
}
```

Output:

First Case

Choose the type of conversion :

1.Fahrenheit to Celsius

2.Celsius to Fahrenheit

1

Enter temperature in Fahrenheit : 98.8

Celsius temperature is = 37.111111111111114

Second Case

Choose the type of conversion :

1.Fahrenheit to Celsius

2.Celsius to Fahrenheit

2

Enter temperature in Celsius: 36.6

Fahrenheit temperature is = 97.88000000000001

Example 37 : To find number of days in a month of a particular year.

```
import java.util.Scanner;
public class DaysOfMonth
{
    public static void main(String[] strings)
    {
        Scanner sc = new Scanner(System.in);
        int num = 0;
        String month="";

        System.out.print("Enter the month Name in lower case : ");
        String name = sc.nextLine();

        System.out.print("Enter the year: ");
        int year = sc.nextInt();

        switch (name) {
```

```
case "january":
    month = "January";
    num = 31;
    break;
case "february":
    month = "February";
    if ((year % 400 == 0) || ((year % 4 == 0) && (year % 100 !=
0)))
    {
        num = 29;
    } else {
        num = 28;
    }
    break;
case "march":
    month = "March";
    num = 31;
    break;
case "april":
    month = "April";
    num = 30;
    break;
case "may":
    month = "May";
    num = 31;
    break;
case "june":
    month = "June";
    num = 30;
    break;
case "july":
    month = "July";
    num = 31;
    break;
case "august":
```



```
        month = "August";
        num = 31;
        break;
    case "september":
        month = "September";
        num = 30;
        break;
    case "october":
        month = "October";
        num = 31;
        break;
    case "november":
        month = "November";
        num = 30;
        break;
    case "december":
        month = "December";
        num = 31;
    }
    System.out.print("Month Name : " +month);
    System.out.print("\nYear : " +year);
    System.out.print("\nNumber of days : " +num);
}
}
```

Output:

```
Enter the month Name in lower case : august
Enter the year: 2020
Month Name : August
Year : 2020
Number of days : 31
```

15.2 Looping Statement

15.2.1 for loop

Syntax:

```
for (initialization; condition; increment/decrement)
{
    Statements;
}
```

Example 38: To illustrate for loop

```
public class ForLoop
{
    public static void main(String args[])
    {
        for (int x = 1; x < 10; x = x + 1)
        {
            System.out.print("value of x : " + x );
            System.out.print("\n");
        }
    }
}
```

Output:

```
value of x : 1
value of x : 2
value of x : 3
value of x : 4
value of x : 5
value of x : 6
value of x : 7
value of x : 8
value of x : 9
```

Example 39: To check whether the given number is prime or not.

```
import java.util.Scanner;
class CheckPrime
{
    public static void main(String args[])
    {
        int t;
        boolean Prime=true;
        Scanner obj= new Scanner(System.in);
        System.out.print("Enter any number:");
        int num=obj.nextInt();

        for(int i=2;i<=num/2;i++)
        {
            t=num%i;
            if(t==0)
            {
                Prime=false;
                break;
            }
        }

        if(Prime)
            System.out.println(num + " is a Prime Number");
        else
            System.out.println(num + " is not a Prime Number");
    }
}
```

Output:

```
Enter any number:23
23 is a Prime Number
```

Example 40: Accept a number and calculate cube of all the numbers upto that number.

```
import java.util.Scanner;
public class NumberCube
{
    public static void main(String[] args)
    {
        int i,num;
        System.out.print("Enter the number up to which cube is required to
be calculated : ");
        Scanner in = new Scanner(System.in);
        num = in.nextInt();
        for(i=1;i<=num;i++)
        {
            System.out.println("Number is : " +i+ " Cube of " +i+ " is : "+(i*i*i));
        }
    }
}
```

Output:

```
Enter the number up to which cube is required to be calculated : 5
Number is : 1 Cube of 1 is : 1
Number is : 2 Cube of 2 is : 8
Number is : 3 Cube of 3 is : 27
Number is : 4 Cube of 4 is : 64
Number is : 5 Cube of 5 is : 125
```

Example 41: To print Floyd's triangle

```
import java.util.Scanner;
class FloydTriangle
{
    public static void main(String args[])
```

```

{
    int rows, num = 1, i, j;

    Scanner length = new Scanner(System.in);
    System.out.print("Enter the number of rows for floyd's triangle:");

    rows = length.nextInt();

    for ( i= 1 ; i <= rows ; i++ )
    {
        for ( j = 1 ; j <= i ; j++ )
        {
            System.out.print(num+" ");
            num++;
        }

        System.out.println();
    }
}

```

Output:

```

Enter the number of rows for floyd's triangle:5
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15

```

Example 42: To accept a number from user and calculate factorial.

```

import java.util.Scanner;
class Factorial{

```

```

public static void main(String args[]){
    int i,fact=1;

    Scanner obj=new Scanner(System.in);
    System.out.print("Enter the number for factorial : ");
    int num=obj.nextInt();

    for(i=1;i<=num;i++){
        fact=fact*i;
    }
    System.out.println("Factorial of "+num+" is: "+fact);
}
}

```

Output:

```

Enter the number for factorial : 5
Factorial of 5 is: 120

```

Example 43: To generate Fibonacci series.

```

import java.util.Scanner;
class Fibonacci
{
    public static void main(String args[])
    {
        int n1=0,n2=1,n3,i;
        System.out.print("Enter the length of the series: ");
        Scanner obj = new Scanner(System.in);
        int num = obj.nextInt();
        System.out.print(n1+" "+n2);    //printing 0 and 1
        for(i=2;i<num;++i)              //loop starts from 2
        {
            n3=n1+n2;
            System.out.print(" "+n3);
            n1=n2;

```

```
        n2=n3;
    }
}
}
```

Output:

Enter the length of the series: 8
0 1 1 2 3 5 8 13

Example 44: To print multiplication table of a number.

```
import java.util.Scanner;
public class Table
{
    public static void main(String[] args)
    {
        Scanner obj = new Scanner(System.in);
        System.out.print("Enter the number : ");
        int num= obj.nextInt();
        for(int i = 1; i <= 10; ++i)
        {
            System.out.printf("%d * %d = %d \n", num, i, num * i);
        }
    }
}
```

Output:

Enter the number : 8
8 * 1 = 8
8 * 2 = 16
8 * 3 = 24
8 * 4 = 32
8 * 5 = 40
8 * 6 = 48

8 * 7 = 56
8 * 8 = 64
8 * 9 = 72
8 * 10 = 80

Example 45: To print factors of a number

```
import java.util.Scanner;
public class Factors
{
    public static void main(String[] args)
    {

        Scanner obj = new Scanner(System.in);
        System.out.print("Enter the number : ");
        int num= obj.nextInt();

        System.out.print("Factors of " + num + " are: ");
        for(int i = 1; i <= num; ++i) {
            if (num % i == 0) {
                System.out.print(i + " ");
            }
        }
    }
}
```

Output:

Enter the number : 50
Factors of 50 are: 1 2 5 10 25 50

Example 46 : To find frequency(occurrence) of character in a word/sentence.

```
import java.util.Scanner;
```



```

public class CharOccurrence
{
    public static void main(String[] args)
    {
        Scanner obj = new Scanner(System.in);
        System.out.print("Enter the word/sentence : ");
        String str = obj.nextLine();

        System.out.print("Enter the character to search : ");
        char ch = obj.next().charAt(0);
        int count = 0;

        for(int i = 0; i < str.length(); i++)
        {
            if(ch == str.charAt(i))
            {
                ++count;
            }
        }
        System.out.println("Frequency of " + ch + " = " + count);
    }
}

```

Output:

```

Enter the word/sentence : Java is a high level programming language
Enter the character to search : a
Frequency of a = 6

```

15.2.1.1 Pattern Programs in java

Example 47 : To print pyramid.

```

import java.util.Scanner;
public class Pyramid

```

```

{
    public static void main(String args[])
    {
        Scanner obj=new Scanner(System.in);
        System.out.print("Enter the size of the Pyramid : ");
        int n=obj.nextInt();

        for (int i=0; i<n; i++)
        {
            for (int j=n-i; j>1; j--)
            {
                System.out.print(" ");
            }
            for (int j=0; j<=i; j++ )
            {
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}

```

Output:

Enter the size of the Pyramid : 6

```

    *
   **
  ***
 ****
*****
*****

```

Example 48: To print right angle triangle star pattern.

```

import java.util.Scanner;
public class RightAnglePattern
{
    public static void main(String args[])
    {
        Scanner obj=new Scanner(System.in);
        System.out.print("Enter the size : ");
        int n=obj.nextInt();

        for (int i=0; i<n; i++)
        {
            for (int j=0; j<=i; j++ )
            {
                System.out.print("* ");
            }
            System.out.println("");
        }
    }
}

```

Output:

```

Enter the size : 5
*
* *
* * *
* * * *
* * * * *

```

Example 49: To print diamond shape pattern.

```

import java.util.Scanner;
public class DiamondPattern
{
    public static void main(String args[])

```

```

{
    int n, i, j, s = 1;
    System.out.print("Enter the size: ");
    Scanner obj = new Scanner(System.in);
    n = obj.nextInt();
    s = n - 1;
    for (j = 1; j <= n; j++)
    {
        for (i = 1; i <= s; i++)
        {
            System.out.print(" ");
        }
        s--;
        for (i = 1; i <= 2 * j - 1; i++)
        {
            System.out.print("*");
        }
        System.out.println("");
    }
    s = 1;
    for (j = 1; j <= n - 1; j++)
    {
        for (i = 1; i <= s; i++)
        {
            System.out.print(" ");
        }
        s++;
        for (i = 1; i <= 2 * (n - j) - 1; i++)
        {
            System.out.print("*");
        }
        System.out.println("");
    }
}
}

```

Output:

```
  *
 ***
*****
*****
 *****
  ***
   *
```

Example 50: To print reverse pyramid

```
import java.util.Scanner;
public class ReversePyramid
{
    public static void main(String[] args)
    {
        Scanner obj = new Scanner(System.in);
        System.out.print("Enter the size: ");
        int n = obj.nextInt();
        for (int i= 0; i<=n-1 ; i++)
        {
            for (int j=0; j<=i; j++)
            {
                System.out.print(" ");
            }
            for (int k=0; k<=n-1-i; k++)
            {
                System.out.print("*" + " ");
            }
            System.out.println();
        }
    }
}
```

Output:

Enter the size: 5

```
* * * * *
 * * * *
  * * *
   * *
    *
```

Example 51: To print Pascal's triangle number program.

```
import java.util.Scanner;
public class PascalTriangle
{
    public static void main(String[] args)
    {
        Scanner obj = new Scanner(System.in);
        System.out.print("Enter the size: ");
        int n = obj.nextInt();

        for (int i = 0; i < n; i++)
        {
            int number = 1;
            System.out.printf("%" + (n - i) * 2 + "s", "");
            for (int j = 0; j <= i; j++)
            {
                System.out.printf("%4d", number);
                number = number * (i - j) / (j + 1);
            }
            System.out.println();
        }
    }
}
```

Output:

Enter the size: 5

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Example 52: To print simple number pattern.

```
import java.util.Scanner;
public class NumberPattern
{
    public static void main(String[] args)
    {
        Scanner obj = new Scanner(System.in);
        System.out.print("Enter the size : ");
        int n = obj.nextInt();
        int i, j, k = 1;
        for (i = 1; i <= n; i++)
        {
            for (j = 1; j < i + 1; j++)
            {
                System.out.print(k++ + " ");
            }
            System.out.println();
        }
    }
}
```

Output:

Enter the size : 4

```
1
2 3
```

4 5 6
7 8 9 10

Example 53 : To display alphabets rhombus structure

```
import java.util.Scanner;
public class CharRhombus
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number: ");
        int n = sc.nextInt();
        int c1 = 1; int c2 = 1;
        char c = 'A';

        for (int i = 1; i < (n * 2); i++)
        {
            for (int space = n - c2; space > 0; space--)
            {
                System.out.print(" ");
            }
            if (i < n)
            {
                c2++;
            }
            else
            {
                c2--;
            }
            for (int j = 0; j < c1; j++)
            {
                System.out.print(c);
```



```

        if (j < c1 / 2)
        {
            c++;
        }
        else
        {
            c--;
        }
    }
    if (i < n)
    {
        c1 = c1 + 2;
    }
    else
    {
        c1 = c1 - 2;
    }
    c = 'A';

    System.out.println();
}
}
}

```

Output:

Enter the number:

5

```

    A
  ABA
ABCBA
ABCDcba
ABCDEDCBA
ABCDcba
ABCBA
  ABA

```

15.2.2 While loop

Syntax:

```
while (condition)
{
    Statement
}
```

Example 54: To illustrate while loop

```
public class WhileLoop
{
    public static void main(String args[])
    {
        int x = 1;
        while( x < 5 )
        {
            System.out.print("value of x : " + x );
            x++;
            System.out.print("\n");
        }
    }
}
```

Output:

```
value of x : 1
value of x : 2
value of x : 3
value of x : 4
```

Example 55: Accept a number from user and reverse it.

```
import java.util.Scanner;
class ReverseNumber
{
    public static void main(String args[])
    {
        int num=0;
        int rev =0;
        System.out.print("Enter the number to reverse: ");
        Scanner in = new Scanner(System.in);
        num = in.nextInt();
        while( num != 0 )
        {
            rev = rev * 10;
            rev = rev + num%10;
            num = num/10;
        }

        System.out.println("Reverse of input number is: "+rev);
    }
}
```

Output:

```
Enter the number to reverse: 4563
Reverse of input number is: 3654
```

Example 56: To check whether the given number is Palindrome or not.

```
import java.util.Scanner;
class Palindrome
{
```

```

public static void main(String args[])
{
    int r,sum=0,temp;
    System.out.print("Enter the number to check for Palindrome: ");
    Scanner obj = new Scanner(System.in);
    int num = obj.nextInt();
    temp=num;
    while(num>0)
    {
        r=num%10; //getting remainder
        sum=(sum*10)+r;
        num=num/10;
    }
    if(temp==sum)
        System.out.println("It is a palindrome number ");
    else
        System.out.println("It is not a palindrome number");
    }
}

```

Output:

```

Enter the number to check for Palindrome: 16561
It is a palindrome number

```

Example 57: To check whether the given number is Armstrong number or not.

```

import java.util.Scanner;
class Armstrong
{
    public static void main(String[] args)
    {
        int i=0,j,temp;

```

```

        System.out.print("Enter the number to check for Armstrong
Number: ");
        Scanner obj = new Scanner(System.in);
        int num = obj.nextInt();
        temp=num;
        while(num>0)
        {
            j=num%10;
            num=num/10;
            i=i+(j*j*j);
        }
        if(temp==i)
            System.out.println("It is an armstrong number");
        else
            System.out.println("It is Not an armstrong number");
    }
}

```

Output:

```

Enter the number to check for Armstrong Number: 153
It is an Armstrong number

```

Example 58: To illustrate addition of two binary numbers.

```

import java.util.Scanner;
public class BinaryAddition
{
    public static void main(String[] args)
    {
        long num1, num2;
        int i = 0, r = 0;
        int[] sum = new int[20];
        Scanner in = new Scanner(System.in);
    }
}

```

```

System.out.print("Enter the first binary number: ");
num1 = in.nextLong();
System.out.print("Enter the second binary number: ");
num2 = in.nextLong();

while (num1 != 0 || num2 != 0)
{
    sum[i++] = (int)((num1 % 10 + num2 % 10 + r) % 2);
    r = (int)((num1 % 10 + num2 % 10 + r) / 2);
    num1 = num1 / 10;
    num2 = num2 / 10;
}
if (r != 0)
{
    sum[i++] = r;
}
--i;
System.out.print("Sum of two given binary numbers: ");
while (i >= 0)
{
    System.out.print(sum[i--]);
}
System.out.print("\n");
}
}

```

Output:

```

Enter the first binary number: 1001
Enter the second binary number: 1010
Sum of two given binary numbers: 10011

```

15.2.3 do-while-loop

Syntax:

```
do
{
    Statements
} while (condition);
```

Example 59: To illustrate do-while-loop

```
public class DoWhileLoop
{
    public static void main(String args[])
    {
        int i=5;
        do
        {
            System.out.println(i);
            i--;
        } while(i>=1);
    }
}
```

Output:

```
5
4
3
2
1
```

Example 60: To calculate HCF and LCM of two numbers.

```
import java.util.Scanner;
public class HcfLcm
{
    public static void main(String args[])
```

```

{
    int t1, t2, num1, num2, temp, res1, res2;
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter First Number: ");
    num1 = sc.nextInt();
    System.out.print("Enter Second Number: ");
    num2 = sc.nextInt();

    t1 = num1;
    t2 = num2;
    do
    {
        temp = t2;
        t2 = t1%t2;
        t1 = temp;
    } while(t2 != 0);
    res1 = t1;
    res2= (num1*num2)/res1;

    System.out.println("HCF of input numbers: "+res1);
    System.out.println("LCM of input numbers: "+res2);
}
}

```

Output:

```

Enter First Number: 15
Enter Second Number: 35
HCF of input numbers: 5
LCM of input numbers: 105

```

15.3 jump Statements

15.3.1 break

Example 61: To illustrate break keyword in Java

```
public class BreakLoop
{
    public static void main(String args[])
    {
        int i;
        for (i =1; i<10; i++)
        {
            System.out.println("var: "+i);
            if (i==7)
            {
                break;
            }
        }
        System.out.println("Loop break");
    }
}
```

Output:

```
value: 1
value: 2
value: 3
value: 4
value: 5
value: 6
value: 7
Loop break
```

15.3.2 continue

Example 62 : To illustrate continue keyword in Java

```

public class ContinueLoop
{
    public static void main(String args[])
    {
        for (int j=0; j<10; j++)
        {
            if (j % 2==0)
            {
                continue;
            }
            System.out.print(j+" ");
        }
    }
}

```

Output:

3 5 7 9

15.3.3 Label

Example 63 : To illustrate Labelled Break

```

public class LabelledBreak1
{
    public static void main(String args[])
    {
        loop1: for(int i=0; i<10; i++)
        {
            if (i==5)
            {
                break loop1;
            }
            System.out.println(" "+i );
        }
    }
}

```

```
}  
}
```

Output:

```
0  
1  
2  
3  
4
```

Example 64 : To illustrate Labelled continue

```
public class LabeledContinue1  
{  
    public static void main(String args[])  
    {  
        loop1: for(int i=1; i<5; i++)  
        {  
            for(int j=1; j<=5; j++)  
            {  
                System.out.println(i*j + " ");  
                if (i==j)  
                {  
                    continue loop1;  
                }  
            }  
        }  
    }  
}
```

Output:

```
1  
2  
4  
3
```

6

9

4

8

12

16

16. Methods

It is declared within a class. These are set of statements or codes that only run when it is called. It is used to reuse the code once defined.

Java offers two types of methods.

- Built-in method such as `System.out.println()` .
- User defined method

16.1 Declaration of Method

Syntax:

```
Public class First
{
    Static void firstMethod()
    {
        Set of statements to be executed
    }
}
```

- `firstMethod()` is the name of the method.
- Method name should start with lowercase.

Methods to read data of different types (table

Method	Task Perform
<code>nextLine()</code>	Reads a String.
<code>nextFloat()</code>	Reads a float value.
<code>nextShort()</code>	Reads a short value.
<code>nextLong()</code>	Reads a long value.
<code>nextDouble()</code>	Reads a double.
<code>nextByte()</code>	Reads a byte value
<code>nextBoolean()</code>	reads a boolean value.

16.2 Build-in Methods

Java has a library of pre defined methods, organized in different packages.

Some of the Built-in methods are:

parseInt()

- This built-in method belongs to Integer class of java.lang package
- Integer.parseInt() method is used to convert strings to integer type.

Example 65: To convert binary number to decimal.

```
public class BinaryToDecimal
{
    public static void main(String args[])
    {
        String s1="1101";
        int res=Integer.parseInt(s1,2);
        System.out.println(res);
    }
}
```

Output:

13

- In the above program, a binary value is stored in string variable s1
- Base value 2 is provided along with s1 because binary numbers have 2 as its base value.

Example 66: To convert octal to decimal.

```
public class OctalToDecimal
{
    public static void main(String args[])
    {
```

```
String octal1="153";
int res=Integer.parseInt(octal1,8);
System.out.println(res);
}
}
```

Output:

107

Example 67 : To convert hexadecimal to decimal.

```
public class HexaToDecimal
{
    public static void main(String args[])
    {
        String hexa1="16B";
        int res=Integer.parseInt(hexa1,16);
        System.out.println(res);
    }
}
```

Output:

363

Integer.toBinaryString(), Integer.toOctalString and **Integer.toHexString()** methods are used to decimal to binary, decimal to octal and decimal to hexadecimal conversions.

Example 68: To illustrate conversion of decimal number to other number system.

```
import java.util.Scanner;
class DecimalConversions
{
```

```

public static void main(String args[])
{
    Scanner input = new Scanner( System.in );
    System.out.print("Enter a decimal number : ");
    int num =input.nextInt();

    String str1 = Integer.toBinaryString(num);
    String str2 = Integer.toOctalString(num);
    String str3 = Integer.toHexString(num);

    System.out.println("Decimal to Binary: "+str1);
    System.out.println("Decimal to Octal: "+str2);
    System.out.println("Decimal to Hexa: "+str3);
}
}

```

Output:

```

Enter a decimal number : 153
Decimal to Binary: 10011001
Decimal to Octal: 231
Decimal to Hexa: 99

```

Example 69: To get current date and time.

```

import java.time.LocalDateTime;
public class DateTime
{
    public static void main(String[] args)
    {
        LocalDateTime res = LocalDateTime.now();
        System.out.println("Current Date and Time is: " + res);
    }
}

```

Output:

Current Date and Time is: 2020-08-29T03:17:25.519

- In the above program `LocalDateTime.now()` method is used to get the current date and time in default format.

16.3 User defined Methods

Java also allows user to create its own methods.

Call a user defined method

```
public class MethodCall
{
    static void methodTest()
    {
        System.out.println("This is my first Method call");
    }
    public static void main(String[] args)
    {
        methodTest();
    }
}
```

Output:

This is my first Method call

16.4 Method Parameter:

- A Parameter is a variable defined by a method.
- It is passed during the definition of a method.
- Parameters don't change during the execution of a program.
- They are specified after the method name, inside the parenthesis.

16.5 Method Argument

- An argument is the value that is passed to a method when it is called.
- Arguments may or may not change during the execution of a program.

Example 70: To illustrate method with parameter

```
public class MethodParameter
{
    public static int myMethod(int i, int j)
    {
        int r=i+j;
        return r;
    }

    public static void main(String[] args)
    {
        int res=myMethod(4,8);
        System.out.println("Sum = " +res);
    }
}
```

Output:

Sum = 12

Example 71 : To calculate power of a number.

```
import java.util.Scanner;
class PowerOfNumber
{
    public static void main(String args[])
    {
        long num,pow,res=1;
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter the number : ");
        num=sc.nextLong();
    }
}
```

```

        System.out.print("Enter the power for the number : ");
        pow=sc.nextLong();
        res=PowerOfNumber.power(num,pow);
        System.out.println(num+"^"+pow+"="+res);
    }
    static long power(long num1,long pow1)
    {
        long res1=1;
        if(num1>=0 && pow1==0)
            res1 =1;
        else if(num1==0 && pow1>=1)
            res1=0;
        else
            for(int i=1;i<=pow1;i++)
                res1=res1 *num1;
        return res1;
    }
}

```

Output:

```

Enter the number : 6
Enter the power for the number : 4
6^4=1296

```

Example 72 : To calculate area of a circle.

```

import java.util.Scanner;
class AreaOfCircle
{
    public static void main(String args[])
    {
        Scanner s= new Scanner(System.in);
        System.out.print("Enter the radius:");
        double radius= s.nextDouble();
    }
}

```

```

        double area=area(radius);
        System.out.println("Area of Circle is: " + area);
    }
    static double area(double r)
    {
        return (22*r*r)/7;
    }
}

```

Output:

```

Enter the radius:5
Area of Circle is: 78.57142857142857

```

Example 73 : Accept marks from user and calculate CGPA.

```

import java.util.Scanner;
class FindCGPA
{
    public static void main(String args[])
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter number of subjects : ");
        int num=sc.nextInt();
        double[] marks=new double[num];

        System.out.println("Enter marks");
        for(int i=0;i<num;i++)
        {
            marks[i]=sc.nextInt();
        }
        double cgpa,sum;
        sum= cgpa(marks);
        cgpa=sum/num;
    }
}

```

```
        System.out.println("CGPA : "+cgpa);
    }
    static double cgpa(double marks[])
    {
        double sum=0;
        double res[]=new double[marks.length];
        for(int i=0;i<marks.length;i++)
        {
            res[i]=(marks[i]/10) ;
        }
        for(int i=0;i<marks.length;i++)
        {
            sum+=res[i];
        }
        return sum;
    }
}
```

Output:

```
Enter number of subjects : 5
Enter marks
90
89
78
90
89
CGPA : 8.72
```

Example 74: To calculate product of two binary numbers.

```
import java.util.Scanner;
public class BinaryMultiplication
{
```

```

public static void main(String[] args)
{
    long num1, num2, mul = 0;
    int i, fact = 1;
    Scanner in = new Scanner(System.in);
    System.out.print("Enter the first binary number: ");
    num1 = in.nextLong();
    System.out.print("Enter the second binary number: ");
    num2 = in.nextLong();
    while (num2 != 0)
    {
        i = (int)(num2 % 10);
        if (i == 1)
        {
            num1 = num1 * fact;
            mul = product((int) num1, (int) mul);
        }
        else
        {
            num1 = num1 * fact;
        }
        num2 = num2 / 10;
        fact = 10;
    }
    System.out.print("Product of two binary numbers: " + mul + "\n");
}

static int product(int num1, int num2)
{
    int i = 0, r = 0;
    int[] sum = new int[20];
    int product = 0;
    while (num1 != 0 || num2 != 0)
    {
        sum[i++] = (num1 % 10 + num2 % 10 + r) % 2;
    }
}

```

```

    r = (num1 % 10 + num2 % 10 + r) / 2;
    num1 = num1 / 10;
    num2 = num2 / 10;
}
if (r != 0)
{
    sum[i++] = r;
}
--i;
while (i >= 0)
{
    product = product * 10 + sum[i--];
}
return product;
}
}

```

Output:

```

Enter the first binary number: 101
Enter the second binary number: 101
Product of two binary numbers: 11001

```

16. 6 Instance Method

It is a non-static method(method without using static keyword).It is necessary to create the object of the class before calling the instance method.

Example 75 : To illustrate Instance method

```

public class InstanceMethod
{
    public static void main(String[] args)
    {
        int num1 = 20;
    }
}

```

```
int num2 = 15;
InstanceMethod obj=new InstanceMethod();
int sum = obj.add(num1, num2);
System.out.println("Sum= " + sum);
}
public int add(int num1, int num2)
{
    int s;
    s=num1+num2;
    return s; //returning the sum
}
}
```

Output:

Sum= 35

17. Constructor

- Java supports a special type of method called constructor
- It enables an object to initialize itself when it is created. Every time an object is created using the new() keyword, at least one constructor is called.
- If there is no constructor in the class, it calls a default constructor.
- Constructor name must be the same as its class name.
- A constructor cannot be abstract, static, final and synchronized
- A constructor must have no explicit return type

Example 76: To illustrate Constructor

```
public class Constructor
{
    Constructor() // constructor
```



```

    {
        System.out.println("This is your constructor with no argument");
    }
    public static void main(String[] args)
    {
        Constructor obj = new Constructor();
    }
}

```

Output:

This is your constructor with no argument

Example 77: To illustrate parameterized (with parameters) constructor

```

public class ConstructorStud
{
    int studId;
    String studName;
    ConstructorStud(int id, String name) //parameterized constructor
    {
        this.studId = id;
        this.studName = name;
    }
    void info()
    {
        System.out.println("Roll: "+studId+" Name: "+studName);
    }
    public static void main(String args[])
    {
        ConstructorStud obj1 = new ConstructorStud(22,"Anil");
        ConstructorStud obj2 = new ConstructorStud(24,"Sunil");
        obj1.info ();
        obj2.info ();
    }
}

```

```
}  
}
```

Output:

```
Roll: 22 Name: Anil  
Roll: 24 Name: Sunil
```

- [this](#) keyword is used to refer to the current object inside a constructor.

18. Java Polymorphism

- It is a concept by which a single task can be performed in different ways.
- Java Polymorphism can be divided in to two types:
 1. Compile time Polymorphism
 2. Run time Polymorphism
- Compile time polymorphism can be achieved through method overlaoding
- Run time polymorphism can be achieved through method overriding.

19. Method Overloading

- Multiple methods can have same name but different
 1. Number of parameters
 2. Data type of parameters
- Method return type doesn't matter in case of method overloading

Example 78: Method overloading with different number of parameters

```
public class MethodOverLoading
```

```

{
    public static int sum(int x, int y)
    {
        int res=x+y;
        return res;
    }
    public static int sum(int x, int y, int z)
    {
        int res1=x+y+z;
        return res1;
    }

    public static void main(String args[])
    {
        int Result=sum(4,5);
        System.out.println("Result of first Method = " +Result);

        int Result2=sum(4,5,6);
        System.out.println("Result of second Method = " +Result2);
    }
}

```

Output:

Result of first Method = 9
Result of second Method = 15

Example 79: Method overloading with different data types of parameters.

```

public class MethodOverLoading2
{
    public static int sum(int x, int y)
    {
        return(x+y);
    }
}

```

```

    }

    public static double sum(double x, double y)
    {
        return (x+y);
    }

    public static void main(String args[])
    {
        int Result=sum(4,5);
        System.out.println("Result of first Method = " +Result);

        double Result2=sum(4.5,5.2);
        System.out.println("Result of second Method = " +Result2);
    }
}

```

Output:

```

Result of first Method = 9
Result of second Method = 9.7

```

20. Method Overloading and Type Promotion

- When a data type of smaller size is promoted to the data type of larger size.
- Sequence of promotion is from left to right i.e. left side data type can be promoted to any data type on the right side of it.

byte → short → int → long

short → int → long

int → long → float → double

float → double

long → float → double

Example 80: To illustrate Method overloading with Type Promotion.

```
class PromotionType
{
    void disp(int a, float b)
    {
        System.out.println("This is First Method body");
    }
    void disp(long a, long b, double c)
    {
        System.out.println("This is second Method body");
    }
    public static void main(String args[])
    {
        PromotionType obj = new PromotionType();
        obj.disp(100,20);
    }
}
```

Output:

This is First Method body

In the above example, the (int,int) value is passed while calling the disp(int,float) method but it is got promoted to the float type as there wasn't any method with the parameters as(int,int).

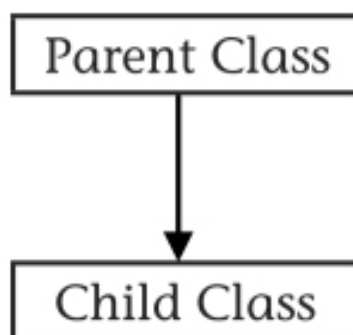
21. Inheritance

- It allows one class to inherit the methods and attributes of other class as well as add its own unique element.
- The class that inherits the other class is called subclass (child class) and the class being inherited from is known as super class (Parent class).
- The keyword used for inheritance is extends
- Inheritance helps to reduce redundancy in code.

Syntax:

```
Class subclass_name extends superclass_name
{
    Method declaration;
}
```

21.1 Single Inheritance: Only one super class.



Example 81: To illustrate single inheritance

```
class Parent
```

```
{
    public void print1()
    {
        System.out.println("This is Parent Class");
    }
}

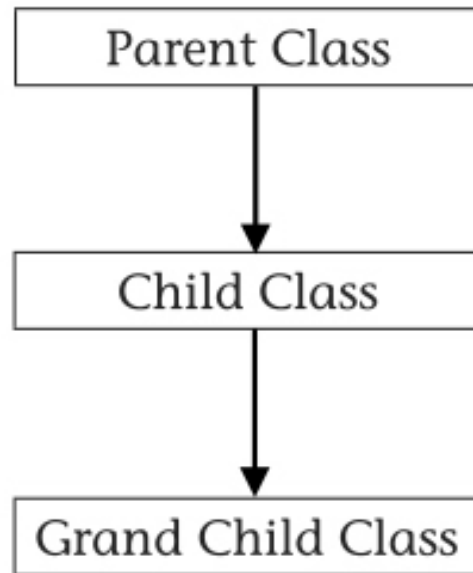
class Child extends Parent
{
    public void print2()
    {
        System.out.println("This is Child Class");
    }
}

public class SingleInheritance
{
    public static void main(String[] args)
    {
        Child obj = new Child();
        obj.print1();
        obj.print2();
    }
}
```

Output:

This is Parent Class
This is Child Class

21.2 Multilevel Inheritance: derived from derived class



Example 82 : To illustrate multilevel inheritance

```
class ParentClass
{
    void print1()
    {
        System.out.println("This is Parent class");
    }
}
class SubClass1 extends ParentClass
{
    void print2()
    {
        System.out.println("This is Sub Class 1");
    }
}
class Subclass2 extends SubClass1
{
    void print3()
    {
        System.out.println("This is Sub Class 2");
    }
}
```



```

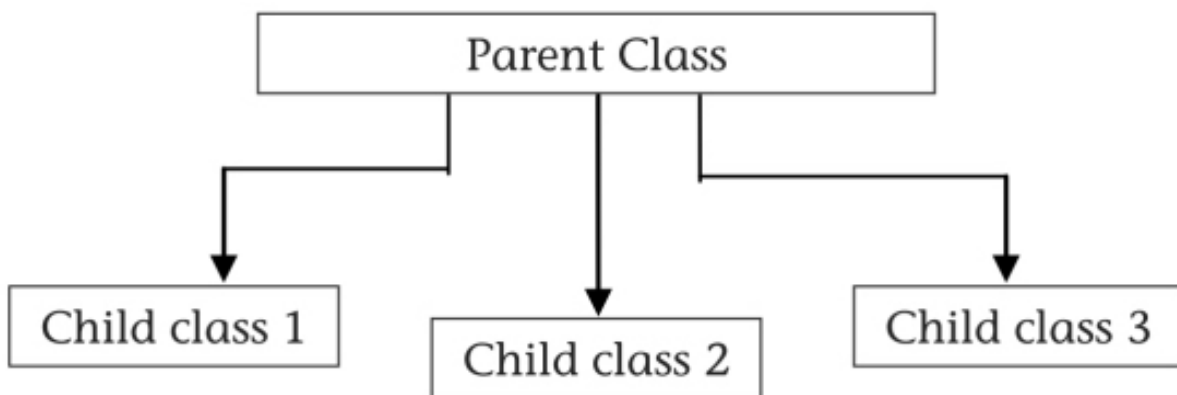
    }
    public class MultilevelInheritance
    {
        public static void main(String args[])
        {
            Subclass2 obj = new Subclass2();
            obj.print1();
            obj.print2();
            obj.print3();
        }
    }
}

```

Output:

This is Parent class
 This is Sub Class 1
 This is Sub Class 2

21.3 Hierarchical Inheritance : One Parentclass, many Childclasses



Example 83: To illustrate hierarchical inheritance

```

class ParentClass
{
    void print1()
}

```

```
    {
        System.out.println("This is Parent class");
    }
}
class SubClass1 extends ParentClass
{
    void print2()
    {
        System.out.println("This is Sub Class 1");
    }
}
class Subclass2 extends ParentClass
{
    void print3()
    {
        System.out.println("This is Sub Class 2");
    }
}
public class HierarchicalInheritance
{
    public static void main(String args[])
    {
        SubClass1 obj1 = new SubClass1();
        obj1.print1();
        obj1.print2();

        Subclass2 obj2 = new Subclass2();
        obj2.print1();
        obj2.print3();
    }
}
```

Output:

This is Parent class
This is Sub Class 1

This is Parent class
This is Sub Class 2

21.4 Multiple Inheritance:

- More than one Parent class, one Child class. Java does not directly support multiple inheritance but can be achieved by using Interfaces (discussed later)

22. Method Overriding

- Overriding is a feature that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes
- The method name and the order of arguments should be identical to that of the superclass method.
- The return type of both the method must be the same.

Example 84 : To illustrate method overriding

```
class BaseClass
{
    public void disp()
    {
        System.out.println("This is Base Class");
    }
}
class ChildClass extends BaseClass
{
    public void disp()
    {
        System.out.println("This is Child Class");
    }
}
```

```

    }

    public class MethodOverriding
    {
        public static void main( String args[])
        {

            BaseClass obj = new BaseClass();
            obj.disp();

            BaseClass obj2 = new ChildClass();
            obj2.disp();
        }
    }

```

Output:

```

This is Base Class
This is Child Class

```

23. super keyword

- The **super** keyword refers to the objects of super class (parent class).
- **super** keyword is used to call method of super(parent) class that is overridden in the subclass(child class)

Example 85 : To illustrate simple overridden program without super keyword.

```

class Parent
{
    public void print1()
    {
        System.out.println("This is Parent Class");
    }
}

```

```

    }
class Child extends Parent
{
    public void print1()
    {
        System.out.println("This is Child Class");
    }
    public void check()
    {
        print1();
    }
}

public class NoSuperClass
{
    public static void main(String[] args)
    {
        Child obj = new Child();
        obj.check();
    }
}

```

Output:

This is Child Class

- In the above program, both super class and subclass have method with the same name and object obj of child class is created.
- When method check() is called, it executes print1() method of child class because print1() method of child class overrides the print1() method of parent class.

Example 86: To illustrate overridden program with super keyword.

```

class Parent

```

```

{
    public void print1()
    {
        System.out.println("This is Parent Class");
    }
}
class Child extends Parent
{
    public void print1()
    {
        System.out.println("This is Child Class");
    }
    public void check()
    {
        super.print1();
    }
}

public class SuperClass
{
    public static void main(String[] args)
    {
        Child obj = new Child();
        obj.check();
    }
}

```

Output:

This is Parent Class

In the above program, super keyword is used to call overridden method of parent class.

Use of [super](#) keyword when super(parent) class and subclass(child) have same data members.

Example 87: To illustrate the program when super class and subclass have same data members without super keyword.

```
class Parent
{
    int age=18;
}
class Child extends Parent
{
    int age=20;
    public void print1()
    {
        System.out.println(("Age mentioned = " +age);
    }
}

public class SuperClass1
{
    public static void main(String[] args)
    {
        Child obj = new Child();
        obj.print1();
    }
}
```

Output:

Age mentioned = 20

- In the above program, both super class and subclass have same data member age and object obj of child class is created.
- When method print1() is called, it gives output of 20.

Example 88: To illustrate the program when super class and subclass have same data members with super keyword.

```
class Parent
{
    int age=18;
}
class Child extends Parent
{
    int age=20;

    public void print1()
    {
        System.out.println("Age mentioned = " +super.age);
    }
}
public class SuperClass2
{
    public static void main(String[] args)
    {
        Child obj = new Child();
        obj.print1();
    }
}
```

Output:

Age mentioned = 18

In the above program, the data member of super (parent) class is accessed in subclass (child) by using **super** keyword.

Use of **super** keyword to access super (parent) class constructors (with or without arguments).

Example 89: To access super(parent) class constructors where super() is implicitly provided by compiler.

```
class Parent
{
    Parent()
    {
        System.out.println("This is Parent Class constructor");
    }
}
class Child extends Parent
{
    Child()
    {
        System.out.println("This is Child Class constructor");
    }
    Child(String st)
    {
        System.out.println("This is Child Class constructor with parameters");
    }
}
public class SuperClass3
{
    public static void main(String[] args)
    {
        Child obj = new Child();
        Child obj2 = new Child("test");
    }
}
```

Output:

This is Parent Class constructor

This is Child Class constructor

This is Parent Class constructor

This is Child Class constructor with parameters

- In the above program, constructor is not explicitly invoked by Parent class constructor.
- In such case, java compiler automatically adds `super()` to call no argument constructor of parent class.
- Object of Child class `obj` is created which implicitly invokes the Parent class with no argument constructor.
- The second object `obj2` is created by Child class using argument constructor which invoke child class (with arguments) which will invoke Parent class with no argument constructor.
- First parent class constructor executed then child class constructor is executed.
- If a parent class with no argument constructor is invoked, it will give compile time error.

Example 90: To access `super(parent)` class constructors using where `super()` explicitly.

```
class Parent
{
    Parent()
    {
        System.out.println("This is Parent Class constructor");
    }
    Parent(int i)
    {
        System.out.println("This is Parent Class constructor with arguments");
    }
}
```

```
class Child extends Parent
{
    Child()
    {
        super(20);
        System.out.println("This is Child Class constructor");
    }

}

public class SuperClass4
{
    public static void main(String[] args)
    {
        Child obj = new Child();
    }
}
```

Output:

```
This is Parent Class constructor with arguments
This is Child Class constructor
```

- In the above program, `super()` is explicitly used to call Parent class with arguments.
- In such case, java compiler does not call the default Parent class constructor with no argument
- `super()` must be the first statement in constructor otherwise it will give compilation error.

24. final

The `final` keywords in java can be used with variables, methods and class.

- Value of `final` declared variables cannot be changed.
- `final` declared methods cannot be override

- **final** declared class cannot be extended.

Example 91 : To illustrate final keyword with variables.

```
class Keyfinal
{
    final int VAL=80;

    void chgFinal()
    {
        VAL=101;
        System.out.println(VAL);
    }
    public static void main(String args[])
    {
        Keyfinal obj=new Keyfinal();
        obj.chgFinal();
    }
}
```

Output: (compile time error)

```
Keyfinal.java:6: error: cannot assign a value to final variable VAL
    VAL=101;
    ^
1 error
```

Example 92 : To illustrate final keyword with method.

```
class BaseClass
{
    final void disp()
    {
```

```

        System.out.println("This is Base Class");
    }
}
class ChildClass extends BaseClass
{
    public void disp()
    {
        System.out.println("This is Child Class");
    }
}
public class FinalMethod
{
    public static void main( String args[])
    {
        BaseClass obj = new BaseClass();
        obj.disp();

        BaseClass obj2 = new ChildClass();
        obj2.disp();
    }
}

```

Output: (compile time error)

```

FinalMethod.java:10: error: disp() in ChildClass cannot override disp()
in
BaseClass
    public void disp()
                ^
    overridden method is final
1 error

```

Example 93: To illustrate final keyword with Class.

```

final class Parent

```

```

{
    public void print1()
    {
        System.out.println("This is Parent Class");
    }
}
class Child extends Parent
{
    public void print2()
    {
        System.out.println("This is Child Class");
    }
}
public class FinalClass
{
    public static void main(String[] args)
    {
        Child obj = new Child();
        obj.print1();
        obj.print2();
    }
}

```

Output: (compile time error)

```

FinalClass.java:9: error: cannot inherit from final Parent
class Child extends Parent
        ^
1 error

```

25. Abstract Class

- These classes are declared using **abstract** keyword and cannot be instantiated.
- To access the abstract class, it should be inherited from other class.

Example 94 : To illustrate abstract class

```
abstract class BaseClass
{
    public void disp()
    {
        System.out.println("This is abstract Base Class");
    }
}
class ChildClass extends BaseClass
{
    public void disp1()
    {
        System.out.println("This is Child Class");
    }
}
public class AbstractClass
{
    public static void main( String args[])
    {
        // object cannot be created of abstract class

        ChildClass obj = new ChildClass();
        obj.disp();
        obj.disp1();
    }
}
```

Output:

```
This is abstract Base Class
This is Child Class
```

26. Abstract Method

- These methods does not have body and can only be used in abstract class.
- Abstract class can have both regular methods as well as abstract methods.

Example 95: To illustrate abstract class with abstract method.

```
abstract class BaseClass
{
    public abstract void disp();
}
class ChildClass extends BaseClass
{
    public void disp()
    {
        System.out.println("This is body of abstract method defined in
child Class");
    }
}
public class AbstractMethod
{
    public static void main( String args[])
    {
        ChildClass obj = new ChildClass();
        obj.disp();
    }
}
```

Output:

This is body of abstract method defined in child Class

27. Interface

- An interface is defined much like a class.

- A class can implement several interfaces at once, whereas a class can extend only one parent class.
- It cannot be used to declare objects.
- Methods declared in interface are abstract in nature and members are constant.
- The keyword **implement** is used to implement interfaces

Example 96: To illustrate interface.

```
interface BaseClass
{
    public abstract void disp();
}

class ChildClass implements BaseClass
{
    public void disp()
    {
        System.out.println("This is body of abstract method of interface Baseclass");
    }
}

public class Interface
{
    public static void main( String args[])
    {
        ChildClass obj = new ChildClass();
        obj.disp();
    }
}
```

Output:

This is body of abstract method of interface Baseclass

Example 97 : To illustrate multiple interfaces.

```
interface BaseClass
{
    public abstract void disp();
}

interface BaseClass2
{
    public abstract void disp2();
}

class ChildClass implements BaseClass, BaseClass2
{
    public void disp()
    {
        System.out.println("This is body of abstract method of interface Baseclass");
    }
    public void disp2()
    {
        System.out.println("This is body of abstract method of interface Baseclass2");
    }
}

public class MultipleInterface
{
    public static void main( String args[])
    {
        ChildClass obj = new ChildClass();
        obj.disp();
        obj.disp2();
    }
}
```

Output:

This is body of abstract method of interface Baseclass
This is body of abstract method of interface Baseclass2

28. Arrays

- An **Array** is a collection of similar type of elements
- **Arrays** are **used** to store multiple values in a single variable, instead of declaring separate variables for each value.
- In java, **Array** is **index based** . It start with 0 means that the first element of array is stored on 0th index, second element is stored on 1st index and so on.

28.1 Single Dimensional Array

Syntax:

```
int Marks[];           // Declaration of Array
int[] Marks;

Marks=new int[5];       // Allocating memory to Array

int[] Marks= new int[5]; // combining both steps in single step

String[] subject;       // Declaration of array

String[] subject ={"English", "Hindi", "Maths", "Computer"};
// inserting values

System.out.println(subject[3]); // Accessing value of the 3rd element
                                // in subject
                                // output: Computer
```

Default assigned values of different type of Arrays in case it is not initialised.

- int=0
- boolean=false
- double=0.0

- string=null
- User defined type=null

Example 98 : To illustrate declaration & initialisation of single dimensional arrays

```
public class Array
{
    public static void main(String[] args)
    {
        String[] subjects ={"English", "Hindi", "Maths", "Computer"};
        int[] marks={80,70,90,60};

        System.out.println(subjects[3]);
        System.out.println(marks[2]);
        System.out.println("Array Length= " +subjects.length);
    }
}
```

Output:

```
Computer
90
Array Length= 4
```

- Elements of Array can be accessed using its index number.
- subjects[3] accesses the 3rd element of Array subjects
- marks[2] accesses the 2nd element of Array marks.
- length is used to know the how many elements are present in an array.

Example 99: To accept the array elements from user and print.

```
import java.util.Scanner;
class ArrayElements
{
    public static void main(String[] args)
    {
        System.out.print("Enter the length of Array : ");
        Scanner sc = new Scanner(System.in);
        int num=sc.nextInt();

        int[] arr = new int[num];
        System.out.println(); // to print a blank line
        for(int i = 0;i<arr.length;i++)
        {
            System.out.print("Enter the value of arr["+i+"] : ");
            arr[i] = sc.nextInt();
        }
        System.out.println();
        for(int i = 0;i<arr.length;i++)
        {
            System.out.println("The value of arr["+i+"] is "+arr[i]);
        }
    }
}
```

Output:

Enter the length of Array : 5

Enter the value of arr[0] : 12

Enter the value of arr[1] : 14

Enter the value of arr[2] : 67

Enter the value of arr[3] : 34

Enter the value of arr[4] : 23

The value of arr[0] is 12
The value of arr[1] is 14
The value of arr[2] is 67
The value of arr[3] is 34
The value of arr[4] is 23

Example 100: To sort an array elements in ascending order.

```
public class SortArray
{
    public static void main(String[] args)
    {
        int num[] = {13, 7, 6, 56, 21, 19, 101, 112};
        int temp,count=num.length;
        for (int i = 0; i < count; i++)
        {
            for (int j = i + 1; j < count; j++) {
                if (num[i] > num[j])
                {
                    temp = num[i];
                    num[i] = num[j];
                    num[j] = temp;
                }
            }
        }
        System.out.println("Array Elements sorted in Ascending Order:");
        for (int i = 0; i < count - 1; i++)
        {
            System.out.print(num[i] + ", ");
        }
        System.out.print(num[count - 1]);
    }
}
```

```
}  
}
```

Output:

Array Elements sorted in Ascending Order:
6, 7, 13, 19, 21, 56, 101, 112

Example 101: To sort an array elements in ascending order using methods of Array class.

```
import java.util.Arrays;  
public class ArrayMethod  
{  
    public static void main(String[] args)  
    {  
        int num[] = {13, 7, 6, 56, 21, 19, 101, 112};  
        String str[]={"computer","mouse","printer","keyboard"};  
        Arrays.sort(num);  
        Arrays.sort(str);  
        System.out.println("Integer type array elements in ascending  
order : " +Arrays.toString(num));  
        System.out.println("String type array elements in ascending  
order : " +Arrays.toString(str));  
    }  
}
```

Output:

Integer type array elements in ascending order : [6, 7, 13, 19, 21, 56, 101, 112]
String type array elements in ascending order : [computer, keyboard, mouse, printer]

- **Arrays** is a inbuilt class of util package(packages will study later) available in java library.

- The **sort()** method is used to sort array of any data type including string.
- The **toString()** method returns a string representation of the contents of the specified array.

Example 102 : To accept the elements of an array from user and reverse.

```
import java.util.Scanner;
public class ReverseArray
{
    public static void main(String args[])
    {
        int i=0, j=0,temp;
        System.out.print("Enter the length of Array : ");
        Scanner sc = new Scanner(System.in);
        int num=sc.nextInt();

        int[] arr = new int[num];
        System.out.println();
        for( i = 0;i<arr.length;i++)
        {
            System.out.print("Enter the value of arr["+i+"] : ");
            arr[i] = sc.nextInt();
        }
        System.out.println();
        j = i - 1;
        i = 0;

        while(i<j)
        {
            temp = arr[i];
            arr[i] = arr[j];
```

```

        arr[j] = temp;
        i++;
        j--;
    }

    System.out.print("Reversed array: ");
    for(i=0; i<arr.length; i++)
    {
        System.out.print(arr[i]+ " ");
    }
}
}

```

Output:

Enter the length of Array : 5

Enter the value of arr[0] : 56

Enter the value of arr[1] : 64

Enter the value of arr[2] : 12

Enter the value of arr[3] : 9

Enter the value of arr[4] : 23

Reversed array: 23 9 12 64 56

Example 103: To print sum of all the elements of an array.

```

import java.util.Scanner;
public class ArraySum
{
    public static void main(String[] args)
    {
        int sum = 0;
    }
}

```

```
System.out.print("Enter the length of Array : ");
Scanner sc = new Scanner(System.in);
int num=sc.nextInt();

int[] arr = new int[num];
System.out.println();
for(int i = 0;i<arr.length;i++)
{
    System.out.print("Enter the value of arr["+i+"] : ");
    arr[i] = sc.nextInt();
}
//Loop to calculate sum of elements
for (int i = 0; i < arr.length; i++)
{
    sum = sum + arr[i];
}
System.out.println("Sum of all the elements of an array: " +
sum);
}
```

Output:

```
Enter the length of Array : 4

Enter the value of arr[0] : 3
Enter the value of arr[1] : 5
Enter the value of arr[2] : 7
Enter the value of arr[3] : 9
Sum of all the elements of an array: 24
```

Example 104: To check for the elements of array whether they are even or odd.

```
import java.util.Scanner;
public class ArrayOddEven
{
    public static void main(String args[])
    {

        System.out.print("Enter the length of Array : ");
        Scanner sc = new Scanner(System.in);
        int num=sc.nextInt();

        int[] arr = new int[num];
        System.out.println();
        for(int i = 0;i<arr.length;i++){
            System.out.print("Enter the value of arr["+i+"] : ");
            arr[i] = sc.nextInt();

        }
        System.out.println("Odd Numbers:");
        for(int i=0;i<arr.length;i++)
        {
            if(arr[i]%2!=0)
            {
                System.out.println(arr[i] + " ");
            }
        }
        System.out.println("Even Numbers:");
        for(int i=0;i<arr.length;i++)
        {
            if(arr[i]%2==0)
            {
                System.out.println(arr[i]);
            }
        }
    }
}
```

```
}  
}  
}
```

Output:

Enter the length of Array : 6

Enter the value of arr[0] : 12

Enter the value of arr[1] : 23

Enter the value of arr[2] : 43

Enter the value of arr[3] : 22

Enter the value of arr[4] : 78

Enter the value of arr[5] : 89

Odd Numbers:

23

43

89

Even Numbers:

12

22

78

Example 105 : To print the largest element of the given array.

```
public class ArrayLargestElement  
{  
    public static void main(String[] args)  
    {  
        int [] arr = new int [] {45, 10, 70, 100, 25};  
        int large = arr[0];  
        for (int i = 0; i < arr.length; i++)  
        {
```

```

        if(arr[i] > large)
            large = arr[i];
    }
    System.out.println("Largest element present in the array= " +
large);
}
}

```

Output:

Largest element present in the array= 100

Example 106: To search for an element in an array.

```

import java.util.Scanner;
public class SearchArrElement
{
    public static void main(String[] args)
    {
        int [] arr = new int [] {45, 10, 70, 100, 25};
        Scanner obj = new Scanner(System.in);
        System.out.print("Enter the element to search : ");
        int num= obj.nextInt();

        boolean found = false;
        for (int n : arr)
        {
            if (n == num)
            {
                found = true;
                break;
            }
        }
    }
}

```

```
        if(found)
            System.out.println(num + " is present in the array.");
        else
            System.out.println(num + " is not found in the array.");
    }
}
```

Output:

First Case

Enter the element to search : 35
35 is not found in the array.

Second Case

Enter the element to search : 45
45 is present in the array.

Example 107: To insert an element at any specific position of the array.

```
import java.util.Scanner;
class InsertArray
{
    public static void main(String[] args)
    {
        int length, pos,num;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter length of an array:");
        length = sc.nextInt();
        int arr[] = new int[length+1];
        System.out.println("Enter the elements of the Array:");
        for(int i = 0; i < length; i++)
        {
```

```

        arr[i] = sc.nextInt();
    }
    System.out.print("Enter the position where you want to insert
an element:");
    pos = sc.nextInt();
    System.out.print("Enter the element which you want to
insert:");
    num = sc.nextInt();
    for(int i = length-1; i >= (pos-1); i--)
    {
        arr[i+1] = arr[i];
    }
    arr[pos-1] = num;
    System.out.print("\nArray after inserting the element : ");
    for(int i = 0; i <=length; i++)
    {
        System.out.print(arr[i]+" ");
    }
}
}

```

Output:

```

Enter length of an array:4
Enter the elements of the Array:
89
98
76
65
Enter the position where you want to insert an element:2
Enter the element which you want to insert:12

Array after inserting the element : 89 12 98 76 65

```


Example 108 : To concatenate two arrays.

```
import java.util.Arrays;
public class ConcatArrays1
{
    public static void main(String[] args)
    {
        int[] arr1 = {11, 22, 33};
        int[] arr2 = {44, 55, 66};

        int len = arr1.length + arr2.length;

        int[] res = new int[len];
        int count = 0;
        for (int i : arr1)
        {
            res[count] = i;
            count++;
        }

        for (int j : arr2)
        {
            res[count] = j;
            count++;
        }

        System.out.println("Result after concatenation of both arrays :
" + Arrays.toString(res));
    }
}
```

Output:

Result after concatenation of both arrays : [11, 22, 33, 44, 55, 66]

Example 109 : To concatenate two arrays using arraycopy() method.

```
import java.util.Arrays;
public class ConcatArrays
{
    public static void main(String[] args)
    {
        int[] arr1 = {11, 22, 33};
        int[] arr2 = {44, 55, 66};

        int len1 = arr1.length;
        int len2 = arr2.length;
        int[] res = new int[len1 + len2];

        System.arraycopy(arr1, 0, res, 0, len1);
        System.arraycopy(arr2, 0, res, len1, len2);
        System.out.println("Result after concatenation of both arrays :
"+Arrays.toString(res));
    }
}
```

Output:

Result after concatenation of both arrays : [11, 22, 33, 44, 55, 66]

28.2 Multi Dimensional Array

Syntax:

<pre>int[] [] myList={ {1,2,3,4}, {5,6,7,8}}</pre>	<pre>// Declaration of two dimensional Array</pre>
--	--

Example 110: To illustrate declaration & initialisation of multi dimensional arrays

```
public class ArrayMultiDimensional
{
    public static void main(String[] args)
    {
        int[][] num = {{11,62,23,14},{15,36,17,18},{49,10,11,12}};
        for(int i=0; i<=2; i++)
        {
            for(int j=0; j<=3; j++)
            {
                System.out.print(num[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Output:

```
11 62 23 14
15 36 17 18
49 10 11 12
```

Example 111: To illustrate addition of two matrices

```
public class MultiArrayAdd
{
    public static void main(String args[])
    {
        //creating two matrices
```

```

int arr1[][]={{1,2,4},{3,2,1},{1,5,4}};
int arr2[][]={{7,3,3},{4,2,7},{1,1,1}};
//creating another matrix to store the sum of two matrices
int sum[][]=new int[3][3];
for(int i=0;i<=2;i++)
{
    for(int j=0;j<=2;j++)
    {
        sum[i][j]=arr1[i][j]+arr2[i][j];
        System.out.print(sum[i][j]+" ");
    }
    System.out.println();
}
}
}

```

Output:

```

8 5 7
7 4 8
2 6 5

```

Example 112 : To illustrate subtraction of two matrices.

```

public class MultiArraySub
{
    public static void main(String args[])
    {
        int arr1[][]={{7,8,4},{9,4,6}};
        int arr2[][]={{6,4,2},{4,2,4}};
        int sub[][]=new int[2][3];

        for(int i=0;i<=1;i++)

```

```

    {
    for(int j=0;j<=2;j++)
    {
        sub[i][j]=arr1[i][j]-arr2[i][j];
        System.out.print(sub[i][j]+" ");
    }
    System.out.println();
    }
}
}

```

Output:

```

1 4 2
5 2 2

```

Example 113 : To illustrate multiplication of two matrices.

```

public class MultiArrayProduct
{
    public static void main(String args[])
    {
        int arr1[][]={{7,8,4},{9,4,6}};
        int arr2[][]={{6,4,3},{4,4,4}};
        int multi[][]=new int[2][3];
        for(int i=0;i<=1;i++)
        {
            for(int j=0;j<=2;j++)
            {
                multi[i][j]=arr1[i][j]*arr2[i][j];
                System.out.print(multi[i][j]+" ");
            }
            System.out.println();
        }
    }
}

```

```
}  
}  
}
```

Output:

```
42 32 12  
36 16 24
```

Example 114 : To check whether the matrices are equal.

```
public class EqualMatrixCheck  
{  
    public static void main(String[] args)  
    {  
        int rownum1, colnum1, rownum2, colnum2;  
        boolean flag = true;  
  
        int arr1[][] = { {4, 2, 6}, {6, 3, 6},{5, 6, 7}};  
        int arr2[][] = { {3, 4, 5},{1, 2, 3},{5, 6, 7}};  
  
        rownum1 = arr1.length;  
        colnum1 = arr1[0].length;  
  
        rownum2 = arr2.length;  
        colnum2 = arr2[0].length;  
  
        if(rownum1 != rownum2 || colnum1 != colnum2)  
        {  
            System.out.println("Matrices are not equal");  
        }  
        else
```

```

    {
        for(int i = 0; i < rownum1; i++)
        {
            for(int j = 0; j < colnum1; j++)
            {
                if(arr1[i][j] != arr2[i][j])
                {
                    flag = false;
                    break;
                }
            }
        }
        if(flag)
            System.out.println("Matrices are equal");
        else
            System.out.println("Matrices are not equal");
    }
}

```

Output:

Matrices are not equal

Example 115 : To transpose matrix.

```

public class MatrixTranspose
{
    public static void main(String args[])
    {
        int arr1[][]={{1,2,3},{4,5,6},{7,8,9}};
        int trans[][]=new int[3][3];
        System.out.println("Original Matrix:");
        for(int i=0;i<3;i++)

```

```
{
    for(int j=0;j<3;j++)
    {
        System.out.print(arr1[i][j]+" ");
    }
    System.out.println();
}

    for(int i=0;i<3;i++)
    {
        for(int j=0;j<3;j++)
        {
            trans[i][j]=arr1[j][i];
        }
    }
    System.out.println("Matrix after transpose:");
    for(int i=0;i<3;i++)
    {
        for(int j=0;j<3;j++)
        {
            System.out.print(trans[i][j]+" ");
        }
        System.out.println();
    }
}
```

Output:

Original Matrix:

1 2 3

4 5 6

7 8 9

Matrix after transpose:


```
1 4 7  
2 5 8  
3 6 9
```

29. Java ArrayList

- The limitation with an array is that it always has its size specified (fixed length). One cannot add elements more than the size specified nor the size is reduced when the elements are removed from the array.
- Java ArrayList has no fixed size limit. ArrayList can dynamically increase or decrease its size on addition and removal of elements.
- Java ArrayList allows to randomly access the list.
- It is found in “java.util” package.

Syntax:

```
ArrayList<Object> obj = new ArrayList <Object><>
```

29.1 Commonly used Java ArrayList methods

add()

- This method is used to add elements to ArrayList

Example 116 : To illustrate add() method of Java ArrayList String type.

```
import java.util.ArrayList;  
public class ArrayListadd  
{  
    public static void main(String[] args)  
    {
```

```

ArrayList<String> mylist1 = new ArrayList<String>();
mylist1.add("keyboard");
mylist1.add("mouse");
mylist1.add("printer");
mylist1.add("speaker");
System.out.println("Element of mylist1 is : " +mylist1);
}
}

```

Output:

Element of mylist1 is : [keyboard, mouse, printer, speaker]

Example 117 : To illustrate add() method of Java ArrayList Integer type.

```

import java.util.ArrayList;
public class ArrayListadd2
{
    public static void main(String[] args)
    {
        ArrayList<Integer> mylist2 = new ArrayList<Integer>();
        mylist2.add(21);
        mylist2.add(34);
        mylist2.add(43);
        mylist2.add(12);
        System.out.println("Element of mylist1 is : " +mylist2);
    }
}

```

Output:

Element of mylist1 is : [21, 34, 43, 12]

size()

- This method is used to get the number of elements in the ArrayList.

get()

- This method is used to access element of ArrayList with the help of index number.
- Array index starts with 0.

Example 118: To illustrate size() and get() method of Java ArrayList.

```
import java.util.ArrayList;
public class ArrayListget
{
    public static void main(String[] args)
    {
        ArrayList<String> mylist1 = new ArrayList<String>();
        mylist1.add("keyboard");
        mylist1.add("mouse");
        mylist1.add("printer");
        mylist1.add("speaker");
        System.out.println("Size of the mylist1 is = " +mylist1.size());
        System.out.println("Element at index 2 of mylist1 is : "
+mylist1.get(2));
    }
}
```

Output:

```
Size of the mylist1 is = 4
Element at index 2 of mylist1 is : printer
```

remove()

- This method is used to remove an element from the ArrayList.

clear()

- This method is used to remove all the elements from the ArrayList

Example 119 : To illustrate remove() & clear() methods of Java ArrayList.

```
import java.util.ArrayList;
public class ArrayListremove
{
    public static void main(String[] args)
    {
        ArrayList<String> mylist1 = new ArrayList<String>();
        mylist1.add("keyboard");
        mylist1.add("mouse");
        mylist1.add("printer");
        mylist1.add("speaker");
        System.out.println("Element of mylist1 is : " +mylist1);
        mylist1.remove(2);
        System.out.println("Element of mylist1 after removing element at
index 2 is : " +mylist1);
        mylist1.clear();
        System.out.println("Element of mylist1 after removing all the
elements : " +mylist1);
    }
}
```

Output:

```
Element of mylist1 is : [keyboard, mouse, printer, speaker]
Element of mylist1 after removing element at index 2 is : [keyboard,
mouse, speaker]
Element of mylist1 after removing all the elements : []
```

contains()

- This method will return true if the element is present in the ArrayList

Example 120 : To illustrate contains() methods of Java ArrayList.

```
import java.util.ArrayList;
public class ArrayListcontains
{
    public static void main(String[] args)
    {
        ArrayList<String> mylist1 = new ArrayList<String>();
        mylist1.add("keyboard");
        mylist1.add("mouse");
        mylist1.add("printer");
        mylist1.add("speaker");
        System.out.println("Element of mylist1 is : " +mylist1);
        System.out.println(mylist1.contains("speaker"));
    }
}
```

Output:

```
Element of mylist1 is : [keyboard, mouse, printer, speaker]
true
```

set()

- This method is used to modify element of the ArrayList

isEmpty()

- This method is used to check whether the ArrayList is empty

Example 121 : To illustrate set() and isEmpty() methods of Java ArrayList.

```
import java.util.ArrayList;
```

```

public class ArrayListset
{
    public static void main(String[] args)
    {
        ArrayList<String> mylist1 = new ArrayList<String>();
        mylist1.add("keyboard");
        mylist1.add("mouse");
        mylist1.add("printer");
        mylist1.add("speaker");
        System.out.println("Element of mylist1 is : " +mylist1);
        mylist1.set(2,"plotter");
        System.out.println("Element of mylist1 after modification : "
+mylist1);
        System.out.println("Is mylist1 is empty : " +mylist1.isEmpty());
    }
}

```

Output:

```

Element of mylist1 is : [keyboard, mouse, printer, speaker]
Element of mylist1 after modification : [keyboard, mouse, plotter,
speaker]
Is mylist1 is empty : false

```

29.2 Iterator

- iterator is used to iterate ArrayList in Java.
- It is the implementation of the Iterator interface.

Example 122 : To illustrate iteration through ArrayList using the Iteration interface.

```

import java.util.*;

```

```
public class ArrayListIterator
{
    public static void main(String[] args)
    {
        ArrayList<String> mylist1 = new ArrayList<String>();
        mylist1.add("keyboard");
        mylist1.add("mouse");
        mylist1.add("printer");
        mylist1.add("speaker");

        System.out.println("Elements of mylist1 :");
        Iterator obj = mylist1.iterator();
        while (obj.hasNext())
        {
            System.out.println(obj.next());
        }
    }
}
```

Output:

```
Elements of mylist1 :
keyboard
mouse
printer
speaker
```

- In the above program **hasNext()** and **next()** method is used.
- The **hasNext()** method returns true if there are more elements in the ArrayList else will return false.
- The **next()** method returns the next element in the ArrayList.

Example 123 : To illustrate iteration through ArrayList using for loop.

```
import java.util.*;
public class ArrayListForLoop
{
    public static void main(String[] args)
    {
        ArrayList<String> mylist1 = new ArrayList<String>();
        mylist1.add("keyboard");
        mylist1.add("mouse");
        mylist1.add("printer");
        mylist1.add("speaker");

        System.out.println("Elements of mylist1 :");
        for(int i=0;i<mylist1.size();i++)
        {
            System.out.println(mylist1.get(i));
        }
    }
}
```

Output:

```
Elements of mylist1 :
keyboard
mouse
printer
speaker
```

Example 124 : To illustrate iteration through ArrayList using for each loop.


```
import java.util.*;
public class ArrayListForEachLoop
{
    public static void main(String[] args)
    {
        ArrayList<String> mylist1 = new ArrayList<String>();
        mylist1.add("keyboard");
        mylist1.add("mouse");
        mylist1.add("printer");
        mylist1.add("speaker");
        System.out.println("Elements of mylist1 :");
        for (String n : mylist1)
        {
            System.out.println(n);
        }
    }
}
```

Output:

```
Elements of mylist1 :
keyboard
mouse
printer
speaker
```

Example 125 : To illustrate iteration through ArrayList using while loop

```
import java.util.*;
public class ArrayListWhileLoop
{
    public static void main(String[] args)
```

```
{
    ArrayList<String> mylist1 = new ArrayList<String>();
    mylist1.add("keyboard");
    mylist1.add("mouse");
    mylist1.add("printer");
    mylist1.add("speaker");
    System.out.println("Elements of mylist1 :");
    int i = 0;
    while (mylist1.size() > i)
    {
        System.out.println(mylist1.get(i));
        i++;
    }
}
```

Output:

```
Elements of mylist1 :
keyboard
mouse
printer
speaker
```

Iterator.remove()

- This method is used to remove element of ArrayList during iteration.

Example 126 : To illustrate Iterator.remove() method of ArrayList in Java.

```
import java.util.*;
public class ArrayListIteratorRemove
{
```

```

public static void main(String[] args)
{
    ArrayList<Integer> mylist2 = new ArrayList<Integer>();
    mylist2.add(15);
    mylist2.add(18);
    mylist2.add(12);
    mylist2.add(21);
    mylist2.add(21);
    System.out.println("Element of mylist2 is : " +mylist2);
    Iterator obj=mylist2.iterator();

    while (obj.hasNext())
    {
        int age = (Integer)obj.next();
        if (age <18)
            obj.remove();
    }
    System.out.println("Element of mylist2 after modification : "
+mylist2);
}
}

```

Output:

```

Element of mylist2 is : [15, 18, 12, 21, 21]
Element of mylist2 after modification : [18, 21, 21]

```

addAll()

- This method is used to join elements of two ArrayList in one ArrayList.

Example 127 : To illustrate addAll() method to join two ArrayList in one ArrayList.

```

import java.util.ArrayList;

```

```
public class ArrayListJoin
{
    public static void main(String [] args)
    {
        ArrayList<String> mylist1 = new ArrayList<String>(); // First
ArrayList
        mylist1.add("keyboard");
        mylist1.add("mouse");
        mylist1.add("printer");
        mylist1.add("speaker");

        ArrayList<String> mylist2 = new ArrayList<String>(); // Second
ArrayList
        mylist2.add("java");
        mylist2.add("c++");
        mylist2.add("c");
        mylist2.add("fortran");

        ArrayList<String> finallist= new ArrayList<String>(); // Final list
        finallist.addAll(mylist1);
        finallist.addAll(mylist2);

        for(String str: finallist)
        {
            System.out.println(str);
        }
    }
}
```

Output:

```
keyboard
mouse
printer
```

```
speaker  
java  
c++  
c  
fortran
```

Collection.sort()

- Collection class is present in java.util package which includes sort() method to sort elements of ArrayList.

Example 128 : To sort elements of ArrayList String type.

```
import java.util.*;  
public class ArrayListSort  
{  
    public static void main(String args[])  
    {  
        ArrayList<String> mylist1 = new ArrayList<String>();  
        mylist1.add("mouse");  
        mylist1.add("keyboard");  
        mylist1.add("speaker");  
        mylist1.add("printer");  
  
        System.out.println("mylist1 Before Sorting:");  
        for(String st: mylist1)  
        {  
            System.out.println(st);  
        }  
        Collections.sort(mylist1);  
        System.out.println("mylist1 after Sorting in ascending order:");  
        for(String st: mylist1)
```

```
    {  
        System.out.println(st);  
    }  
}  
}
```

Output:

mylist1 Before Sorting:
mouse
keyboard
speaker
printer
mylist1 after Sorting in ascending order:
keyboard
mouse
printer
speaker

Example 129 : To sort elements of ArrayList integer type.

```
import java.util.*;  
public class ArrayListSort1  
{  
    public static void main(String args[])  
    {  
        ArrayList<Integer> mylist2 = new ArrayList<Integer>();  
        mylist2.add(55);  
        mylist2.add(34);  
        mylist2.add(98);  
        mylist2.add(67);  
        mylist2.add(39);  
        mylist2.add(76);  
    }  
}
```

```
mylist2.add(81);

System.out.println("mylist2 Before Sorting:");
for(int marks: mylist2)
{
    System.out.println(marks);
}
Collections.sort(mylist2);
System.out.println("mylist2 after Sorting in ascending order:");
for(int marks: mylist2)
{
    System.out.println(marks);
}
}
```

Output:

```
mylist2 Before Sorting:
55
34
98
67
39
76
81
mylist2 after Sorting in ascending order:
34
39
55
67
76
81
98
```

30. Wrapper Class

This class come under java.util package.The Wrapper class wraps primitive data types to use them as objects. In other words it converts primitive data types into objects.

Primitive Data types with their corresponding Wrapper class

Data Type	Wrapper Class
boolean	Boolean
byte	Byte
char	Character
float	Float
int	Integer
long	Long
short	Short
double	Double

Example 130 : To convert primitive data type to Wrapper object.

```
class PrimitiveToWrapper
{
    public static void main(String[] args)
    {
        int num = 500;
        Integer wobj = Integer.valueOf(num);

        System.out.println("Value of primitive data type int = "+num);
        System.out.println("Value of Wrapper Object = "+wobj);
    }
}
```



```
}  
}
```

Output:

Value of primitive data type int = 500

Value of Wrapper Object = 500

Example 131 : To convert Wrapper class object to primitive data type

```
class WrapperToPrimitive  
{  
    public static void main(String[] args)  
    {  
        Integer wobj = new Integer(500);  
        int num = wobj.intValue();  
  
        System.out.println("Value of Wrapper Object = "+wobj);  
        System.out.println("Value of primitive data type int = "+num);  
    }  
}
```

Output:

Value of Wrapper Object = 500

Value of primitive data type int = 500

31. Strings

- In Java, a String is a sequence of characters.
- Java implements strings as objects of type String.
- In java, strings are class objects and implemented using two classes

namely, String and StringBuffer.

- Both the String and StringBuffer classes are defined in java.lang package
- Implementing strings as built in objects allow to java to provide a full complement of features that make string handling convenient in java.

Example 132: To check for the length and total number of characters in a given string.

```
public class StringCharacter
{
    public static void main(String[] args)
    {
        String st = "This is my first string program";
        int count = 0;
        int length= st.length(); // returns length of the string including
        //Counts each character excluding space

        for(int i = 0; i < length; i++) {
            if(st.charAt(i) != ' ')
                count++;
        }

        System.out.println("Length of the given string: " + length);
        System.out.println("Total number of characters in the given string
        excluding space: " + count);
    }
}
```

Output:

```
Length of the given string: 31
Total number of characters in the given string excluding space: 26
```

- The length of the string can be found with the length() method. In the

above program `string.length()` gives the length of string `st`.

- The `CharAt(n)` returns the `n`th character of the string. In the above program `st.CharAt(i)` will give the `i`th character

Example 133 : To count total number of vowel and consonant in a given string.

```
public class VowelConsonant
{
    public static void main(String[] args)
    {

        int vowel = 0, consonant = 0;
        String st = "This is my program for counting vowel and
consonant";

        st = st.toLowerCase();

        for(int i = 0; i < st.length(); i++)
        {

            if(st.charAt(i) == 'a' || st.charAt(i) == 'e' || st.charAt(i) == 'i' ||
st.charAt(i) == 'o' || st.charAt(i) == 'u') // check for vowel
            {
                vowel++;
            }
            else if(st.charAt(i) >= 'a' && st.charAt(i) <= 'z') //check for
consonant character
            {
                consonant++;
            }
        }
    }
}
```

```

        System.out.println("Number of Vowels : " + vowel);
        System.out.println("Number of Consonants " + consonant);
    }
}

```

Output:

```

Number of Vowels : 14
Number of Consonants 29

```

- The toLowerCase() method converts the string characters to lower case.
- The toUpperCase() method converts the string characters to upper case.
- In the above program string st characters are converted to lowercase using the statement st.toLowerCase().

Example 134 : To capitalize the first letter of each word in a sentence(Title Case).

```

import java.util.*;
public class StringTitleCase
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a Sentence: ");
        String str = sc.nextLine(); String str2 = "";
        Scanner scr = new Scanner(str);
        while(scr.hasNext())
        {
            String str3 = scr.next();
            str2 += Character.toUpperCase(str3.charAt(0)) +
str3.substring(1) + " ";
        }
        System.out.println("Title Case Result :"+str2.trim());
    }
}

```

```
}  
}
```

Output:

Enter a Sentence: learning java programming
Title Case Result :Learning Java Programming

Example 135: To reverse a given string.

```
public class StringReverse  
{  
    public static void main(String[] args)  
    {  
        String st = "Programming language";  
        String newst = "";  
  
        for(int i = st.length()-1; i >= 0; i--)  
        {  
            newst = newst + st.charAt(i);  
        }  
  
        System.out.println("Original string: " + st);  
        System.out.println("Reverse of given string: " + newst);  
    }  
}
```

Output:

Original string: Programming language
Reverse of given string: egaugnal gnimmargorP

Example 136: To concatenate two strings.

```
public class StringConcatenate
```

```

{
    public static void main(String[] args)
    {
        String st1 = "String ";
        String st2 = "Concatenation";
        String res = st1.concat(st2);
        System.out.println(res);
    }
}

```

Output:

String Concatenation

- The concat() method is used to concatenate two strings in java
- In the above program, string st1 and string st2 are concatenated using statement st1.concat(st2).

Example 137 : To find duplicate characters in a string.

```

public class StrDuplicate
{
    public static void main(String argu[])
    {
        String st = "Programming"; int count = 0;
        char[] ch = st.toCharArray();
        System.out.println("Duplicate characters are");
        for (int i = 0; i < st.length(); i++)
        {
            for (int j = i + 1; j < st.length(); j++)
            {
                if (ch[i] == ch[j])
                {

```

```

        System.out.println(ch[j]);
        count++;
        break;
    }
}
}
}
}
}

```

Output:

Duplicate characters are
r
g
m

Example 138: Comparison of two strings.

```

public class EqualString
{
    public static void main(String args[])
    {
        String st1 = "Java Programming";
        String st2 = "java programming";
        if(st1.equalsIgnoreCase(st2))
        {
            System.out.println("Both strings are same ignoring case");
        } else
        {
            System.out.println("Both strings are not same.");
        }

        if(st1.equals(st2))
        {

```

```

        System.out.println("Both strings are same.");
    }
    else
    {
        System.out.println("Both strings are not same");
    }
}
}

```

Output:

Both strings are same ignoring case
Both strings are not same

- Java equalsIgnoreCase() method is used to compare two strings ignoring case.
- In the above program st1 & st2 are compared ignoring case using the statement st1.equalsIgnoreCase(st2).
- Java equals() method is used to compare two strings (case sensitive) .
- In the above program st1 & st2 are compared using the statement st1.equals(st2)

Example 139: Replacing old character with new character.

```

public class CharReplace
{
    public static void main(String args[])
    {
        String st = new String("java programming");
        System.out.println("Original String : " +st);
        System.out.print("After replacing all a with e : ");
        System.out.print(st.replace('a', 'e'));
    }
}

```



```
}  
}
```

Output:

Original String : java programming

After replacing all a with e : jeve progremming

Example 140: To check for string palindrome.

```
public class StringPalindrome  
{  
    public static void main(String[] args)  
    {  
        String st = "Madam";  
        boolean flag = true;  
  
        st = st.toLowerCase();  
  
        for(int i = 0; i < st.length()/2; i++)  
        {  
            if(st.charAt(i) != st.charAt(st.length()-i-1))  
            {  
                flag = false;  
                break;  
            }  
        }  
        if(flag)  
            System.out.println("Given string is palindrome");  
        else  
            System.out.println("Given string is not a palindrome");  
    }  
}
```

Output:

Given string is palindrome

31.1 Some more most commonly used Strings methods:

- **st1.trim()**: remove white spaces at the beginning and end from the string st1.
- **st1.indexOf('n')** : returns the position of the first occurrence of n in the string st1.
- **st1.indexOf('n', p)** returns the position of 'n' that occurs after pth position in the string st1.
- **boolean isEmpty()** : checks whether the string is empty.
- **st1.substring(m)** : returns the substring starting from mth character.
- **st1.substring(m, n)** : returns the substring from mth (not including mth) character upto nth

32. Java Package

- A collection of variety of related classes, interfaces and sub packages.
- Packages helps in proper organization of classes and interfaces.
- It is used in java for controlled access, name conflicts, re-usability and data encapsulation.
- Package inside another package is known as subpackage.
- The **import** keyword is used to use a package from library.
- Fully qualified class name of the class used in program can also be used to access that class.

- Two types of packages are offered by Java.

32.1 Java API packages(Built in Package)

- Java API stands for Application Programming Interface.
- It is collection of large number of predefined classes included in Java Development kit.
- One can import a single class along with its methods and attributes or the complete package with all the classes defined in that particular package.
- Some of the commonly used Java inbuilt packages are java.io, java.lang, java. util, java.awt etc.

Syntax: To import a single class

```
import package.subpackage.Class;  
Ex. Import java.util.Scanner; // java is package, util is sub package and  
Scanner is class of util
```

Syntax: to import the whole package

```
import package.name.*;  
Ex. import java.util.*; // import all the classes and interfaces of sub  
package util.  
import package name.* // import all the classes & interfaces of the  
package but not the subpackages
```

Example 141: Accessing Scanner class of Java API package using import keyword

```
import java.util.Scanner;  
public class ScannerClass  
{  
    public static void main(String[] args)  
    {  
        Scanner obj = new Scanner(System.in);
```

```
System.out.print("Enter your name ");
String name = obj.nextLine();
System.out.print("Enter your Company name ");
String cname = obj.nextLine();

System.out.println( "Your name is " + name);
System.out.println( "Your company name is " + cname);
}
}
```

Output:

```
Enter your name : Mukesh Kumar
Enter your Company name : Reliance
Your name is Mukesh Kumar
Your company name is Reliance
```

Example 142: Accessing Scanner class of Java API package using fully qualified class name.

```
public class ScannerClass1
{
    public static void main(String[] args)
    {
        java.util.Scanner obj = new java.util.Scanner(System.in); //using
fully qualified name
        System.out.print("Enter your name : ");
        String name = obj.nextLine();
        System.out.print("Enter your Company name : ");
        String cname = obj.nextLine();

        System.out.println( "Your name is " + name);
        System.out.println( "Your company name is " + cname);
    }
}
```

```
}  
}
```

Output:

```
Enter your name : Mukesh Kumar  
Enter your Company name : Reliance  
Your name is Mukesh Kumar  
Your company name is Reliance
```

32.2 User defined Packages

- These packages are created by user.
- Package name should be written in lower case.
- To create a user defined package, `package` keyword is used

Syntax:

```
package package name;  
Ex : package mypack;
```

Step for creating API package

- Create the package using the keyword `package` as:

```
package package name  
package myPack1
```

- Create a subdirectory under the directory where the main source files are stored
- The subdirectory name must be same as package name.
- Declare a public class for the package as

```

package myPack1;
public class MyDisp
{
    public void display()
    {
        System.out.println("This is the example of API package");
    }
}

```

- Save this file as classname.java in the subdirectory created
- Compile the file as

```
javac MyDisp.java
```

- This creates a .class file in the subdirectory
- Now we can use this class MyDisp in our program.

```

import myPack1.MyDisp;
public class Print
{
    public static void main(String args[])
    {
        MyDispobj=new MyDisp();
        obj.display();
    }
}

```

33. Access Modifiers in Java

Access Modifier	Scope
Private	within class
Default	within class & package
Protected	within class & package, outside package but only by subclass (using inheritance)
Public	within class & package, outside package

33.1 Private Access Modifier

Example 143 : Program Without Private access modifier.

```
class Private
{
    void disp()
    {
        System.out.println("Without Private Access Modifier");
    }
}
public class Access1
{
    public static void main(String args[])
    {
        Private obj = new Private();
        obj.disp();
    }
}
```

Output:

Without Private Access Modifier

Example 144 : Program with Private access modifier added

```
class Private
{
    private void disp()
    {
        System.out.println("Private Access Modifier");
    }
}
```

```

    }
}
public class Access1
{
    public static void main(String args[])
    {
        Private obj = new Private();
        obj.disp();
    }
}

```

Output: (compile time error)

```

Access1.java:12: error: disp() has private access in Private
    obj.disp();
        ^
1 error

```

33.2 Default Access Modifier

Example 145: Without Default access modifier

```

package myPack1;
public class MyDisp
{
    public void display()
    {
        System.out.println("This is the example of API package");
    }
}

import myPack1.MyDisp;
public class Print

```



```

{
    public static void main(String args[])
    {
        MyDisp obj=new MyDisp();
        obj.display();
    }
}

```

Output:

This is the example of API package

Example 146 : With Default access modifier.

```

package myPack1;
public class MyDisp
{
    void display()    // no modifier mentioned is treated as default.
    {
        System.out.println("This is the example of API package");
    }
}

```

```

import myPack1.MyDisp;
public class Print
{
    public static void main(String args[])
    {
        MyDisp obj=new MyDisp();
        obj.display();
    }
}

```

Output: (compile time error)

Print.java:7: error: display() is not public in MyDisp; cannot be accessed from outside package

```
    obj.display();  
      ^
```

1 error

33.3 Protected Access Modifier

Example 147 : To illustrate Protected access modifier

```
package myPack1;  
public class MyDisp  
{  
    protected void display()  
    {  
        System.out.println("This is the example of API package");  
    }  
}
```

Example 148 : Accessing without inheritance

```
import myPack1.MyDisp;  
public class Print1  
{  
    public static void main(String args[])  
    {  
        MyDisp obj=new MyDisp();  
        obj.display();  
    }  
}
```

Output: (compile time error)

```
Print1.java:7: error: display() has protected access in MyDisp
    obj.display();
      ^
1 error
```

Example 149: Accessing using inheritance

```
import myPack1.*;
public class Print1 extends MyDisp
{
    public static void main(String args[])
    {
        Print1 obj=new Print1();
        obj.display();
    }
}
```

Output:

This is the example of API package

33.4 Public Access Modifier

Example 150: To illustrate Public access modifier

```
package myPack1;
public class MyDisp
{
    public void display()
    {
        System.out.println("This is the example of API package");
    }
}
```

```
}  
}
```

```
import myPack1.MyDisp;  
public class Print  
{  
    public static void main(String args[])  
    {  
        MyDisp obj=new MyDisp();  
        obj.display();  
    }  
}
```

Output:

This is the example of API package

34. Exception handling in Java

An exception is an unwanted event that arises during the execution of a program. It disrupts the normal flow of a program. Exception handling is possible in java.

It is one of the important features of java to handle the runtime errors caused by exception. The error handling code is developed to Hit the exception (detecting problem), Throw the exception (informing about the error), catch the exception (receive error details) and finally handle the exception (take appropriate actions).

Exceptions are different from Errors as errors are irrecoverable whereas exceptions are handled by taking necessary actions.

34.1 Exceptions in java can be categorized into two types:

34.1.1 Checked exceptions

- Checked exceptions are extended from the `java.lang.Exception` class
- These exceptions are checked at compile time.
- If some code within a method throws a checked exception, then the method either handle the exception using `try-catch` or it must specify the exception using `throws` keyword.
- Example includes `IOException`, `SQLException`, `FileNotFoundException` etc.

34.1.2 Unchecked exceptions

- Unchecked exceptions are extended from the `java.lang.RuntimeException` class.
- These exceptions occur at the time of execution thus called as Runtime Exceptions.
- Examples includes `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException`.

34.1.3 Common Java built in Exceptions

Exception Type	Cause of Exception
ArithmeticException	Occurs by arithmetical errors such as division by zero(0).
ArrayIndexOutOfBoundsException	Occurs when tries to access an invalid index
FileNotFoundException	Occurs when nonexistent file is trying to be accessed
IOException	Occurs when an input output operation failed
FileNotFoundException	Occurs when file is not accessible
NullPointerException	Occurs when referencing a null object.
RuntimeException	Exception Occurs during runtime
StringIndexOutOfBoundsException	
NumberFormatException	Occurs when a string and numeric conversion fails

34.2 Java **try catch** keywords for exception handling

Syntax:

```
Try
{
    code that may throw an exception
}
catch(Exception type e)
```

Example 151 : Program without exceptional handling

```
public class ExceptionTryCatch
{
    public static void main(String args[])
    {
        int data=25/0; // throw exception
```

```
    System.out.println("Division by Zero");  
    }  
}
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at ExceptionTryCatch.main(ExceptionTryCatch.java:5)
```

In the above program statement after exception is not executed.

Example 152: Exceptional handling using try-catch keyword

```
public class ExceptionTryCatch  
{  
    public static void main(String[] args)  
    {  
        try  
        {  
            int data=25/0; // throw exception  
        }  
        catch(ArithmeticException e)  
        {  
            System.out.println("Division by zero");  
        }  
        System.out.println("Exception handling using try catch");  
    }  
}
```

Output:

```
Division by zero  
Exception handling using try catch
```

Example 153 : To illustrate NullPointerException

```
class ExceptionNullPointer
{
    public static void main(String args[])
    {
        try
        {
            String a = null; //null value
            System.out.println(a.charAt(0));
        }
        catch(NullPointerException e)
        {
            System.out.println(" Referencing a Null object");
        }
    }
}
```

Output:

Referencing a Null object

Example 154: To illustrate ArrayIndexOutOfBoundsException

```
class ExceptionArrayIndexOutOfBounds
{
    public static void main(String args[])
    {
        try{
            int arr[] = new int[5];
            arr[7] = 25; // accessing index 7 which does not exist
        }
        catch(ArrayIndexOutOfBoundsException e) {
```



```
        System.out.println (" Invalid Array Index number");
    }
}
}
```

Output:

Invalid Array Index number

Example 155: To illustrate multiple catch block

```
public class ExceptionTryCatch2
{
    public static void main(String[] args)
    {
        try
        {
            int arr[]= new int[5];
            System.out.println(arr[7]); // throw exception
        }
        catch(ArithmeticException e)
        {
            System.out.println("Division by zero");
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println("Invalid Array Index number");
        }
        catch(Exception e)
        {
            System.out.println("Exception name not present in any
catch block");
        }
        System.out.println("Multiple catch block");
    }
}
```

```
}  
}
```

Output:

```
Invalid Array Index number  
Multiple catch block
```

In the above program, if exception is generated and corresponding exception type is not provided, then the catch block containing “Exception name not present in any catch block “ will be executed.

Example 156: To illustrate nested try block

```
class ExceptionNestedTry  
{  
    public static void main(String args[])  
    {  
        try  
        {  
            try  
            {  
                System.out.println("First try block");  
                int a =25/0;  
            }  
            catch(ArithmeticException e1)  
            {  
                System.out.println("Division by Zero");  
            }  
        }  
        try  
        {  
            System.out.println("Second try block");  
            int arr[] =new int[5];  
            arr[7]=25;  
        }  
    }  
}
```

```
    }  
    catch(ArrayIndexOutOfBoundsException e2)  
    {  
        System.out.println("Invalid index number");  
    }  
    }  
    catch(Exception e3)  
    {  
        System.out.println("Parent try catch statement");  
    }  
    }  
}
```

Output:

```
First try block  
Division by Zero  
Second try block  
Invalid index number
```

34.3 finally block

- The **finally** block is used with try block
- Statements within this block must be executed whether exception occurs or not.
- The **finally** block is optional
- The **finally** block can be used in java for execution codes such as closing a file, closing a connection, stream etc.

Example 157: To illustrate finally block when exception doesn't occur.

```
class ExceptionFinally
{
    public static void main(String args[])
    {
        try
        {
            int num=25/5;
        }
        catch(ArithmeticException e){
            System.out.println("Division by Zero");
        }
        Finally
        {
            System.out.println("This is finally block");
        }
    }
}
```

Output:

This is finally block

Example 158: To illustrate finally block when exception occur and handled

```
class ExceptionFinally1
{
    public static void main(String args[])
    {
        try
        {
            int num=25/0;
        }
    }
}
```

```
    catch(ArithmeticException e){
        System.out.println("Division by Zero");
    }
    finally
    {
        System.out.println("This is finally block");
    }
}
```

Output:

Division by Zero
This is finally block

Example 159: To illustrate finally block when exception occur and not handled

```
class ExceptionFinally2
{
    public static void main(String args[])
    {
        try
        {
            int num=25/0;
        }
        catch(NullPointerException e){
            System.out.println("Referencing a null object");
        }
        finally
        {
            System.out.println("This is finally block");
        }
    }
}
```

```
}
```

Output:

This is finally block

Exception in thread "main" java.lang.ArithmeticException: / by zero
at ExceptionFinally2.main(ExceptionFinally2.java:7)

34.4 User defined Exception/Custom Exception

- Exception created and customize by the user according to their need.
- The **throw** keyword is used for throwing custom exception from a method or any block of code.

Example 160: To illustrate throw keyword

```
public class ExceptionThrow
{
    static void ageCheck(int age)
    {
        if(age<18)
            throw new ArithmeticException("Sorry age < 18, You are not eligible to vote");
        else
            System.out.println("You are eligible to vote");
    }
    public static void main(String args[])
    {
        ageCheck(17);
    }
}
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: Sorry  
age < 18, You are not eligible to vote  
    at ExceptionThrow.ageCheck(ExceptionThrow.java:6)  
    at ExceptionThrow.main(ExceptionThrow.java:12)
```

35. throws

- The **throws** keyword in java is used to declare multiple exceptions.
- The **throws** keyword inform caller of the method about the types of exceptions the method might throw.

Example 161: To illustrate throws keyword in Java.

```
class ThrowsExample  
{  
    void myNum()throws ArithmeticException  
    {  
        int num=25/0;  
    }  
}  
public class ExceptionThrows{  
    Public Static Void Main(String Args[]){  
        try  
        {  
            ThrowsExample obj=new ThrowsExample();  
            obj.myNum();  
        } catch(ArithmeticException e)  
        {  
            System.out.println("Exception handled : " +e);  
        }  
    }  
}
```

```
}  
}
```

Output:

Exception handled : java.lang.ArithmeticException: / by zero

36. Thread

Java support Multithreading programming that allow execution of two or more parts (sub program) of a single program to run concurrently. Each part of such program are known as Threads. Threads are used to do parallel processing. Each thread has its own program counter, stack and local variables. Threads are sometimes called as lightweight processes.

36.1 Life Cycle of a thread

During its life cycle, a thread can be in any of the following states.

1. New

When a thread is created, it is in new state also known as new born state. It continues to be in this state until the program starts the thread using start() method.

2. Runnable

After the new thread is started using start () method, the thread is in runnable state. In this state, the thread is ready for execution.

3. Running

The thread is in running state. In this state, a thread can change state to suspend(), sleep() or wait() depends on time slicing or need of some resources.

4. Blocked/Waiting

When a thread is on hold state for any reason it is in this state.

5. Terminated

A thread is said to be terminated or dead in this state.

36.2 Creating a Thread in java

6. By extending Thread class.
7. By implementing Runnable interface

Example 162: To illustrate thread creating by extending Thread class.

```
class TestThread extends Thread
{
    public void run()
    {
        System.out.println("A Thread is running");
    }
    public static void main(String args[])
    {
        TestThread obj=new TestThread();
        obj.start();
    }
}
```

Output:

A Thread is running

- In the above program TestThread class is extending the Thread class.
 - run() method is called for executing the thread codes.
 - Object of the class TestThread is created using new keyword.
 - Start() method is called to move the thread into runnable state.
-

Example 163 : Another example of creating threads by extending Thread class.

```
class First extends Thread
{
    public void run()
    {
        for (int i=1; i<=5;i++)
        {
            System.out.println("First Thread is running");
        }
    }
}
class Second extends Thread
{
    public void run()
    {
        for (int j=1; j<=5;j++)
        {
            System.out.println("Second Thread is running");
        }
    }
}
class Third extends Thread
{
    public void run()
    {
        for (int k=1; k<=5;k++)
        {
            System.out.println("Third Thread is running");
        }
    }
}
```

```
class TestThread2
{
public static void main(String args[])
{
    First f1=new First();
    f1.start();
    Second s1=new Second();
    s1.start();
    Third t1=new Third();
    t1.start();
}
}
```

Output: (after first run):

First Thread is running
First Thread is running
First Thread is running
Second Thread is running
Second Thread is running
First Thread is running
First Thread is running
Second Thread is running
Second Thread is running
Third Thread is running
Third Thread is running
Second Thread is running
Third Thread is running
Third Thread is running
Third Thread is running

Output (after second run):

First Thread is running
First Thread is running
First Thread is running

Second Thread is running
Second Thread is running
Second Thread is running
Second Thread is running
Second Thread is running
First Thread is running
First Thread is running
Third Thread is running
Third Thread is running
Third Thread is running
Third Thread is running
Third Thread is running

Output (after third run):

First Thread is running
First Thread is running
First Thread is running
First Thread is running
First Thread is running
Second Thread is running
Second Thread is running
Second Thread is running
Second Thread is running
Second Thread is running
Third Thread is running
Third Thread is running
Third Thread is running
Third Thread is running
Third Thread is running

- In the above program, three threads are created
- They all are running concurrently
- After every run, output is different and not following any sequence.

- Each thread is executing independently whenever it has a chance.

Example 164 : To illustrate Thread creation by implementing Runnable interface

```
class TestThread3 implements Runnable
{
    public void run()
    {
        System.out.println("A Thread is running");
    }
    public static void main(String args[])
    {
        TestThread3 obj=new TestThread3();
        Thread obj2=new Thread(obj);
        obj2.start();
    }
}
```

Output:

A Thread is running

In the above program where the class implementing Runnable instance, the thread can be executed by passing an instance of the class to a Thread object's constructor and then calling the thread's start() method.

36.3 Setting Priorities for Threads

- Each thread has some priority according to which they are scheduled.
- By default, a thread inherits its priority from parent thread (the thread that created it).
- It can be changed using the method setPriority() of class Thread.

- A thread priority can be set to any range between MIN_PRIORITY(constant value of 1) and MAX_PRIORITY(constant value of 10) in thread class.
- By default, each thread gets a NORM_PRIORITY that is equal to 5.
- Setting Thread priorities cannot guarantee that higher priority thread executes first as selection of thread for execution depends upon thread scheduler which is platform dependent.

Example 165 : To illustrate Thread priority

```
class ThreadPriority extends Thread
{
    public void run()
    {
        System.out.println("A Thread is Running");
    }
    public static void main(String[]args)
    {
        ThreadPriority obj = new ThreadPriority();
        obj.start();
        System.out.println("Maximum thread priority : " + obj.MAX_
PRIORITY);
        System.out.println("Minimum thread priority : " + obj.MIN_
PRIORITY);
        System.out.println("Normal thread priority : " + obj.NORM_
PRIORITY);
    }
}
```

Output:

```
A Thread is Running
Maximum thread priority : 10
Minimum thread priority : 1
```

Normal thread priority : 5

Commonly used methods of Thread class

isAlive()

- To tests whether the thread is still running or alive.
- This methods returns true if the thread is still running else returns false.

Example 166 : To illustrate isAlive() method of class Thread

```
public class IsAlive extends Thread
{
    public void run()
    {
        try
        {
            Thread.sleep(300);
            System.out.println("Status of run() method "+Thread.
currentThread().isAlive());
        }
        catch (InterruptedException ie) {
        }
    }
    public static void main(String[] args)
    {
        IsAlive obj = new IsAlive();
        System.out.println("Thread Status : "+obj.isAlive());
        obj.start();
        System.out.println("Thread Status : "+obj.isAlive());
    }
}
```

Output:

Thread Status : false
Thread Status : true
Status of run() method true

join()

- This method allows one thread to wait until another thread completes its execution (dead).

Example 167 : Program without join() method of class Thread.

```
public class WithoutJoin
{
    public static void main(String[] args)
    {
        Thread thread1 = new Thread(new MyClass2(), "thread1");
        Thread thread2 = new Thread(new MyClass2(), "thread2");
        Thread thread3 = new Thread(new MyClass2(), "thread3");
        Thread thread4 = new Thread(new MyClass2(), "thread4");

        thread1.start();
        thread2.start();
        thread3.start();
        thread4.start();
    }
}
class MyClass2 implements Runnable
{
    public void run()
    {
        Thread t = Thread.currentThread();
        System.out.println("Thread started: "+t.getName());
        try
```



```

        {
            Thread.sleep(500);
        } catch (InterruptedException ie) {
            }
        System.out.println("Thread " +t.getName()+ " ended");
    }
}

```

Output:

```

Thread started: thread1
Thread started: thread4
Thread started: thread3
Thread started: thread2
Thread thread1 ended
Thread thread3 ended
Thread thread4 ended
Thread thread2 ended

```

Example 168 : Program with join() method of class Thread.

```

public class WithJoin
{
    public static void main(String[] args)
    {
        Thread thread1 = new Thread(new MyClass(), "thread1");
        Thread thread2 = new Thread(new MyClass(), "thread2");
        Thread thread3 = new Thread(new MyClass(), "thread3");

        thread1.start();
        try {
            thread1.join();
        } catch (InterruptedException ie) {

```

```

    }

    thread2.start();
    try {
        thread2.join();
    } catch (InterruptedException ie) {
    }

    thread3.start();
    try {
        thread3.join();
    } catch (InterruptedException ie) {
    }
}
class MyClass implements Runnable
{
    public void run()
    {
        Thread t = Thread.currentThread();
        System.out.println("Thread started: "+t.getName());
        try {
            Thread.sleep(500);
        } catch (InterruptedException ie) {
        }
        System.out.println("Thread ended: " +t.getName());
    }
}

```

Output:

```

Thread started: thread1
Thread ended: thread1
Thread started: thread2

```

Thread ended: thread2
Thread started: thread3
Thread ended: thread3

In the above program sleep() method is used to pause/suspend execution of current thread for specified number of milliseconds.

yield()

- This method temporarily pause the currently executing method and allow other waiting threads of the same priority to execute.

Example 169 : To illustrate yield() method.

```
class Thread2 extends Thread
{
    public void run()
    {
        for (int i=0; i<5 ; i++)
        {
            System.out.println("Thread2 is running");
            Thread.yield();
        }
    }
}
public class YieldThread
{
    public static void main(String[]args)
    {
        Thread2 t = new Thread2();
        t.start();
        for (int i=0; i<5 ; i++)
        {
            System.out.println( "Main Thread is running");
```

```
    }  
  }  
}
```

Output:

```
Main Thread is running  
Main Thread is running  
Thread2 is running  
Main Thread is running  
Thread2 is running  
Main Thread is running  
Main Thread is running  
Thread2 is running  
Thread2 is running  
Thread2 is running
```

In the above program Main thread and Thread2 has the same priority so every time the program will be executed it is the responsibility of Thread scheduler to execute one of them. Thus outputs may vary in every execution.

stop()

- This method is used to terminate(kill) the execution of thread
- Once killed cannot be restarted by start() method.

Example 170 : To illustrate stop() method in java thread

```
class First extends Thread  
{  
    public void run()  
    {  
        for (int i=1; i<=3;i++)  
        {
```

```
        System.out.println("First Thread is running");
    }
}

class Second extends Thread
{
    public void run()
    {
        for (int j=1; j<=3;j++)
        {
            if (j==2)
                stop();
            System.out.println("Second Thread is running");
        }
    }
}

class Third extends Thread
{
    public void run()
    {
        for (int k=1; k<=3;k++)
        {
            System.out.println("Third Thread is running");
        }
    }
}

class StopThread
{
    public static void main(String args[])
    {
        First f1=new First();
    }
}
```

```
f1.start();
Second s1=new Second();
s1.start();
Third t1=new Third();
t1.start();
}
}
```

Output:

Program When Compiled

Note: StopThread.java uses or overrides a deprecated API.

Note: Recompile with -Xlint:deprecation for details.

When Executed

First Thread is running
First Thread is running
First Thread is running
Second Thread is running
Third Thread is running
Third Thread is running
Third Thread is running

Some popularly used methods for managing threads.

Method	Description
setName()	To name a thread
getName()	Returns thread name
getState()	Returns the state of the thread
interrupt()	Interrupts the thread
isInterrupted()	Tests whether the current thread has been interrupted.
checkAccess	Determines if the currently thread has permission to modify this thread
getThreadGroup	Returns the thread group to which this thread belongs

37. Java Input/Output

- Java handles all input and output operations in the form of streams.
- Java.io package contains all the classes that supports input output operations.

37.1 Stream Classes

- A stream is a sequence of bytes that abstract the details of input/output operations.
- Stream classes belongs to java.io package
- It allows handling all types of data.

There are two basic types of stream classes defined by Java.

1. Byte stream classes
2. Character stream classes

37.2 Byte Stream Classes

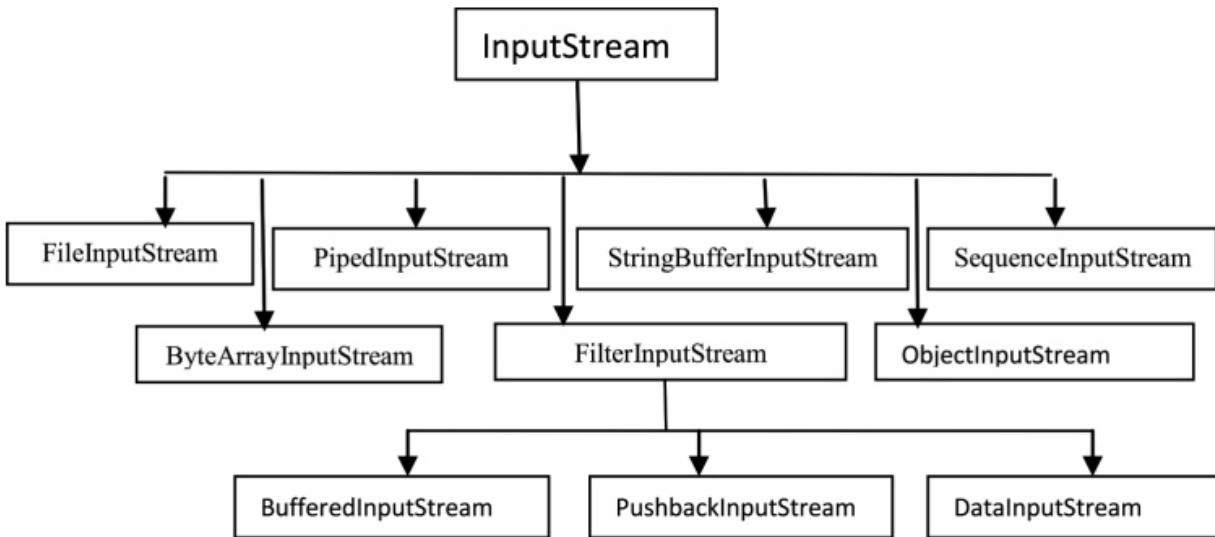
- Byte Stream Classes have been designed for processing I/O operations on bytes.

The two types of Byte Stream Classes are

1. InputStream
2. OutputStream

37.2.1 InputStream

- The InputStream class lays the foundation for the input class hierarchy.



InputStream Class Hierarchy

InputStream methods

read()

- This method reads a byte of data from the input stream.

A text file is created with the name “testfile”

testfile.txt has the content “ Learning Java Programming”

Example 171 : To illustrate read() method of InputStream

```
import java.io.FileInputStream;
public class StreamRead
{
    public static void main(String args[])
    {
        try {
            FileInputStream f=new FileInputStream("D:/testfile.txt");
            int i=f.read();
            System.out.print((char)i);
        }
    }
}
```



```
f.close();
} catch (Exception e) { System.out.println(e); }
}
```

Output:

L

In the above program **close()** method is called to close the input stream.

A text file is created with the name “testfile”

testfile.txt has the content “ Learning Java Programming”

Example 172 : To read all characters of any text file (testfile.txt)

```
import java.io.FileInputStream;
public class StreamRead1
{
    public static void main(String args[])
    {
        try{
            FileInputStream f1=new FileInputStream("D:/testfile.txt");
            int i=0;
            while((i=f1.read())!=-1)
            {
                System.out.print((char)i);
            }
            f1.close();
        } catch (Exception e) { System.out.println(e); }
    }
}
```

Output:

Learning Java Programming

available()

- This method returns the number of available bytes.

A text file is created with the name “testfile”

testfile.txt has the content “ Learning Java Programming”

Example 173: To illustrate available() method of InputStream.

```
import java.io.FileInputStream;
public class StreamAvailable
{
    public static void main(String args[])
    {
        try{
            FileInputStream f1=new FileInputStream("D:/testfile.txt");
            int a1=f1.available();
            System.out.print("Available bytes at the beginning is "
+ a1);
            int i=0;
            while((i=f1.read())!=-1)
            {
                int a2=f1.available();
                char c=(char)i;
                System.out.print("\nAvailable bytes is " +a2);
                System.out.print(" at input "+c);
            }

            f1.close();
        }catch(Exception e)
```

```
        {  
            System.out.println(e);  
        }  
    }  
}
```

Output:

Available bytes at the beginning is 25
Available bytes is 24 at input L
Available bytes is 23 at input e
Available bytes is 22 at input a
Available bytes is 21 at input r
Available bytes is 20 at input n
Available bytes is 19 at input i
Available bytes is 18 at input n
Available bytes is 17 at input g
Available bytes is 16 at input
Available bytes is 15 at input J
Available bytes is 14 at input a
Available bytes is 13 at input v
Available bytes is 12 at input a
Available bytes is 11 at input
Available bytes is 10 at input P
Available bytes is 9 at input r
Available bytes is 8 at input o
Available bytes is 7 at input g
Available bytes is 6 at input r
Available bytes is 5 at input a
Available bytes is 4 at input m
Available bytes is 3 at input m
Available bytes is 2 at input i
Available bytes is 1 at input n
Available bytes is 0 at input g

skip(n)

- This method skip over n bytes available in the input
- A text file is created with the name “testfile”
- testfile.txt has the content “ Learning Java Programming”

Example 174 : To illustrate skip() method of InputStream

```
public class StreamSkip
{
    public static void main(String args[])
    {
        try{
            FileInputStream f1=new FileInputStream("D:/testfile.txt");
            f1.skip(5);
            System.out.print("After skipping 5 bytes : ");
            int i=0;

            while((i=f1.read())!=-1)
            {
                System.out.print((char)i);
            }

            f1.close();
        }catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Output:

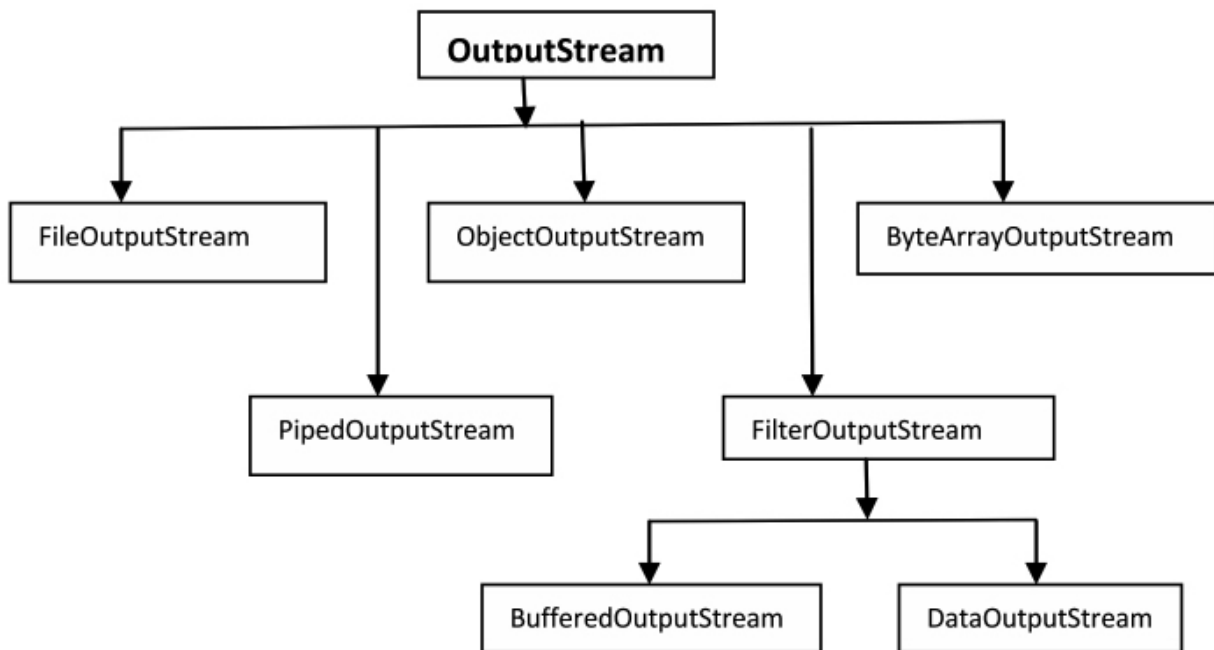
After skipping 5 bytes : ing Java Programming

Some more methods of InputStream

Methods	Description
read(byte b[])	Reads an array of bytes into b
read(byte b[], int i, int j)	Reads j th bytes into b starting from i th byte
reset()	Returns the control to the beginning of the stream
mark()	Mark the position in InputStream up to which data has been read.
getFD()	Returns the file descriptor associated with the input stream

37.2.2 OutputStream

- The OutputStream class lays the foundation for the output class hierarchy.



OutputStream Class Hierarchy

OutputStream methods

write()

- This method writes a byte to the OutputStream
-

Example 175 : To illustrate write() method of OutputStream.

```
import java.io.FileOutputStream;
public class StreamWrite
{
    public static void main(String args[])
    {
        try{
            FileOutputStream f=new
FileOutputStream("D:/testfile1.txt");
            f.write(67);
            f.close();
            System.out.println("Successfully Added");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Output:

Successfully Added

The testfile1.txt content can be checked with the data

C

In the above program, **close()** method is called to close the output stream.

Example 176 : To add string to a file.

```
import java.io.FileOutputStream;
```

```

public class StreamWriter2
{
    public static void main(String args[])
    {
        try{
            FileOutputStream f1=new
FileOutputStream("D:/testfile2.txt");
            String s="Learning Java OutputStream";
            byte b[]=s.getBytes();
            f1.write(b);
            f1.close();
            System.out.println("Successfully Added");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

```

Output:

Successfully Added

The testfile2.txt content can be checked with the data

Learning Java OutputStream

Some more methods of OutputStream

Methods	Description
write(byte arr[])	Writes all bytes in the array arr to the output stream
write(byte arr[], int i, int j)	Writes j th bytes from array arr starting from i th byte
flush()	Flushes the output stream.

37.3 Character Stream Classes

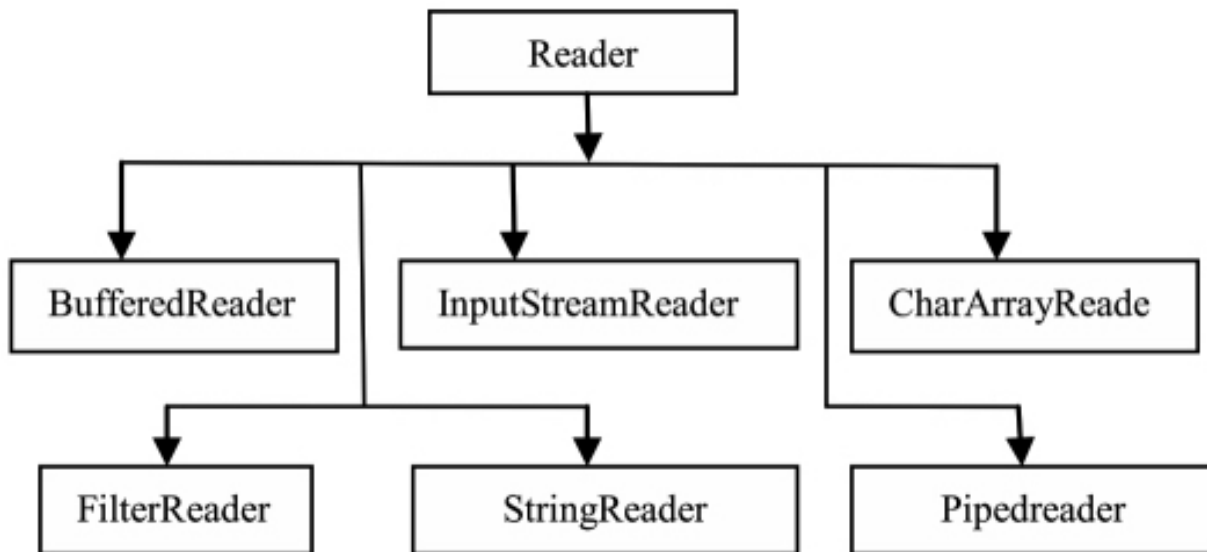
- Character Stream Classes have been designed for processing I/O operations on characters.
- JDK 1.1 introduces the Character Stream Classes for Unicode I/O.

Java provides two types of Character Stream Classes

1. Reader Stream Class
2. Writer Stream Class

37.3.1 Reader Stream Class

- The `java.io.Reader` class is an abstract class for reading character stream.
- The subclasses of Reader class are used to read data.



Reader Class Hierarchy

A text file is created with the name “testfile”

testfile.txt has the content “ Learning Java Programming”

Example 177 : To read data from testfile.txt using BufferedReader class.


```
import java.io.FileReader;
import java.io.BufferedReader;

class ReaderStream
{
    public static void main(String[] args)
    {
        char arr[] = new char[100];
        try {
            FileReader f1 = new FileReader("D:/testfile.txt");
            BufferedReader r1 = new BufferedReader(f1);
            r1.read(arr);
            System.out.println(arr);
            r1.close();
        } catch (Exception e) {

        }
    }
}
```

Data in the testfile.txt

Learning Java Programming

Output:

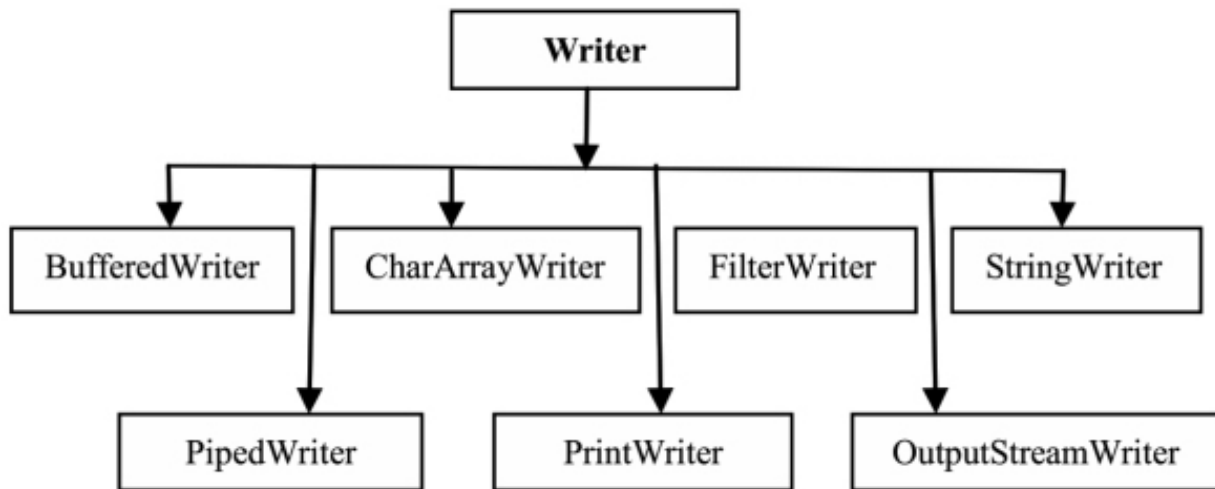
Learning Java Programming

Some methods of Reader class

Methods	Description
String readLine()	For reading a line of text
boolean ready()	Tells whether the stream is ready to read
void reset()	Reset the stream to the most recent mark
long skip(long n)	Skip characters

37.3.2 Writer Stream Class

- The java.io.Writer class is an abstract class for writing character stream.
- The subclasses of Writer class are used to write data.



Writer Class Hierarchy

Example 178 : To illustrate Writer using the FileWriter class

```
import java.io.*;
class WriterStream
{
    public static void main(String[] args)
    {
        try
        {
            File fl = new File("D:/testfile3.txt");
            String str="Learning Java Writer Stream Class";
            FileWriter f2 = new FileWriter(fl) ;
            f2.write(str);
            f2.close();
            System.out.println("Data Successfully Added");
        }
    }
}
```

```
catch (IOException e) {  
    }  
}  
}
```

Output:

Data Successfully Added

The testfile3.txt content can be checked with the data

Learning Java Writer Stream Class

Some methods of Writer class

Methods	Description
write(char arr[])	Writes the characters from the specified array to the output stream
write (int c)	It writes a single character
append (char c)	It appends the specified character to this writer.
flush()	It flushes the stream

37.4 Random Access Files

java.io.RandomAccessFile class enables that data can be read from or written to random location within a file.

Two ways of creating a Random Access File

1. RandomAccessFile(File file, String mode)
2. RandomAccessFile(String name, String mode)

Access mode

Access mode	Description
r	Read only mode
rw	Read and write mode
rws	Read write mode and any changes to the file's content and its metadata take effect immediately.
rwd	Read write mode and any changes to the file's content but not its metadata take effect immediately.

RandomAccessFile class implements both the DataInput and DataOut interfaces for reading and writing.

Some methods of RandomAccessFile class

Methods	Description
read()	Reads a byte of data from this file
read(byte[] b)	Reads up to b.length bytes of data from this file into an array of bytes.
readLine	reads the next line of text from this file
seek(long pos)	Sets the file-pointer offset, measured from the beginning of this file, at which the next read or write occurs.
skipBytes(int n)	This skip over n bytes of input
getFilePointer()	Returns the current position of the file pointer.
write(int b)	Writes the specified byte to this file
write(byte[] b)	Writes b.length bytes of data from the specified byte array to this file, starting at the current file pointer.

Example 179 : To illustrate RandomAccessFile class methods

```
import java.io.*;

public class RandomAccess{
    public static void main(String[] args) {

        try {
```

```

        RandomAccessFile file1 = new
RandomAccessFile("D:/testfile3. txt","rw");

        file1.seek(0);
        System.out.println("Original data : \n" + file1.readLine());
        System.out.println("\n");

        file1.seek(14);
        System.out.println("Reading file after 14th position : \n " +
file1. readLine());
        System.out.println("\n");
        file1.seek(9);
        file1.writeBytes("Random Access File class");
        System.out.println("\n");

        file1.seek(0);
        System.out.println("Reading Original file after adding data : \n"
+ file1.readLine());

    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

Output:

Original data :
Learning Java Writer Stream Class

Reading file after 14th position :
Writer Stream Class

Reading Original file after adding data :

37.5 Using the File Class

- The File class belongs to java.io package.
- The File class provides methods for several file operations such as creating a file, opening a file, closing a file, deleting a file, renaming a file.

createNewFile()

- The createNewFile() method of File class is used to create a file in Java.

Example 180 : To illustrate createNewFile() method of File class

```
import java.io.*;
public class FileCreation
{
    public static void main(String[] args)
    {
        try {
            File f1 = new File("D:/myfile1.txt");
            if (f1.createNewFile())
            {
                System.out.println("File successfully created: " +
f1.getName());
            } else
            {
                System.out.println("File already exists.");
            }
        } catch (IOException e)
```

```
        {
            e.printStackTrace();
        }
    }
}
```

Output:

First Run

File successfully created: myfile1.txt

Second Run

File already exists.

write()

- The write() method of FileWriter class is used to write data to the text file.

Example 181 : To illustrate write() method of FileWriter class.

```
import java.io.*;
public class WriteFile {
    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("D:/myfile2.txt");
            fw.write("Learning Java File handling operations");
            fw.close();
            System.out.println("Data Added to the file.");
        } catch (IOException e) {

        }
    }
}
```

Output:

Data Added to the file.

The content of myfile2.txt can be checked for the data.

Learning Java File handling operations

read()

- The read() method of FileReader class is used to read data from the text file.

Example 182 : To illustrate read() method of FileReader class.

```
import java.io.*;
class FileRead
{
    public static void main(String[] args) throws IOException
    {
        int ch;
        FileReader fr1=null;
        try
        {
            fr1 = new FileReader("D:/myfile4.txt");
        }
        catch(FileNotFoundException e)
        {
            System.out.println("File not found");
        }

        while ((ch=fr1.read())!=-1)
            System.out.print((char)ch);
        fr1.close();
    }
}
```



```
}
```

Output:

Learning Java File handling operations

delete()

- The delete() method of File Class is used to delete a file.

Example 183 : To illustrate delete() method of Java File class

```
import java.io.File;
public class FileDelete
{
    public static void main(String[] args)
    {
        File f1 = new File("D:/myfile2.txt");
        f1.delete();
        System.out.println("The File " + f1.getName() + " is deleted");
    }
}
```

Output:

The File myfile2.txt is deleted

renameTo()

- The renameTo() method of File class is used to rename a file.

Example 184 : To illustrate rename() method of Java file class

```
import java.io.*;
public class FileRename
```

```
{
public static void main(String[] args)
{
    File f1 = new File("D:/myfile1.txt");

    try {
        f1.createNewFile();
    }
    catch(Exception e) {
        e.printStackTrace();
    }

    File newf1 = new File("D:/newfile.txt");
    boolean value = f1.renameTo(newf1);
    if(value)
    {
        System.out.println("File name changed");
    }
    else
    {
        System.out.println("File name cannot be changed.");
    }
}
}
```

Output:

File name changed

Some useful methods for getting information about files.

Type	Methods	Description
Boolean	canRead()	To check whether the file is readable or not.
Boolean	canWrite()	To check whether the file is writable or not.
Boolean	exists()	To check whether the file exists
String	getName()	To get the name of the file
String	getAbsolutePath()	To get the absolute pathname of the file.
String[]	list()	Returns an list of the files in the directory
Long	length()	Returns the size of the file.

Example 185 : To illustrate few methods for getting information about a file.

```
import java.io.File;
public class FileInformation
{
    public static void main(String[] args)
    {
        File f1 = new File("D:/testfile.txt");
        if (f1.exists())
        {
            System.out.println("File name: " + f1.getName());
            System.out.println("Absolute path of the file: " +
f1.getAbsolutePath());
            System.out.println("File Size : " + f1.length());
            System.out.println("Writeable: " + f1.canWrite());
            System.out.println("Readable " + f1.canRead());
        }
        else {
            System.out.println("The file does not exist.");
        }
    }
}
```

Output:

File name: testfile.txt
Absolute path of the file: D:\testfile.txt
File Size : 25
Writeable: true
Readable true

Example 186 : To illustrate list() method

```
import java.io.File;
class FileList
{
    public static void main(String[] args)
    {
        File f1 = new File("C:/book/");
        String[] str = f1.list();
        System.out.println("List of files : \n");
        for(String str2 : str)
        {
            System.out.println(str2);
        }
    }
}
```

Output:

38. Database Management

A database is a collection of related information organized in table format. Data Base Management System (DBMS) is a software that supports mechanism for creating and managing data in the databases. Some of the most popular database management systems includes MS-Access, MS-SQL Server, Oracle, Sybase.

38.1 Java Data Base Connectivity (JDBC)

- JDBC helps to write java application that manages not only to connect to a wide range of database but to create, insert, query, update and process the database.
- JDBC provides a database programming interface for Java programs.
- The JDBC driver manager is used to connect a Java application to the appropriate driver
- The JDBC-ODBC bridge allows to use the ODBC driver as JDBC drivers.

38.2 Open Data Base Connectivity (ODBC)

- It is a standard database access method by Microsoft Corporation, to make possible to access any data from any application regardless of which data base management software is being use to handle the data. ODBC manages this with the help of a middle layer, called a driver.

38.3 Steps for Java Data Base Connectivity

1. Register the driver

The forName() method of the Class class is used to load driver class.

2. Create the Connection

The `getConnection()` method of `DriverManager` class is used to create connection with the database.

3. Create the statement

The statement object is created using `createStatement()` method. The `Statement` object allows to execute simple queries.

4. Execute the query

A query is executed using `executeQuery()` method (to execute simple query and result a single `ResultSet` object), `executeUpdate()` method (to execute an SQL `INSERT`, `UPDATE` or `DELETE` statement), `execute()` method (to execute an SQL statement that may result multiple results) of the `Statement` object.

5. Close the connection object

The `close()` method of connection interface is used to close the connection.

Before establishing connection, you have to create a table in oracle database.

(Here Oracle 11g database version is used. The 'g' stands for "grid computing".)

Steps

1. Open oracle window
2. Create a table "Student" having the fields namely `stud_id`, `name` and `course`.
3. SQL query to create a table `create table student(stud_id integer, name varchar2(15), course varchar2(15));`
4. Close the oracle window

5. Open a notepad and write the java program.

Example 187 : To insert a record in to the table Student using java environment.

```
import java.sql.*;
public class InsertRecord
{
    public static void main (String args[])
    {
        try
        {
            Connection con=null;
            PreparedStatement st=null;
            ResultSet r=null;

            Class.forName("oracle.jdbc.driver.OracleDriver");
            con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:ORCL","scott","tiger");
            String sql="insert into Student values(101,'shekhar', 'BCA')";
            st=con.prepareStatement(sql);
            r=st.executeQuery();

            System.out.println("Record Succesfully Added");
            con.close();
        } catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Compile and run same as simple java programs.

Output:

Record Successfully Added

Open Oracle and write SQL query to fetch records of the table Student.

Select * from Students

Output:

STUD_ID	NAME	COURSE
101	shekhar	BCA

Example188 : To view record in to the table Student using java environment.

```
import java.sql.*;
public class ViewRecord
{
    public static void main (String args[])
    {
        try
        {
            Connection con=null;
            PreparedStatement st=null;
            ResultSet r=null;

            Class.forName("oracle.jdbc.driver.OracleDriver");
            con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:orcl","scott","tiger");
            String sql="select * from Student";
            st=con.prepareStatement(sql);
            r=st.executeQuery();
            while(r.next())
```



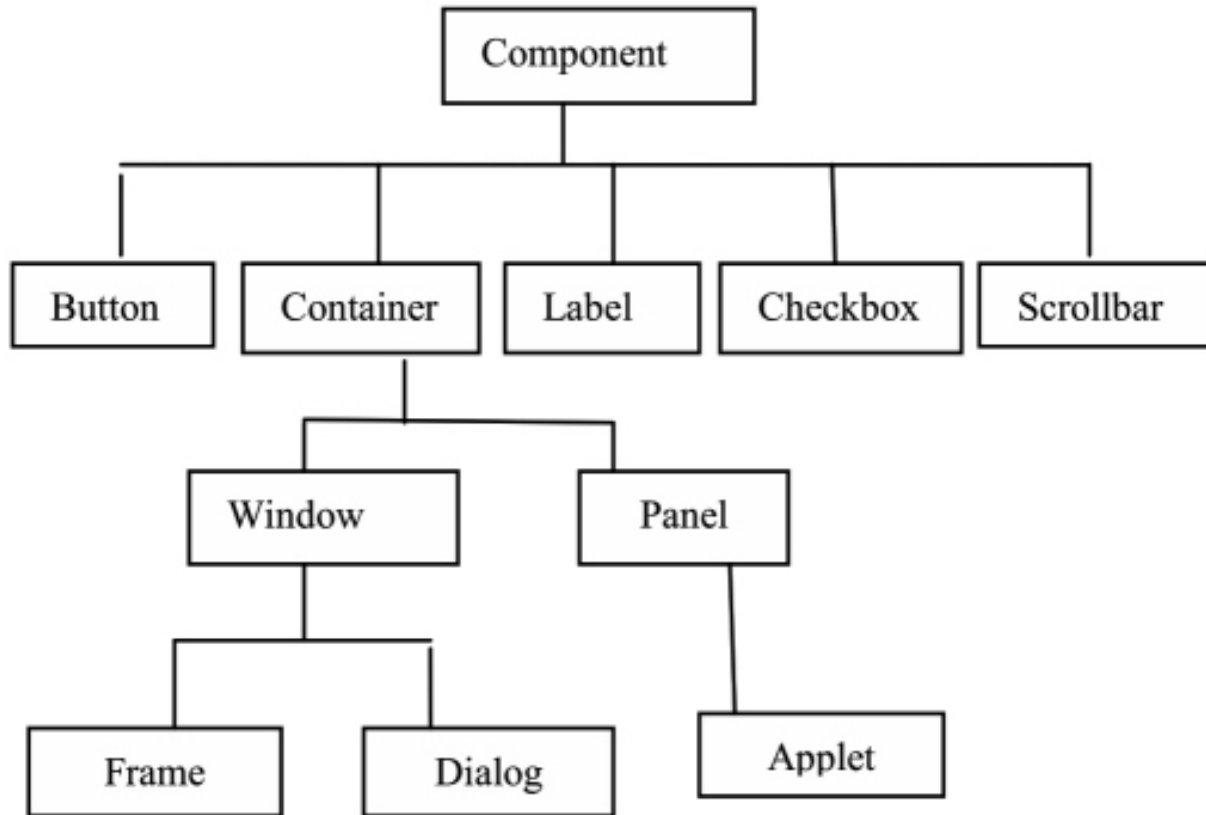
```
    {  
        System.out.print("Student ID : " + r.getInt("stud_id"));  
        System.out.print(" Student Name : " + r.getString("name"));  
        System.out.print(" Course : " + r.getString("course"));  
    }  
    r.close();  
    con.close();  
} catch(Exception e)  
{  
    System.out.println(e);  
}  
}  
}
```

Output:

Student ID : 101 Student Name : shekhar Course : BCA

39. Java AWT

Java AWT(Abstract Window Toolkit) is a platform dependent API for creating Graphical User Interface. Java AWT consists of classes and methods that are required for GUI.



AWT Hierarchy

39.1 Java AWT components

Components	Package with class name
Labels	java.awt.Label
Buttons	java.awt.Button
CheckBoxes	java.awt.Checkbox
Single-line text fiels	java.awt.TextField
Larger text display/editing areas	java.awt.TextArea
Pop-up list	java.awt.Choice
Lists	java.awt.List
Sliders and scrollbars	java.awt.Scrollbar
Drawing areas	java.awt.Canvas
Menus	java.awt.Menu, java.awt.MenuItem
Containers	Java.awt.Panel

Containers are component that can contain other components such as text field, buttons etc.

39.2 Types of container

Window :This container have no borders, title or menu bar.

Panel : this containers doesn't contain border, title or menu bar but can hold components like button, text fields, scroll bars etc.

Frame and Dialog are subclasses of Window class.

Frame : A Frame is the most widely used container that contains border, title and menu bar. It can hold several components such as buttons, text fields, scroll bars etc.

Dialog : This subclass container has border and title.

Two ways to create a **Frame** in AWT

1. By extending Frame Class
2. By creating the instance of Frame class.

40. Event Handling

Change in the state of an object is known as Event. Button press, entering character through keyboard, mouse click, scrolling pages and many such actions lead to event generation. Event handling is mechanism used to handle such type of events. When an event occurs, an event object of the suitable type is created. This object is passed to the listener (event handler) that implements the interface that has method for handling the event.

The Process

- An event is generated.
- The object of corresponding event class is created.
- Event object is passed to the listener with methods
- The method is executed.

Event	Object	Listener
When component is activated	ActionEvent	ActionListener
When adjustable elements are used such as scrollbars	AdjustmentEvent	AdjustmentListener
When components are added or removed from a container	ContainerEvent	ContainerListener
When a component receives focus for input	FocusEvent	FocusListener
When an item from a list is selected	ItemEvent	ItemListener
When a keyboard is pressed	KeyEvent	KeyListener
When mouse is scroll or click	MouseEvent	MouseListener MouseMotionListener
When component is painted	PaintEvent	PaintListener
When a text component is modified	TextEvent	TextListener

Example 189 : Java AWT button with ActionListener

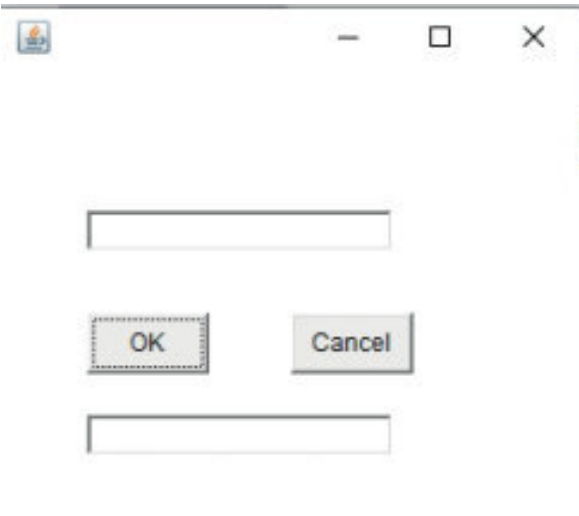
```
import java.awt.*;
import java.awt.event.*;
public class ButtonEvent extends Frame
{
    public static void main(String[] args)
    {
        Frame f=new Frame();
        TextField t1=new TextField();
        t1.setBounds(50,100, 150,20);
        TextField t2=new TextField();
        t2.setBounds(50,200, 150,20);
```

```
Button ok=new Button("OK");
ok.setBounds(50,150,60,30);
Button cancel=new Button("Cancel");
cancel.setBounds(150,150,60,30);

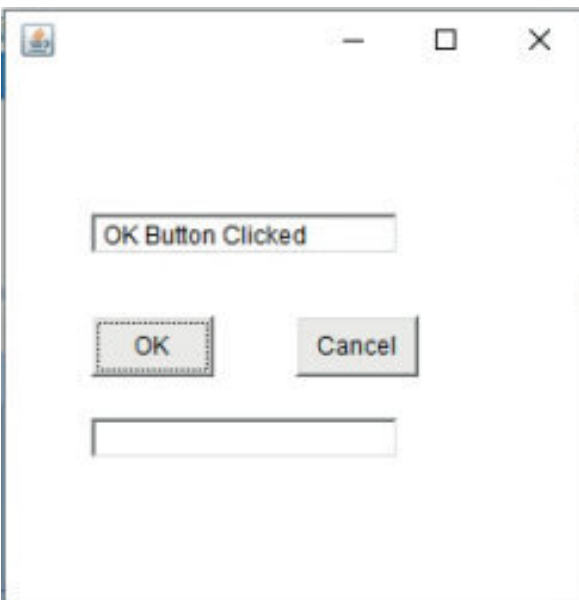
ok.addActionListener(new ActionListener(){
    public void actionPerformed(java.awt.event.ActionEvent e){
        t1.setText(" OK Button Clicked");
    }
});

cancel.addActionListener(new ActionListener(){
    public void actionPerformed(java.awt.event.ActionEvent e){
        t2.setText(" Cancel Button Clicked");
    }
});
f.add(ok);
f.add(cancel);
f.add(t1);
f.add(t2);
f.setSize(300,300);
f.setLayout(null);
f.setVisible(true);
}
}
```

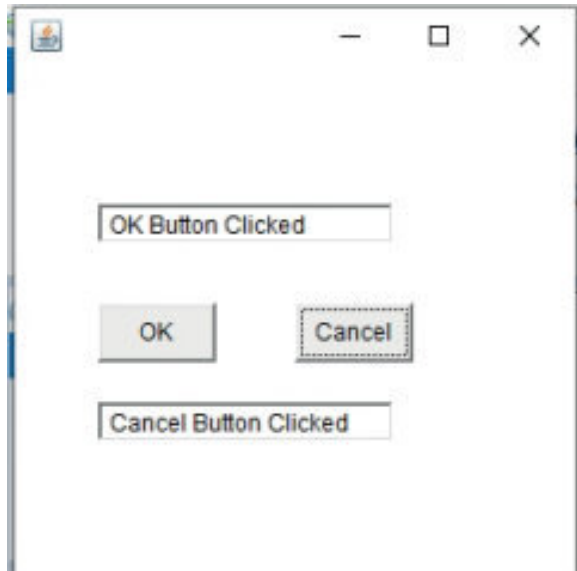
Output:



When OK button is clicked



When Cancel button is clicked



41. Applet Programming

48.1 Introduction

An Applet is a java program that runs in a web browser. Java language allows making web pages interactive and user friendly. Applets run on any browser that is Java-enabled. In order to run an applet it must be included in a web page, using HTML tags. Internet Explorer is a browser that support Java Applet.

An Applet is a Java class that extends the `java.applet.Applet` class.

48.2 Java Application and Java Applet

Java programs are divided into two types

1. Java Application
2. Java Applets

To understand how an Java Applet is different from an Java application programs

Java Application	Java Applet
These are standalone java programs that can be executed independently without having to use a web browser.	These are small java programs that runs in a java embedded browser. Applets are designed to be embedded within an HTML page.
The main() method is required for execution of Java Application	No main() method is required for execution of Java Applets
Java Application programs are executed using the “java” command.	Java Applet programs are executed using the “appletviewer” command.
It can access all types of resources available on the system	It can only access the browser specific services.
Java application programs may or may not extends class as per requirement	Java Applets must extends java.applet.Applet class.

48.3 Life Cycle of an Applet

An Applet life cycle can be described through four important methods.

1. The init() method : When an Applet is loaded into the memory of a computer, the init() method is called.

Syntax:

```
import java.awt.*;
import java.applet.*;
public class Applet1 extends Applet
{
    public void init()
    {
        // initialization
    }
}
```

2. The start() method : The start() method is called immediately after the init() method. This method is used to restart a process.

Syntax:

```
public void start()
{
    // resume or start statements
}
```

3. The stop() method : The stop() method is used to stop the threads that are running. The stop() method is always called before destroy() method.

Syntax:

```
public void stop()
{
    // Suspend execution statements
}
```

4. The destroy() method : The destroy() method is used to destroy the Applet. This method is called when the memory used for the Applet has to be freed or when the user exits the browser.

Syntax:

```
public void destroy()
{
    // shutdown statements
}
```

paint() : This method is another type of method defined by the AWT(Abstract Window Toolkit) component class which is used to create graphical user interface(GUI). java. awt package is the main package of the AWT. The paint() method is automatically called every time the applet's need to be redrawn. The paint() method takes an object of the Graphics class as a parameter.

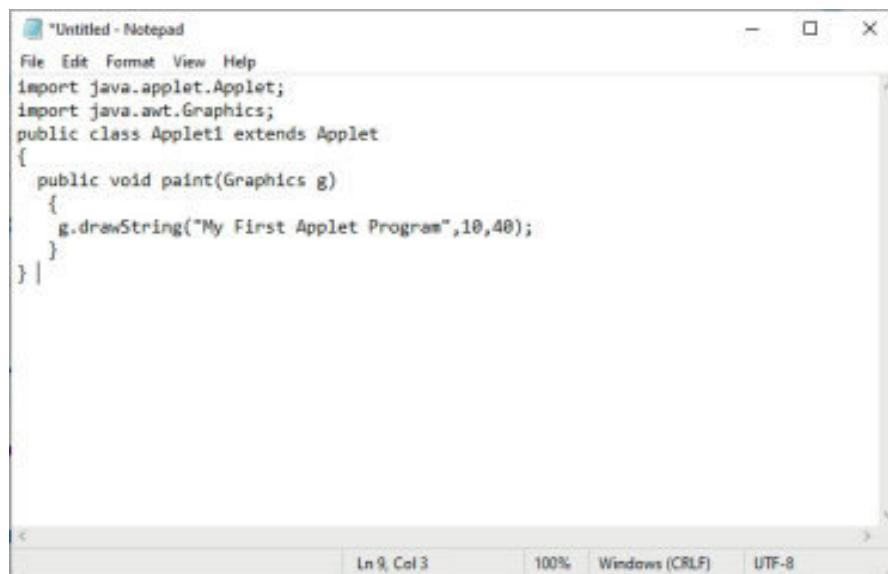
import java.awt.* imports all the components, containers and layout managers necessary for designing the user interface.

48.4 Applet Program Execution

To write, compile and run an Applet program.

Steps

1. Write the code of an Applet program.

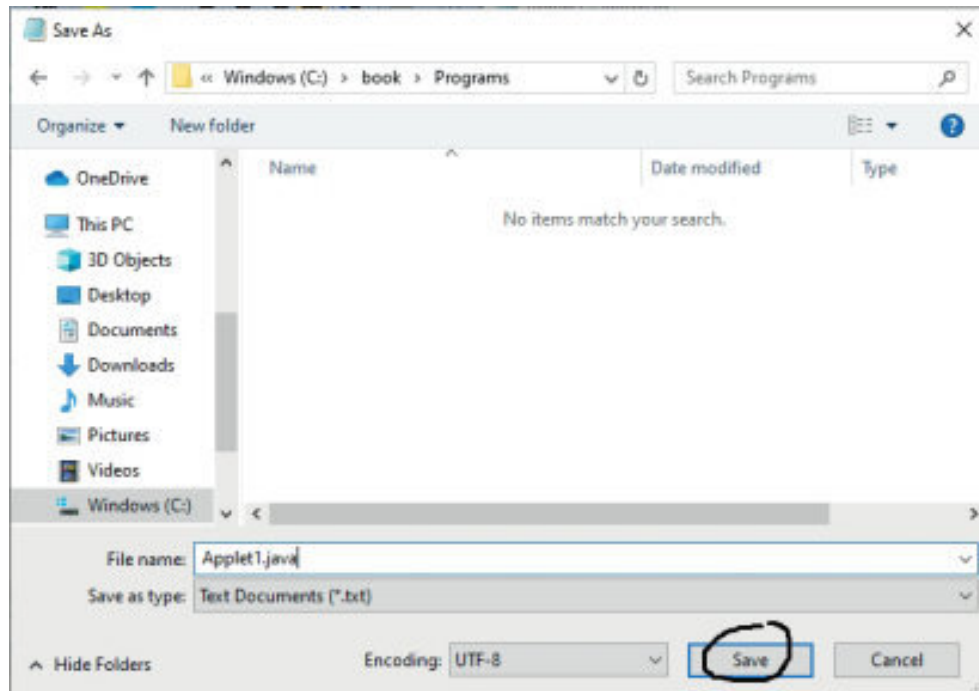
A screenshot of a Notepad window titled "Untitled - Notepad". The window contains the following Java code:

```
File Edit Format View Help
import java.applet.Applet;
import java.awt.Graphics;
public class Applet1 extends Applet
{
    public void paint(Graphics g)
    {
        g.drawString("My First Applet Program",10,40);
    }
}
```

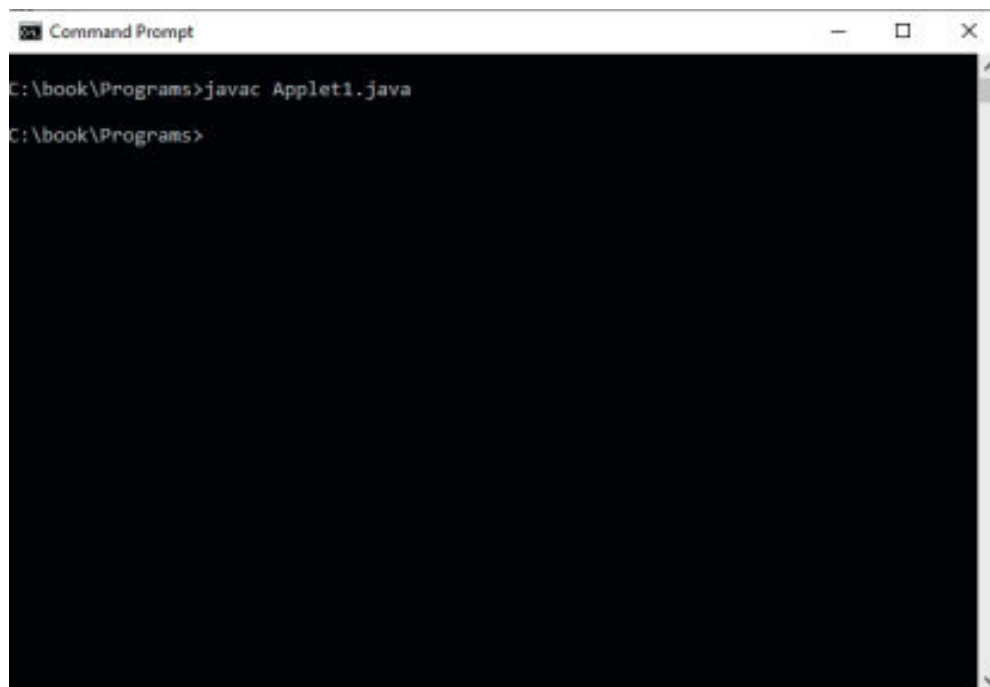
The status bar at the bottom indicates "Ln 9, Col 3", "100%", "Windows (CRLF)", and "UTF-8".

The Graphics class is an abstract class used for drawing on the display area of the applet. In the above program drawString() method of Graphics class is used to display a message in the applet window.

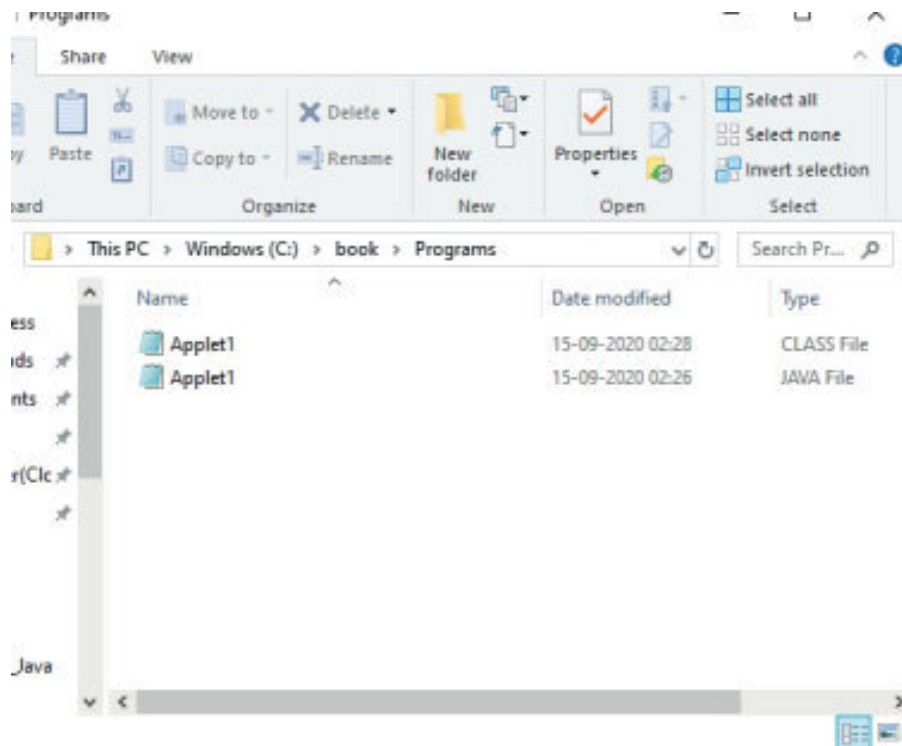
2. Save the program with the class name with .java extension.



3. Then the compile the Applet1.java at the command prompt



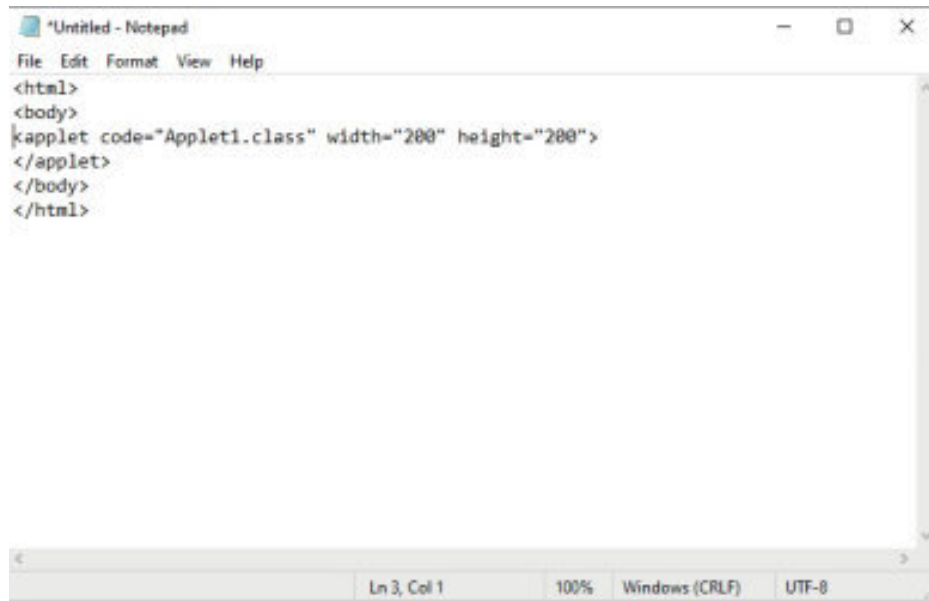
4. Applet1.class is generated after compiling the program.



Applets1.java cannot be run using `java` keyword as this program does not contain any main method.

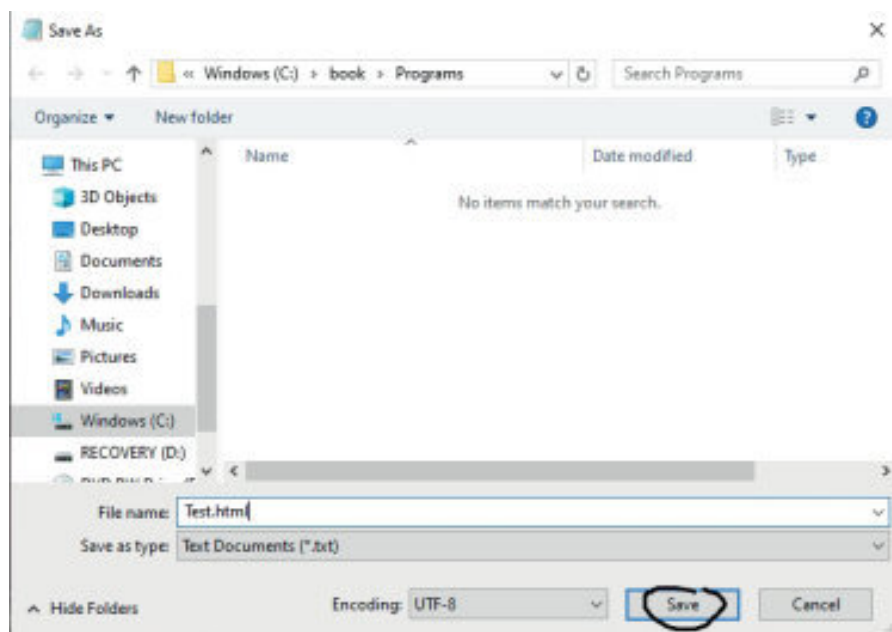
```
Command Prompt
C:\book\Programs>javac Applet1.java
C:\book\Programs>java Applet1
Error: Main method not found in class Applet1, please define the main method as:
  public static void main(String[] args)
or a JavaFX application class must extend javafx.application.Application
C:\book\Programs>
```

5. To run the Applet1.java, we have to create HTML file.

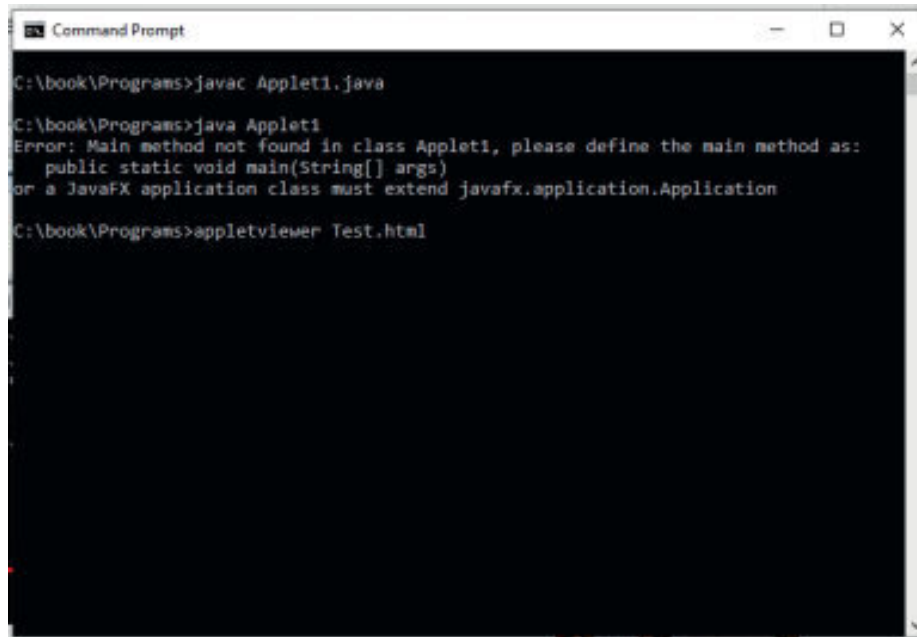


6. Save this file with any name with .html extension.

Here the file is saved as Test.html

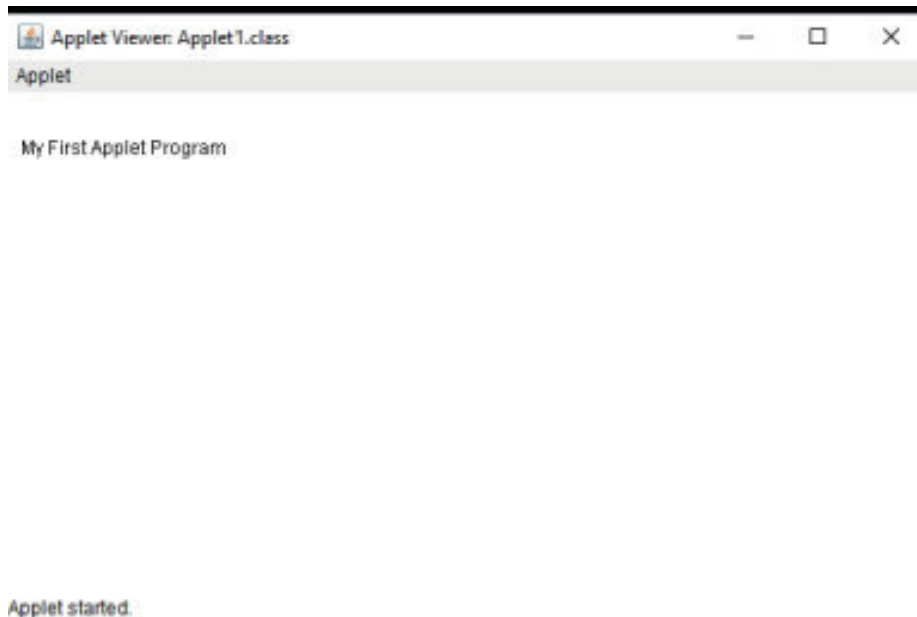


7. To run this Test.html file, appletviewer is used at command prompt.



```
C:\book\Programs>javac Applet1.java
C:\book\Programs>java Applet1
Error: Main method not found in class Applet1, please define the main method as:
    public static void main(String[] args)
or a JavaFX application class must extend javafx.application.Application
C:\book\Programs>appletviewer Test.html
```

8. The Output



Example 190 : To illustrate an Applet program to set font, background and foreground colour.

```
import java.applet.*;
import java.awt.Color;
import java.awt.Graphics;
```

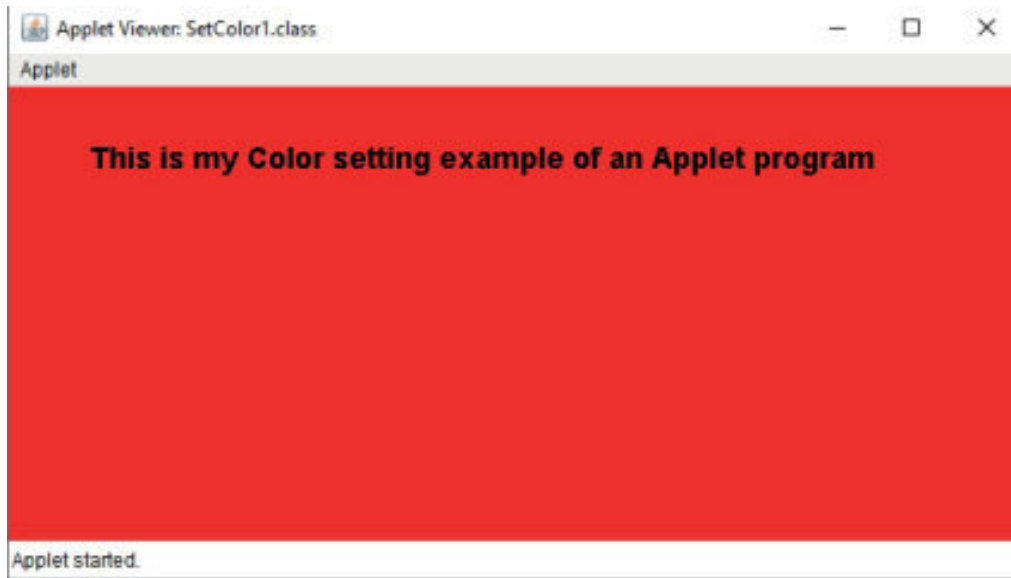
```
import java.awt.Font;

public class SetColor1 extends Applet
{
    Font f1=new Font("Arial", Font.BOLD,18);;
    public void paint(Graphics g)
    {
        setBackground(Color.red);
        setForeground(Color.black);
        g.setFont(f1);
        g.drawString("This is my Color setting example of an Applet
program", 50, 50);
    }
}
```

Html code

```
<html>
<body>
<applet code="SetColor1.class" width="200" height="200">
</applet>
</body>
</html>
```

Output:



Example 191 : To insert image in an Applet

```
import java.applet.*;  
import java.awt.Graphics;  
import java.awt.Image;
```

```
public class InsertImage extends Applet  
{  
    Image pic1;  
    public void init()  
    {  
        pic1 = getImage(getDocumentBase(),"scene.jpg");  
    }  
    public void paint(Graphics g)  
    {  
        g.drawImage(pic1, 100, 50,this);  
    }  
}
```

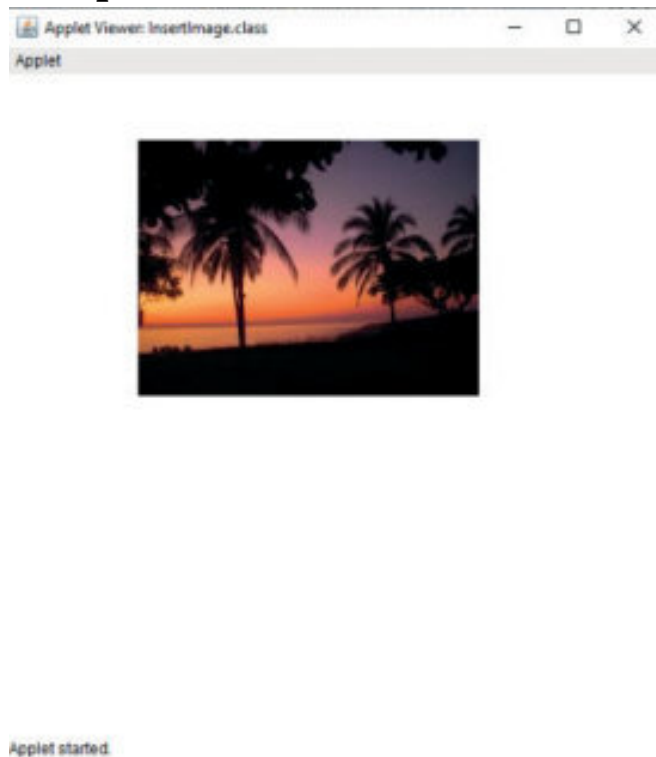
/* image file "scene.jpg" and program files are saved in the same directory


```
*/
```

Html code

```
<html>  
<body>  
<applet code="InsertImage.class" width="500" height="500">  
</applet>  
</body>  
</html>
```

Output:



The `drawImage()` method of `Graphics` class is used to draw an image in an Applet.

The `getImage()` method of `Applet` class is used to load an image.

The `getDocumentBase()` method is used to return the URL of the directory in which the applet is embedded.

Another method `getCodeBase` is used to return the base URL.

Example 192 : An applet program to add two numbers with layout setting.

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
public class AddNum extends Applet
{
    Label l1,l2,l3;
    TextField t1,t2,t3;
    Button b;
    int num1,num2,res;
    public void init()
    {
        l1 = new Label("First Number");
        t1 = new TextField();
        l2 = new Label("Second Number");
        t2 = new TextField(); l3 = new Label("Result");
        t3 = new TextField(10);
        b = new Button("Add");

        b.addActionListener(new add1());

        setLayout(null);
        l1.setBounds(40,50,100,30);
        l2.setBounds(40,100,100,30);
        l3.setBounds(40,150,100,30);
        t1.setBounds(150,50,100,30);
        t2.setBounds(150,100,100,30);
        t3.setBounds(150,150,100,30);
        b.setBounds(150,200,80,50);
```

```

        add(l1);
        add(t1);
        add(l2);
        add(t2);
        add(l3);
        add(t3);
        add(b);
    }
class add1 implements ActionListener
{
    public void actionPerformed(ActionEvent e)
    {
        if(e.getSource() == b)
        {
            num1 = Integer.parseInt(t1.getText());
            num2 = Integer.parseInt(t2.getText());
            res = num1+num2;
            t3.setText(String.valueOf(res));
        }
    }
}
}

```

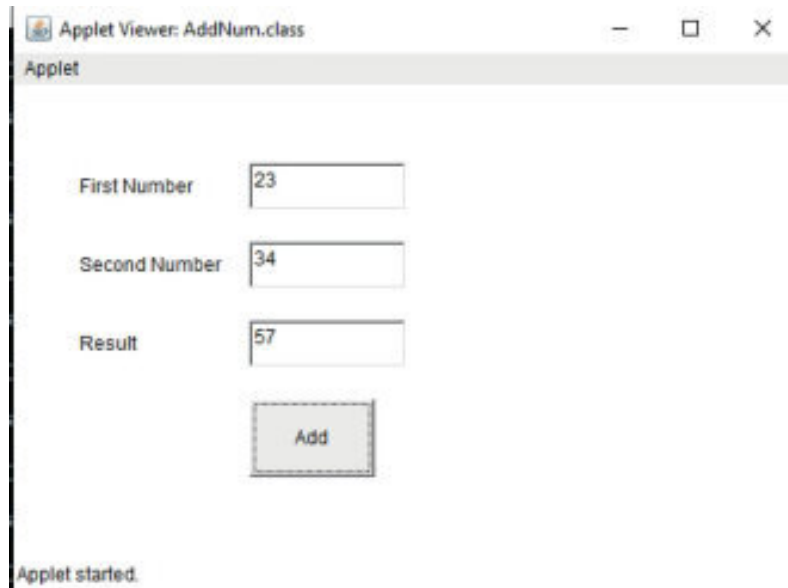
Html code

```

<html>
<body>
<applet code="AddNum.class" width="500" height="300">
</applet>
</body>
</html>

```

Output:



- Java AWR Label is used to display a single line of read only text.
- TextField is an AWT component that allows entering a single line of text in GUI.
- Button is an AWT component that has a label and result in action when clicked
- The setBounds() method is used to set layout of the components.
- The setBounds() methods has four arguments.
- The first and second arguments are x and y coordinates of the top-left corner of the component.
- The third and fourth arguments are width and height of the component.

48.5 Passing Parameters to Applets

The `<param>` tag is used in between `<applet>` and `</applet>` to pass parameters in Applets. The `<param>` tag has two attributes *name* and *value* used to specify the name and value of the parameters.

Syntax:

```
<param name="Subject"
```

```
value = "Java"  
</>
```

getParameter() method of the Applet class can be used to access these parameters.

Syntax:

```
String getParameter(String param-name)
```

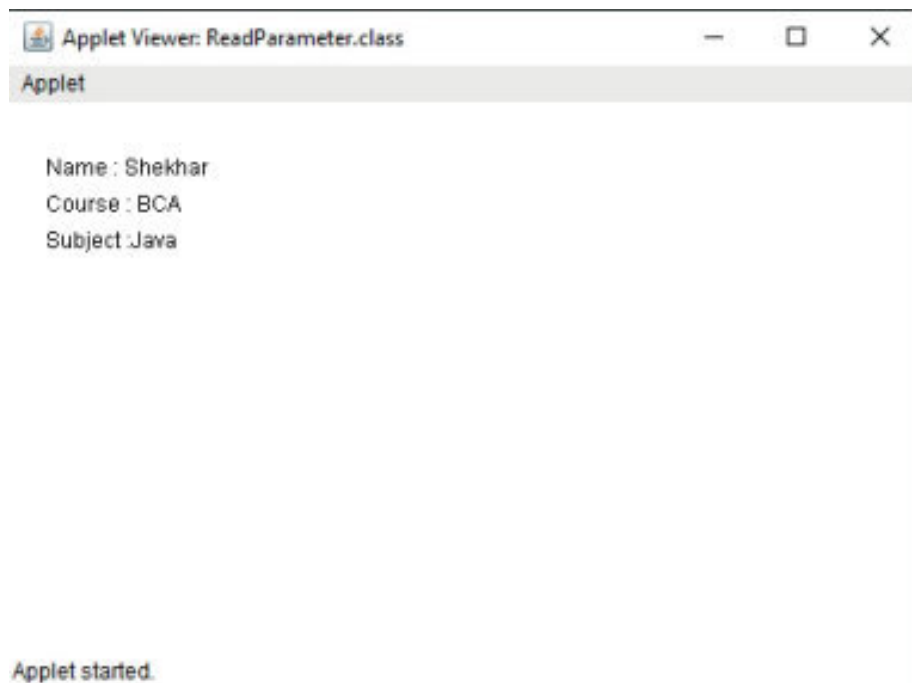
Example 193 : Program to illustrate parameter passing in Applet program.

```
import java.awt.*;  
import java.applet.*;  
  
public class ReadParameter extends Applet  
{  
    String name;  
    String course;  
    String subject;  
  
    public void init()  
    {  
        name=getParameter("Name");  
        course = getParameter("Course");  
        subject = getParameter("Subject");  
    }  
    public void paint(Graphics g)  
    {  
        g.drawString("Name : " +name, 20, 40);  
        g.drawString("Course : " +course, 20, 60);  
        g.drawString("Subject : " +subject, 20, 80);  
    }  
}
```

Html file code

```
<html>
<body>
<applet code="ReadParameter.class" width="500" height="300">
<param name="name" value="Shekhar">
<param name="course" value="BCA">
<param name="subject" value="Java">
</applet>
</body>
</html>
```

Output:



Some real world Programs to practice

EMI calculator

EMI or Equated Monthly Installment is the monthly amount payments towards loan.

EMI can be calculated as

$$\text{EMI} = \frac{P \times r \times (1 + r)^n}{(1+r)^n - 1}$$

Where

- P = Principal Amount
- r = rate of interest per annum
- n = no. of installment

Program 194: To Calculate EMI on a given loan amount

```
import java.util.*;
import java.util.Scanner;
class Calculator
{
    double installment;
    Calculator(double l,double r,double t)
    {
        installment= (l*r*Math.pow(1+r,t))/(Math.pow(1+r,t)-1);
    }
}
class CalculateEMI
{
    public static void main(String []args)
    {
        Scanner sc = new Scanner(System.in);
        double loanamount, rate, time, installment;
        System.out.print("Enter Principal Amount: ");
```

```
loanamount = sc.nextFloat();
System.out.print("Loan term: ");
time = sc.nextFloat();
System.out.print("Rate of Interest: ");
rate = sc.nextFloat();
rate=rate/(12*100);
time=time*12;

Calculator obj= new Calculator(loanamount,rate,time);
System.out.print("Monthly EMI is= "+obj.installment+"\n");
}
}
```

Output:

```
Enter Principal Amount: 75000
Loan term: 3
Rate of Interest: 12.5
Monthly EMI is= 2509.021918253725
```

Encryption

The Caesar Cipher is simplest and most commonly known encryption technique named after Julius Caesar. This technique replaces each letter in the plain text by a letter with some fixed number of positions down the alphabets. For example, with a shift of 7, A becomes H, B becomes I and similarly C is replaced by J.

Program 195 : To encrypt a plain text into Caesar cipher encryption technique

```
import java.util.*;
class CaesarCipher
{
```



```

void encrypt(String w)
{
    char ch;
    int alpha = 0;
    String res = "";
    for(int i = 0; i<w.length(); i++)
    {
        ch = w.charAt(i);
        if(Character.isLetter(ch))
        {
            alpha = ch + 7;
            if((Character.isUpperCase(ch) && alpha>90) || (Character.
isLowerCase(ch) && alpha>122))
            {
                alpha = alpha - 26;
            }
            ch = (char)alpha;
        }
        res = res + ch;
    }
    System.out.println("The Encrypted text is : "+res);
}

public static void main(String args[])
{
    CaesarCipher obj = new CaesarCipher();
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter the text for encryption : ");
    String s = sc.nextLine();
    obj.encrypt(s);
}
}

```

Output:

Enter the text for encryption : My password for login is
JavaProgramming

The Encrypted text is : Tf whzzdvyk mvy svnpu pz
QhchWyvnyhttpun

ISBN number verification

The ISBN(International Standard Book Number) is a unique number to identify a specific book. The ISBN were used to be 10 digits number up to 2006, but from January, 2007, switched to a 13-digit format. To convert 10-digit ISBN to 13-digit format, drop the last character of 10-digit ISBN number and place “978” to the front of it. Calculate new check digit and add it to the end.

Program 196 : To check for valid 10-digit ISBN number.

```
import java.util.Scanner;
class ValidISBN
{
    static boolean isValid(String isbn)
    {
        int l = isbn.length(); // check for length
        if (l != 10)
            return false;
        int sum = 0;
        for (int i = 0; i < 9; i++)
        {
            int num = isbn.charAt(i) - '0'; // first nine digit operation
            if (0 > num || 9 < num)
                return false;
            sum += (num * (10 - i));
        }
    }
}
```

```

    }

    char chk = isbn.charAt(9);
    if (chk != 'X' && (chk < '0' || chk > '9'))
        return false;
    sum += ((chk == 'X') ? 10 : (chk - '0'));
    return (sum % 11 == 0);
}

public static void main(String[] args)
{
    System.out.println("Enter the number : ");
    Scanner sc=new Scanner(System.in);
    String isbn=sc.nextLine();
    if (isValid(isbn))
        System.out.print("The entered ISBN number is Valid");
    else
        System.out.print("The entered ISBN number is not Valid");
}
}

```

Output:

```

Enter the number :
8120311655
The entered ISBN number is Valid

```

Calender

Program 197 : To Print calendar of given month and year.

```

import java.util.Scanner;

```

```

public class Calendar
{
public static void main(String[] args)
{
    Scanner scan = new Scanner (System.in );
    System.out.print("Enter the year : ");
    int year = scan.nextInt();
    System.out.print("Enter month number( 1 - 12) : ");
    int month = scan.nextInt();
    if (month < 1 || month > 12 || year < 1900)
        System.out.println("Invalid Entry");
    else
    {
        System.out.println("\n          " + MName(month) + " " + year);
        System.out.println(" \n Sun Mon Tue Wed Thu Fri Sat");
        getmonth(year, month);
    }
}
static String MName(int month)
{
    String name = null;
    switch (month)
    {
        case 1: name = "January"; break;
        case 2: name = "February"; break;
        case 3: name = "March"; break;
        case 4: name = "April"; break;
        case 5: name = "May"; break;
        case 6: name = "June"; break;
        case 7: name = "July"; break;
        case 8: name = "August"; break;
        case 9: name = "September"; break;
        case 10: name = "October"; break;
    }
}
}

```

```

        case 11: name = "November"; break;
        case 12: name = "December";
    }
    return name;
}

static void getmonth(int year, int month)
{
    int day = firstDay(year, month);
    int numdays = monthDays(year, month);
    int i = 0;
    for (i = 0; i < day; i++)
        System.out.print(" ");
    for (i = 1; i <= numdays; i++)
    {
        if (i < 10)
            System.out.print(" " + i);
        else
            System.out.print(" " + i);
        if ((i + day) % 7 == 0)
            System.out.println();
    }
    System.out.println();
}

static int firstDay(int year, int month)
{
    int f1 = 3;
    int numdays = yearDays(year, month);
    return (numdays + f1) % 7;
}

static int yearDays(int year, int month)
{
    int total = 0;
    for (int i = 1800; i < year; i++)

```

```

        if (isLeapYear(i))
            total = total + 366;
        else
            total = total + 365;
        for (int i = 1; i < month; i++)
            total = total + monthDays(year, i);
        return total;
    }
    static int monthDays(int year, int month)
    {
        if (month == 1 || month == 3 || month == 5 || month == 7 ||
            month == 8 || month == 10 || month == 12)
            return 31;
        if (month == 4 || month == 6 || month == 9 || month == 11)
            return 30;
        if (month == 2) return isLeapYear(year) ? 29 : 28;
        return 0;
    }
    static boolean isLeapYear(int year)
    {
        return year % 400 == 0 || (year % 4 == 0 && year % 100 !=
0);
    }
}

```

Output:

Enter the year : 2020

Enter month number(1 - 12) : 9

September 2020

Sun Mon Tue Wed Thu Fri Sat

1 2 3 4 5

6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

Denomination of Rupees

Program 198: To find the number of denominations for a given amount

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

class DenominationOfRupees
{
    public static void main(String args[])throws IOException
    {
        BufferedReader br=new BufferedReader(new
        InputStreamReader(System.in));
        int Rs[]={1000,500,100,50,20,10,5,2,1};

        System.out.print("\nEnter any Amount : ");
        int amount=Integer.parseInt(br.readLine());
        int copy=amount;
        int total=0,count=0;
        for(int i=0;i<9;i++)
        {
            count=amount/Rs[i];
            if(count!=0)
            {
                System.out.println(Rs[i]+"\\tx\\t"+count+"\\t=
```

```

        "+Rs[i]*count);
    }
    total=total+count;
    amount=amount%Rs[i];
}
System.out.println("\nTOTAL\t\t\t= "+copy);
System.out.println("-----");
System.out.println("Total Number of Notes\t= "+total);
}
}

```

Output:

```

Enter any Amount : 45365
1000   x    45   =45000
100    x    3    =300
50     x    1    =50
10     x    1    =10
5      x    1    =5

TOTAL                                = 45365
-----
Total Number of Notes  = 51

```

Simple interest and Compound interest

Simple interest is calculated on the principle or loan amount.

$$\text{Simple Interest} = \frac{(P \times R \times T)}{100}$$

Compound Interest is computed based on principal amount as well as on accumulated interest of previous periods.

$$\text{Compound Interest} = P(1+R/100)^T - P$$

Where

P=Principle amount

R=Rate of interest per annum

T=Time period

Example 199 : To calculate simple interest and compound interest.

```
import java.util.Scanner;
class CalculateInterest
{
    public static void main(String args[])
    {

        double p,rate,t,si,c,samount=0,camount=0,f=1;
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter principle amount : ");
        p=sc.nextDouble();
        System.out.println("Enter rate of interest : ");
        rate=sc.nextDouble();
        System.out.println("Enter time period");
        t=sc.nextDouble();
        si=(p*rate*t)/100;
        samount=p+si;

        System.out.println("-----");
        System.out.println("Simple Interest =" +si);
        System.out.println("Amount on maturity=" +samount);
        System.out.println("-----");
    }
}
```

```

    rate=(1+rate/100);
    f=calculate(rate,t,f);
    camount=p*f;
    c=camount-p;

    System.out.println("Compount Interest =" +c);
    System.out.println("Amount on maturity=" +camount);
    System.out.println("-----");
}
static double calculate(double r,double t,double a)
{
    if(t<1)
        return a;
    else
    {
        a*=r;
        return calculate(r,t-1,a);
    }
}
}

```

Output:

```

Enter principle amount :
5000
Enter rate of interest :
5
Enter time period 2
-----
Simple Interest =500.0
Amount on maturity=5500.0
-----
Compount Interest =512.5
Amount on maturity=5512.5

```

Depreciation

The monetary value of an asset decreases over time. The reduction in the value of an asset is measured as depreciation.

$$\text{Depreciation} = P(1 - i)^t$$

Where

P= Original rate

i= depreciation percentage

t= time period

Example 200: To calculate Depreciation

```
import java.util.Scanner;
class Calculate
{
    double p,i,t,amount,temp;
    Calculate(double p,double i,double t )
    {
        amount=p;
        for(int j=0;j<t;j++)
            amount=((100-i)*amount)/100;
    }
}

class CDepreciation
{
    public static void main(String arg[])
    {
        double p,i,t,amount,temp;
```

```

Scanner sc=new Scanner(System.in);
System.out.print("Enter the original price : ");
p=sc.nextLong();
System.out.print("Enter depreciation percentage : ");
i=sc.nextLong();
System.out.print("Enter the time period : ");
t=sc.nextLong();
Calculate d=new Calculate(p,i,t);
System.out.println("Rate after depreciation = "+d.amount);
}
}

```

Output:

```

Enter the original price : 900000
Enter depreciation percentage : 20
Enter the time period : 4
Rate after depreciation = 368640.0

```

Most frequently asked Java questions

1. How Java is different from C++ ?

The difference between Java and C++ includes

C++	Java
Developed by Bjarne Stroustrup at Bell Labs.	Developed by James Gosling at Sun Microsystems.
Platform dependent	Platform Independent
Compiled Language	Both Compiled and interpreted language
Supports both single and multiple inheritance	Support Single inheritance, multiple inheritance achieved by using interfaces.

Does not support multi-threading	Supports multi-threading
Does not support garbage collection	Supports garbage collection
Supports destructors	Does not support destructor
Support concept of pointer	Does not support pointer

2. Why Java is called as platform independent language?

Java is known as platform-independent language because it supports the concept of WORA i.e. “Write once, run anywhere”. Java programs are compiled by the compiler it generates bytecode. This bytecode is platform independent and runs on any system irrespective of their operating system.

3. What is the concept of Object and Class in java?

Refer pg-6

4. Explain JDK, JRE and JVM

Refer pg-2

5. Explain about Access modifier in java

Refer pg-131

6. What are the differences between Method Overloading and Method Overriding?

Differences between Method Overloading and Overriding are:

Mehod Overloading	Method Overriding
More than one method with same name, different prototype in same scope.	More than one method with same name, same prototype in different scope.
It is done within class	It occurs between two classes.
Return type may or may not be same	Return type must be same
Parameter must be different	Parameter must be same
Static methods can be overloaded	Static methods cannot be overridden.
Compile time polymorphism	Run time polymorphism

7. List the features of Java Programming.

Refer pg-2,3

8. What are differences between Java Constructors and Java Methods?

Differences between Java Constructors and Java Methods includes

Java Methods	Java Constructors
Must have a return type	Must not have return type
Invoked explicitly	Invoked explicitly
Not support concept of default methods	Supports default constructor
Any name other than keywords	Must have the same name as the class name

9. What is inheritance?

Refer pg-75

10. What is the difference between Compile time polymorphism and Runtime polymorphism

Difference between compile time polymorphism and runtime polymorphism

Compile time polymorphism	Runtime polymorphism
Occurs at compile time	Occurs at runtime
Known as Static binding and early binding	Known as dynamic binding and late binding
Fast execution	Slower execution
Less flexible	More flexible
Implemented through method overloading	Implemented through method overriding.

Index

- Java [1](#)
 - Introduction [1](#)
 - components [2](#)
 - Features [2](#)
 - java virtual machine(jvm) [2](#)
 - java runtime environment(jre) [2](#)
 - java development kit(jdk) [2](#)
 - Installation [3](#)
 - path setting [4](#)
 - objects [5](#)
 - classes [5](#)
 - Keywords [6](#)
 - main() system.out.println() [7](#)
 - println() [7](#)
 - java comments [7](#)
 - single line comment [7](#) *to be made bold*
 - multi line comment [7](#) *to be made bold*
 - program execution [8](#)
 - variables [8](#)
 - data types [10](#)
 - primitive data type [10](#)
 - byte [11](#)
 - short [11](#)

- int [11](#)
- long [12](#)
- float [12](#)
- double [12](#)
- char [13](#)
- boolean [13](#)
- non primitive data type [13](#)
- type casting [13](#)
 - widening/implicit type casting [14](#)
 - narrowing/explicit type casting [14](#)
- operators [15](#)
 - arithmetic operators [15](#)
- import [16](#)
- java.lang [16](#)
- scanner [16](#)
- increment operators [18](#)
- decrement operators [18](#)
- relational operators [19](#)
- comparison operators [19](#)
- logical operators [19](#)
- assignment operators [20](#)
- conditional operators [21](#)
- ternary operators [21](#)
- bitwise operators [23](#)
- shift operators [24](#)
- instance of operator [24](#)
- operator precedence and associativity [25](#)
- control flow statement [25](#)
 - decision making [26](#)
 - simple if statement [26](#)
 - if-else statement [26](#)
 - nested-if-else statement [27](#)
 - if-else-if ladder [28](#)

- switch statement [30](#)
- looping statement [36](#)
- for loop [36](#)
- while loop [48](#)
- do-while-loop [51](#)
- jump statement [53](#)
- break [53](#)
- continue [53](#)
- declaration of methods [55](#)
- methods [55](#)
 - built in methods [56](#)
 - user defined methods [58](#)
- method parameter [59](#)
- method argument [59](#)
- instance method [63](#)
- constructor [63](#)
- this keyword [65](#) *to be bold*
- polymorphism [65](#)
- method overloading [66](#)
- type promotion [66](#)
- inheritance [67](#)
 - single inheritance [68](#)
 - multilevel inheritance [69](#)
 - hierarchichal inheritance [70](#)
 - multiple inheritance [71](#)
- method overriding [71](#)
- super keyword [72](#)
- final [77](#)
- abstract class [79](#)
- abstract method [80](#)
- interface [80](#)
- arrays [82](#)
 - single dimensional array [82](#)

- multi dimensional array [92](#)
- arrayList [96](#)
 - arrayList methods [97](#)
- iterator [100](#)
- wrapper class [107](#)
- strings [108](#)
 - strings methods [113](#)
- packages [114](#)
 - API package [114](#)
 - user defined package [115](#)
- access modifier in java [117](#)
 - private [117](#)
 - default [118](#)
 - protected [119](#)
 - public [120](#)
- exception handling [121](#)
 - checked exception [121](#)
 - unchecked exception [121](#)
- try catch keyword [122](#)
- finally block [125](#)
- custom exception [127](#)
- user defined exception [127](#)
- throws [127](#)
- thread [128](#)
 - life cycle of a thread [128](#)
 - creating a thread [129](#)
 - setting priorities [132](#)
 - methods of Thread class [133](#)
- input/output [138](#)
- stream classes [138](#)
- byte stream class [139](#)
- character stream class [145](#)
- input stream [139](#)

- output stream [143](#)
- reader stream class [145](#)
- writer stream class [146](#)
- random access files [147](#)
- file class [149](#)
- database management [154](#)
- java database connectivity (jdbc) [154](#)
- odbc [154](#)
- java AWT [157](#)
 - awt components [157](#)
- types of containers [158](#)
- event handling [158](#)
- java application and java applet [161](#)
- applet programming [161](#)
 - life cycle of an applet [161](#)
 - applet program execution [163](#)
 - passing parameters to applets [169](#)